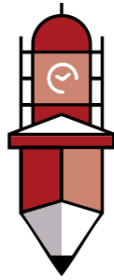


The University of Faisalabad

Department of Computer Science



The
University of
Faisalabad

Name: Humna Imran

Registration Number: 2023-BS-AI-017

Instructor: Noorhan Yasin

Computer Organization and Assembly Language

CS-318 Lab

BS Artificial Intelligence

Table of Content

Lab 1.....	4
Lab 2.....	22
Lab 3.....	27
Lab 4.....	41
Lab 5.....	52
Lab 6.....	62
Lab 7.....	69
Lab 8.....	73
Lab 9.....	78
Lab 10	84
Lab 11	90
Lab 12	93
Lab 13	99
Lab 14	105
Lab 15	112

MIDS

LAB 1

Conversions

Objective

This lab provides a foundational understanding of how data is represented and processed within a computer system. You will learn the principles of number systems, including decimal, binary, octal, and hexadecimal, and perform conversions between them to understand how high-level data is executed by hardware.

Number System Architecture

Number systems are the primary tools used by computers to represent and process data. While humans commonly use the decimal system, internal computer logic operates on binary states.

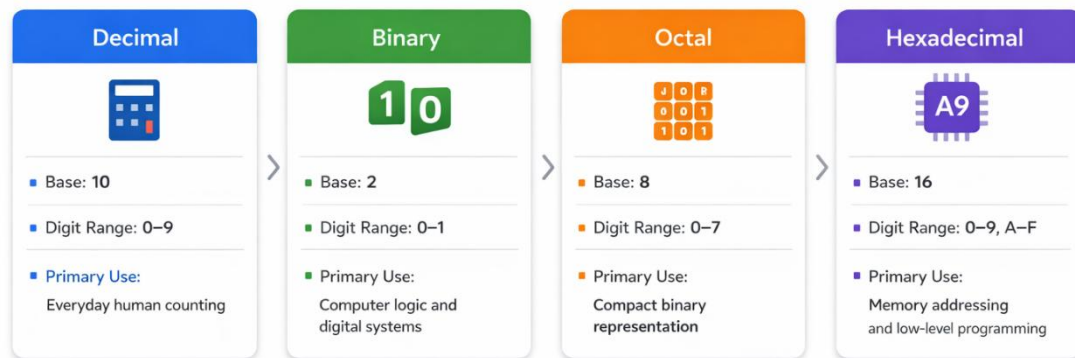


Figure 1: Number System

In this lab, you will utilize the following methods to perform number system conversions:

Decimal to Other Bases:

- Binary: Repeatedly divide the decimal value by 2 and record the remainders from bottom to top.
- Octal: Repeatedly divide by 8, recording remainders from bottom to top.
- Hexadecimal: Repeatedly divide by 16, replacing remainders 10-15 with letters A-F.

Binary to Decimal:

Multiply each bit by 2 raised to the power of its position (starting from 0 on the right) and sum the results.

Binary to Hexadecimal:

Group the binary bits into sets of four starting from the right and convert each group to its hexadecimal equivalent.

Hexadecimal to Binary:

Directly convert each individual hexadecimal digit into its corresponding 4-bit binary equivalent.

Task 1: Decimal Conversions

Objective: Convert Decimal (base-10) numbers into Binary (base-2), Octal (base-8), and Hexadecimal (base-16) formats.

Question 1: Convert Decimal 25

1. 25		Octal	
Binary			
2	25	8	25
2	12 - 1		3 - 1
2	6 - 0		
1	1 - 1		
	(11001)		= (31)
= (11001)			
Hexadecimal			
16	25		
	1 - 9		
= (19)			

Figure 2: The image displays handwritten calculations for converting the number 25. It shows three distinct division tables: division by 2 for Binary resulting in 11001, division by 8 for Octal resulting in 31, and division by 16 for Hexadecimal resulting in 19.

Procedural Steps:

- **To Binary:** Divide 25 by 2. Record the quotient and the remainder (0 or 1). Continue dividing the quotient by 2 until you reach 0. Read the remainders from bottom to top to get the binary result (11001).
- **To Octal:** Divide 25 by 8. The result is 3 with a remainder of 1. Read from the quotient to the remainder to get the octal result (31).
- **To Hexadecimal:** Divide 25 by 16. The result is 1 with a remainder of 9. Combine them to get the hexadecimal result (19).

Question 2: Convert Decimal 73

75

Binary

2	73
2	36 - 1
2	18 - 0
2	9 - 0
2	4 - 1
2	2 - 0
	1 - 0

Octal

8	73
8	9 - 1
	1 - 1

Hexadecimal

16	73
	4 - 9

Results:

Binary: $= (1001001)$

Octal: $= (111)$

Hexadecimal: $= (49)$

Figure 3: The image shows the manual division process for the number 73. The binary section lists repeated divisions by 2 yielding 1001001. The octal section shows division by 8 yielding 111. The hexadecimal section shows division by 16 yielding 49.

Procedural Steps:

- **To Binary:** Repeatedly divide 73 by 2. Note the remainders: 1, 0, 0, 1, 0, 0, 1. Arranging these from the last remainder to the first gives 1001001.
- **To Octal:** Divide 73 by 8. You get 9 with a remainder of 1. Divide 9 by 8 to get 1 with a remainder of 1. The result is 111.
- **To Hexadecimal:** Divide 73 by 16. You get 4 with a remainder of 9. The result is 49.

Question 3: Convert Decimal 255

255

Binary

2	255
2	127 - 1
2	63 - 1
2	31 - 1
2	15 - 1
2	7 - 1
1	1 - 1

(11111111)

Octal

8	255
3	31 - 7
	3 - 7

= (377)

Hexadecimal

16	255
	15 - 1F

= (FE) ∴ 15 = F

Figure 4: This image illustrates the conversion of the maximum 8-bit value, 255. The binary division produces eight consecutive 1s. The octal division yields 377. The hexadecimal division results in 15 with a remainder of 15, which are converted to letters F

Procedural Steps:

- **To Binary:** Divide 255 by 2 repeatedly. Since the number is odd at every step, every remainder is 1. The final sequence is 11111111.
- **To Octal:** Divide 255 by 8 to get 31 remainder 7. Divide 31 by 8 to get 3 remainder 7. The result is 377.
- **To Hexadecimal:** Divide 255 by 16. The result is 15 with a remainder of 15. In Hexadecimal, the number 15 is represented by the letter F. Therefore, the result is FF.

Question 4: Convert Decimal 1023

4. 1023

Binary	Octal
2 1023	8 1023
2 511 - 1	8 127 - 7
2 255 - 1	8 15 - 7
2 127 - 1	1
2 63 - 1	
2 31 - 1	= (1777)
2 15 - 1	
2 7 - 1	
2 1 - 1	

= (11111111)

Hexadecimal

16 1023
16 63 - 15
3 - 15

.. 15 = F

= (3FE)

Figure 5: The image depicts the conversion of the number 1023. The binary calculation shows a long string of ten 1s. The octal calculation results in 1777. The hexadecimal calculation results in 3FE, where the remainder 15 is noted as F and remainder 14 is E.

Procedural Steps:

- **To Binary:** Divide 1023 by 2 repeatedly. Continue until the quotient is 0. The result is ten consecutive 1s: 1111111111.
- **To Octal:** Divide 1023 by 8. Continue dividing the quotients by 8. The remainders read from bottom to top are 1, 7, 7, 7.
- **To Hexadecimal:** Divide 1023 by 16 to get 63 with a remainder of 15 (F). Divide 63 by 16 to get 3 with a remainder of 15 (F). *Correction based on standard math:* 1023 divided by 16 is 63 remainder 15 (F). 63 divided by 16 is 3 remainder 15 (F). Result: 3FF. (Note: The handwritten image shows "3FE" but the math for 1023 should result in 3FF. Students should verify this calculation).

Task 2: Binary Conversions

Objective: Convert Binary (base-2) values into Decimal (base-10) and Hexadecimal (base-16).

Question 1: Convert Binary 101101

1. 101101

Decimal

$$= 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$= 1 \times 32 + 0 \times 16 + 1 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1$$

$$= 32 + 0 + 8 + 4 + 0 + 1$$

$$= 45$$

Hexadecimal

Group from right in 4 bits → 0010 1101

$$0010 = 2$$

$$1101 = D$$

$$= 2D$$

Figure 6: The image shows the calculation for binary 101101. It uses the positional weight method (multiplying bits by powers of 2) to get Decimal 45. For Hexadecimal, it groups the bits into 0010 and 1101, converting them to 2 and D resulting in 2D.

Procedural Steps:

- **To Decimal:** Assign powers of 2 starting from the right (1, 2, 4, 8, 16, 32). Multiply the bit by its weight: $32 + 0 + 8 + 4 + 0 + 1 = 45$.
- **To Hexadecimal:** Split the binary number into groups of 4 starting from the right: 0010 and 1101. Convert 0010 to 2. Convert 1101 to 13, which is D. The result is 2D.

Question 2: Convert Binary 11110000

$$\begin{aligned}
 & (111110000)_2 \\
 & \text{Decimal} \\
 & = (1 \times 2^7) + (1 \times 2^6) + (1 \times 2^5) + (1 \times 2^4) + \\
 & \quad (1 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (1 \times 2^0) \\
 & = 128 + 64 + 32 + 16 + 0 + 0 + 0 + 0 = (240)_{10} \\
 & \text{Hexadecimal} \\
 & \begin{array}{cccc} 8 & 4 & 2 & 1 \end{array} \quad \begin{array}{cccc} 8 & 4 & 2 & 1 \\ 1 & 1 & 1 & 1 \end{array} \quad \begin{array}{cccc} 8 & 4 & 2 & 1 \\ 0 & 0 & 0 & 0 \end{array} \\
 & = 15 \\
 & = (F)_{16}
 \end{aligned}$$

Figure 7: The image displays the binary number 11110000. The decimal calculation sums the weights 128, 64, 32, and 16 to equal 240. The hexadecimal conversion groups the first four 1s (F) and the last four 0s (0) to get F0.

Procedural Steps:

- **To Decimal:** Add the weights for the active bits (the 1s): $128 + 64 + 32 + 16$. The sum is 240.
- **To Hexadecimal:** Group into 1111 and 0000. 1111 equals 15 (F). 0000 equals 0. The result is F0.

Question 3: Convert Binary 10011011

3. 10011011

Decimal

$$\begin{aligned}
 & \cdot 1 \times 2^7 + 0 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 \\
 & = 1 \times 128 + 0 \times 64 + 0 \times 32 + 1 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 \\
 & = 128 + 16 + 8 + 2 + 1 \\
 & = 155
 \end{aligned}$$

Hexadecimal

Groups: 1001 1011

1001 = 9

1011 = B

= 9B

Figure 8: The image shows the conversion of 10011011. The decimal sum is calculated as $128 + 16 + 8 + 2 + 1 = 155$. For Hexadecimal, the groups are 1001 (9) and 1011 (B), resulting in 9B.

Procedural Steps:

- **To Decimal:** Multiply each "1" by its position value: $128 + 16 + 8 + 2 + 1$. The total is 155.
- **To Hexadecimal:** Group bits into 1001 and 1011. Convert 1001 to 9. Convert 1011 to 11 (B). The result is 9B.

Question 4: Convert Binary 110011001

$$\begin{aligned}
 & \text{(iv) } (110011001)_2 \\
 & \text{Decimal} \\
 & = (1 \times 2^8) + (1 \times 2^7) + (0 \times 2^6) + (0 \times 2^5) \\
 & \quad (1 \times 2^4) + (1 \times 2^3) + (0 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) \\
 & = 256 + 128 + 0 + 0 + 16 + 8 + 0 + 0 + 1 \\
 & = (409)_{10} \\
 & \text{Hexadecimal} \\
 & \begin{array}{cccccccccccc}
 8 & 4 & 2 & 1 & 8 & 4 & 2 & 8 & 4 & 2 & 1 \\
 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 0 & 1
 \end{array} \\
 & = (199)_{16}
 \end{aligned}$$

Figure 9: The image details the conversion of a 9-bit number, 110011001. The decimal calculation sums $256 + 128 + 16 + 8 + 1$ to equal 409. The hexadecimal conversion pads the front with zeros to make groups of four, resulting in 1, 9, 9.

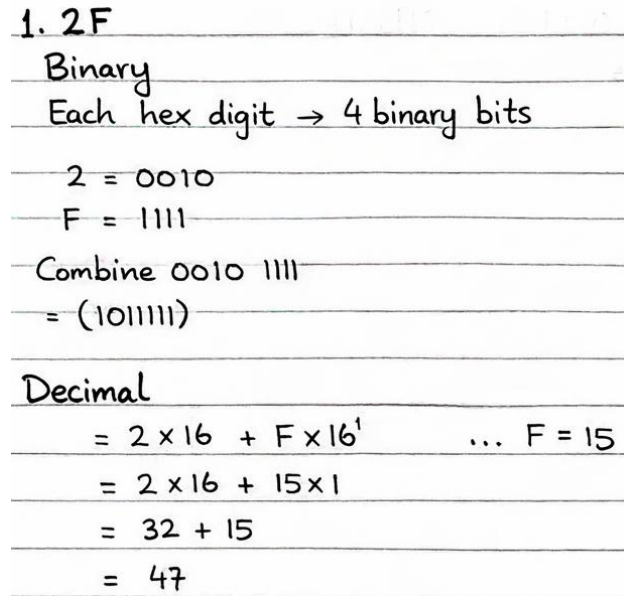
Procedural Steps:

- **To Decimal:** Identify weights for the 1s: 256, 128, 16, 8, and 1. Sum them up to get 409.
- **To Hexadecimal:** Group into 4-bit chunks from right to left: 1001 (9), 1001 (9), and 0001 (1). The result is 199.

Task 3: Hexadecimal Conversions

Objective: Convert Hexadecimal numbers into Binary and Decimal.

Question 1: Convert Hexadecimal 2F



1. 2F

Binary

Each hex digit $\rightarrow$ 4 binary bits

$2 = 0010$

$F = 1111$

Combine 0010 1111

$= (1011111)$

Decimal

$= 2 \times 16 + F \times 16^1 \quad \dots F = 15$

$= 2 \times 16 + 15 \times 1$

$= 32 + 15$

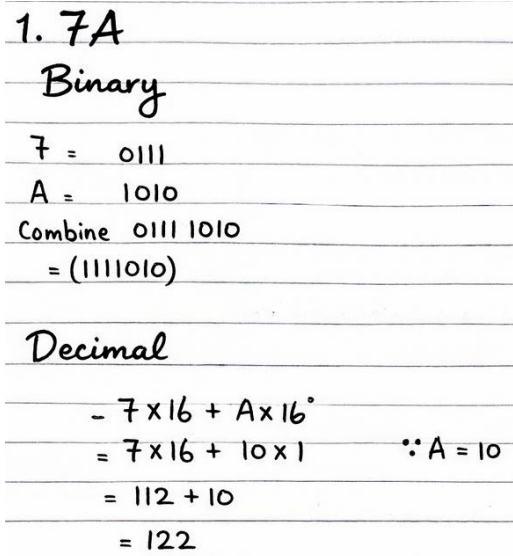
$= 47$

Figure 10: The image shows the breakdown of Hex digits 2 and F. 2 is converted to 0010 and F to 1111. The decimal calculation multiplies 2 by 16 and adds 15 (F) to get 47.

Procedural Steps:

- **To Binary:** Convert each hex digit to 4 binary bits. 2 becomes 0010. F becomes 1111. Combine them to get 00101111.
- **To Decimal:** Multiply the first digit (2) by 16. Add the value of the second digit (F = 15). Calculation: $32 + 15 = 47$.

Question 2: Convert Hexadecimal 7A



1. 7A

Binary

$$7 = 0111$$
$$A = 1010$$

Combine 0111 1010

$$= (1111010)$$

Decimal

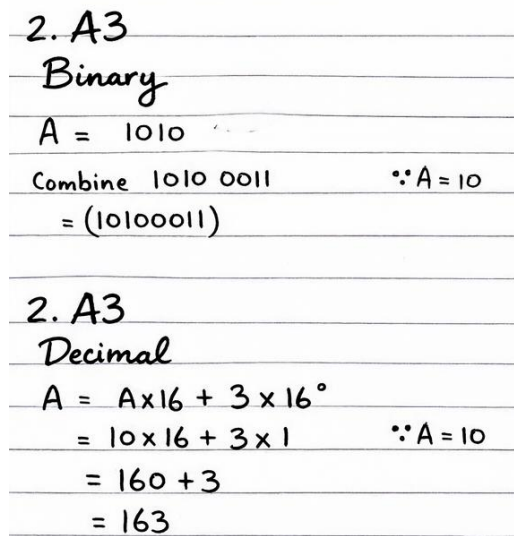
$$= 7 \times 16 + A \times 1$$
$$= 7 \times 16 + 10 \times 1 \quad \because A = 10$$
$$= 112 + 10$$
$$= 122$$

Figure 11: The image lists 7 converted to 0111 and A converted to 1010 for binary. For decimal, it shows 7 multiplied by 16 plus 10 (value of A) resulting in 122.

Procedural Steps:

- **To Binary:** Convert 7 to 0111. Convert A to 1010. Combine to get 01111010.
- **To Decimal:** Calculate $(7 * 16) + 10$. This equals $112 + 10$, which is 122.

Question 3: Convert Hexadecimal A3



2. A3
Binary
 $A = 1010$
Combine $1010\ 0011$ $\because A = 10$
 $= (10100011)$

2. A3
Decimal
 $A = A \times 16 + 3 \times 16^0$
 $= 10 \times 16 + 3 \times 1$ $\because A = 10$
 $= 160 + 3$
 $= 163$

Figure 12: The image shows A (10) converted to 1010 and 3 to 0011 for binary. The decimal calculation is 10 multiplied by 16 plus 3, totaling 163.

Procedural Steps:

- **To Binary:** Convert A to 1010. Convert 3 to 0011. Combine to get 10100011.
- **To Decimal:** Multiply A (10) by 16 to get 160. Add 3. The total is 163.

Question 4: Convert Hexadecimal FF

4. FF

Binary

F = 1111

F = 1111

Combine $\rightarrow$ 1111 1111

= (11111111)

Decimal

= $F \times 16^1 + F \times 16^0$

= $15 \times 16 + 15 \times 1$ " F = 15

= $240 + 15$

= 255

Figure 13: The image shows two Fs, each converted to 1111 in binary. The decimal calculation is 15 multiplied by 16 plus 15, totaling 255.

Procedural Steps:

- **To Binary:** Convert F to 1111. Combine two Fs to get 11111111.
- **To Decimal:** Calculate $(15 * 16) + 15$. This equals $240 + 15$, which is 255.

Task 4: Octal Conversions

Objective: Convert Octal (base-8) numbers into Binary and Decimal.

Question 1: Convert Octal 57

1. 57

Binary

Each octal digits $\rightarrow$ 3 binary bits

$$5 = 101$$

$$7 = 111$$

combine 101 111

$$= 101111$$

Decimal

$$= 5 \times 8 + 7 \times 1$$

$$= 5 \times 8 + 7 \times 1$$

$$= 40 + 7$$

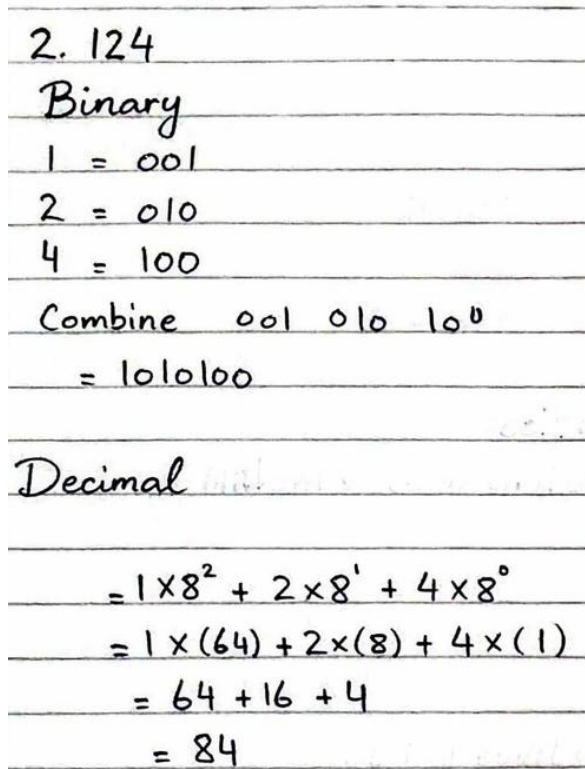
$$= 47$$

Figure 14: The image displays the octal digits 5 and 7 converted to 3-bit binary segments (101 and 111). The decimal calculation sums $(5 * 8) + 7$ to get 47.

Procedural Steps:

- **To Binary:** Convert each octal digit to 3 binary bits. 5 is 101. 7 is 111. Combine them to get 101111.
- **To Decimal:** Multiply the first digit (5) by 8. Add the second digit (7). Calculation: $40 + 7 = 47$.

Question 2: Convert Octal 124



2. 124

Binary

$$1 = 001$$
$$2 = 010$$
$$4 = 100$$

Combine 001 010 100

$$= 1010100$$

Decimal

$$= 1 \times 8^2 + 2 \times 8^1 + 4 \times 8^0$$
$$= 1 \times (64) + 2 \times (8) + 4 \times (1)$$
$$= 64 + 16 + 4$$
$$= 84$$

Figure 15: The image shows the breakdown of 1, 2, and 4 into binary 001, 010, and 100. The decimal calculation sums $(1 * 64) + (2 * 8) + 4$ to get 84.

Procedural Steps:

- **To Binary:** Convert 1 to 001, 2 to 010, and 4 to 100. Combine to get 001010100.
- **To Decimal:** Multiply the first digit by 64, the second by 8, and the third by 1. Calculation: $64 + 16 + 4 = 84$.

Question 3: Convert Octal 377

$$\begin{array}{r}
 (124)_8 \rightarrow \text{Binary} \\
 \begin{array}{ccc}
 1 & 2 & 4 \\
 001 & 421 & 100 \\
 010 & 010 & 100 \\
 \hline
 = (0010100)_2
 \end{array} \\
 \text{Decimal} \\
 = 1 \times 8^2 + 2 \times 8^1 + 4 \times 8^0 \\
 = 64 + 16 + 4 \\
 = (84)_{10} = (124)_{10}
 \end{array}$$

Figure 16: Partial calculations are visible in the image. It lists 3 converted to 011 and 7 converted to 111.

Procedural Steps:

- **To Binary:** Convert 3 to 011. Convert 7 to 111. Convert the second 7 to 111. Combine to get 011111111.
- **To Decimal:** Calculate $(3 * 64) + (7 * 8) + 7$. This equals $192 + 56 + 7 = 255$.

Question 4: Convert Octal 645

(iv) 645

Binary

6	4	5
4 2 1	4 2 1	4 2 1
1 1 0	1 0 0	1 0 1

$= (10100101)_2$

Decimal

$$= 6 \times 8^2 + 4 \times 8^1 + 5 \times 8^0$$

$$= 384 + 32 + 5 = (421)_{10}$$

Figure 17: The image shows 6, 4, and 5 converted to binary 110, 100, and 101. The decimal conversion sums $(6 * 64) + (4 * 8) + 5$ to equal 421.

Procedural Steps:

- **To Binary:** Convert 6 to 110. Convert 4 to 100. Convert 5 to 101. Combine to get 110100101.
- **To Decimal:** Calculate $(6 * 64) + (4 * 8) + 5$. Calculation: $384 + 32 + 5 = 421$.

LAB 2

Introduction

This lab is focused on setting up your digital workspace by installing the **EMU8086** software. This program is an "emulator," which means it acts like a virtual 8086 microprocessor inside your computer, allowing you to write and test Assembly language code without needing old hardware.

Here is a simple breakdown of what we will do in this lab:

- **Step 1: Language Selection** – You will start by choosing the language for the installation process to ensure you can follow the instructions easily.
- **Step 2: Launching the Setup** – You will open the installer (the download manager) to begin copying the software files to your computer.
- **Step 3: Following the Welcome Screen** – You will click through the "Welcome" prompts, which confirm that the installation is ready to finish.
- **Step 4: Creating Your First File** – Once the software is installed, you will open the "Welcome Dashboard." This is the home screen where you click "New" to start a fresh project.
- **Step 5: Choosing a Code Template** – Instead of writing everything from scratch, you will learn to select a "template". This is like a pre-filled form that gives your program a basic structure to help you start coding faster.
- **Step 6: Accessing the Code Editor** – Finally, you will reach the **IDE (Integrated Development Environment)**. This is the main screen where you will type your instructions, see your results, and manage your registers in future labs.

Installation

Step By Step Installation of EMU8086

Step 1: Select Language

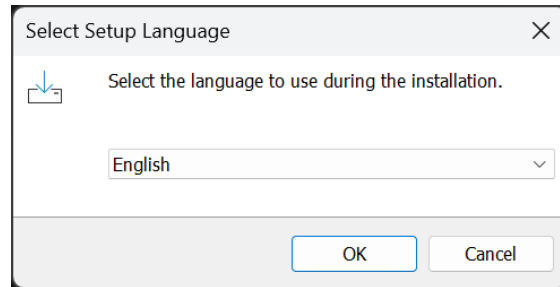


Figure 18: EMU8086 Setup Language Selection Window

Step 2: Launch

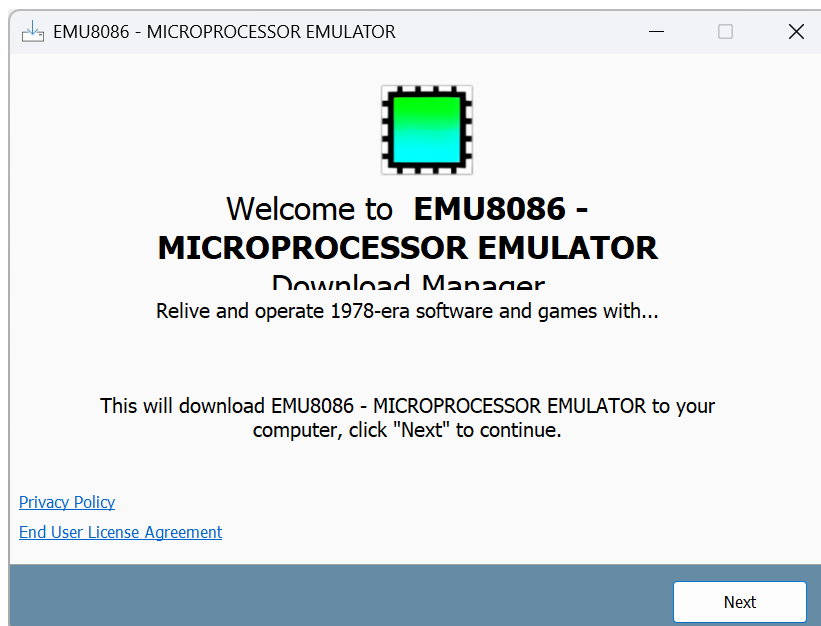


Figure 19: EMU8086 Download Manager Launch Screen

Step 3: Welcome Screen

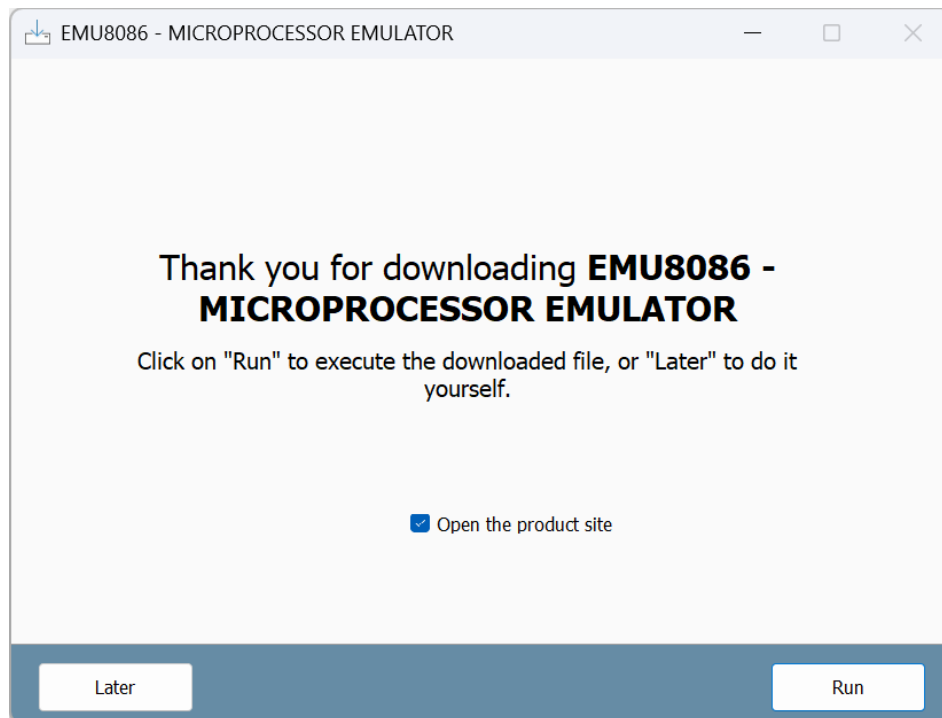


Figure 20: EMU8086 Installation Completion and Execution Prompt

Step 4: Start with creating new File

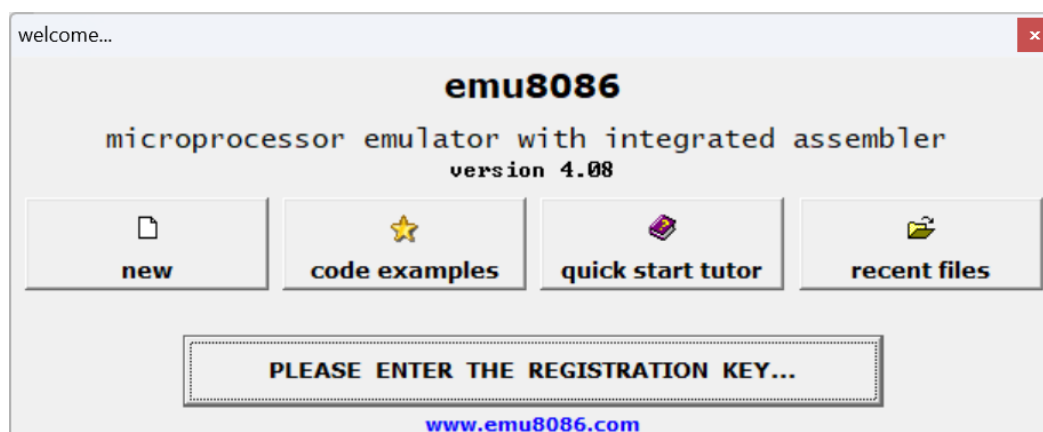


Figure 21: EMU8086 Microprocessor Emulator Welcome Dashboard

Step 5: Choose Code Template

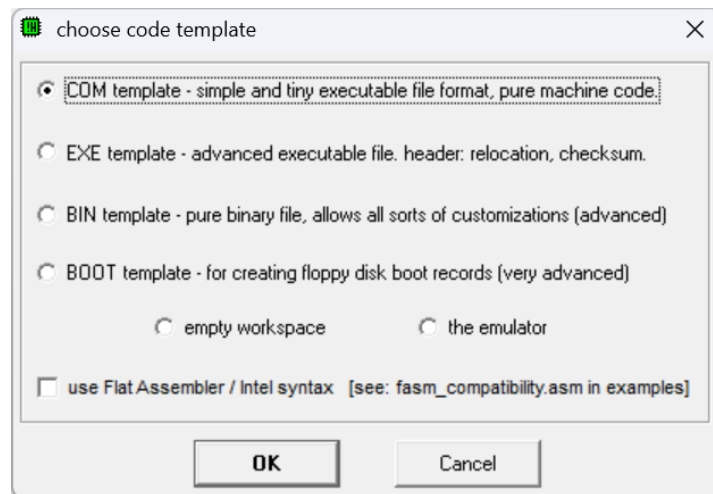


Figure 22: EMU8086 Code Template Selection Interface

Step 6: Code Editor Screen

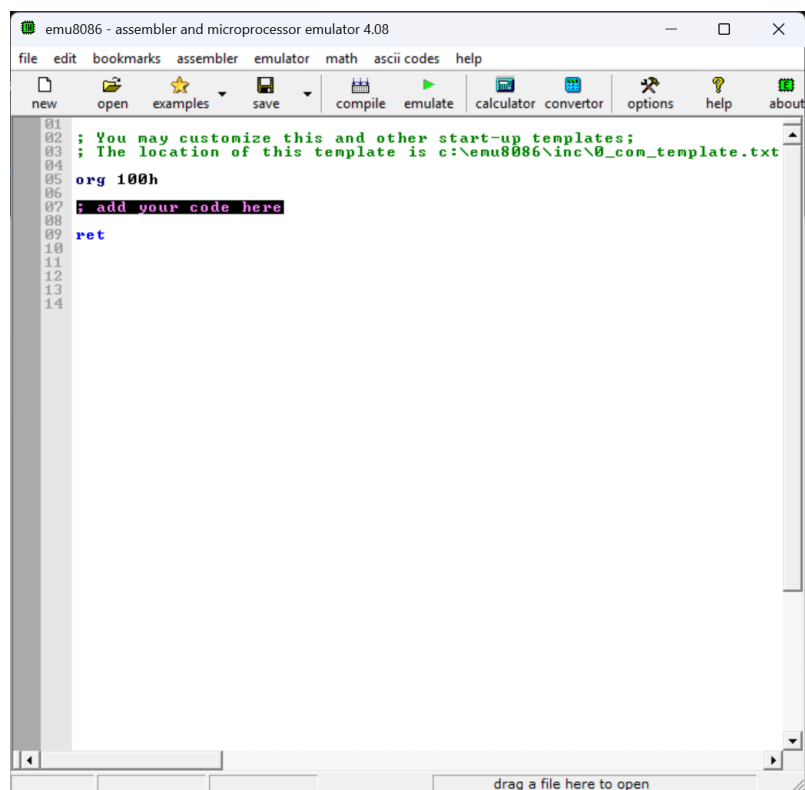


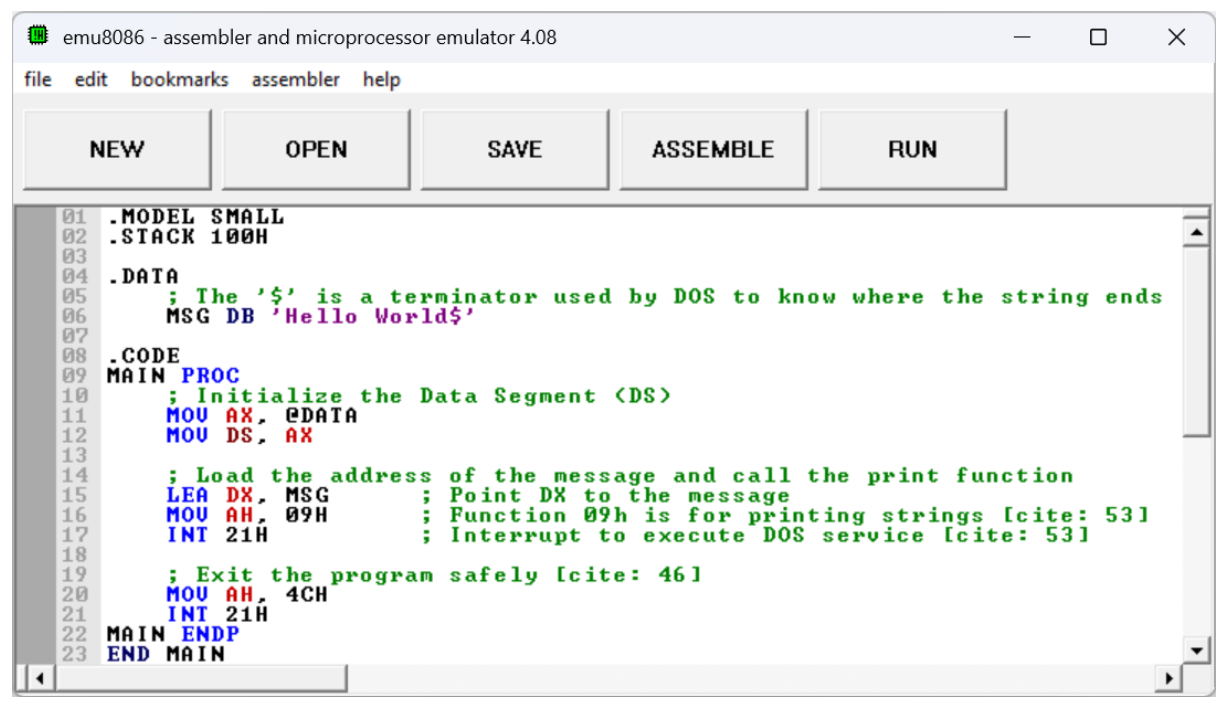
Figure 23: EMU8086 Integrated Development Environment (IDE) Editor

To print "Hello World" in EMU8086, you will use the standard structure involving a data segment for the message and a code segment for the instructions.

Question No 1

Task: Program to Display "Hello World" on Screen

EMU8086 "Hello World" Code



```
01 .MODEL SMALL
02 .STACK 100H
03
04 .DATA
05 ; The '$' is a terminator used by DOS to know where the string ends
06 MSG DB 'Hello World$'
07
08 .CODE
09 MAIN PROC
10 ; Initialize the Data Segment <DS>
11 MOV AX, @DATA
12 MOV DS, AX
13
14 ; Load the address of the message and call the print function
15 LEA DX, MSG ; Point DX to the message
16 MOV AH, 09H ; Function 09h is for printing strings [cite: 53]
17 INT 21H ; Interrupt to execute DOS service [cite: 53]
18
19 ; Exit the program safely [cite: 46]
20 MOV AH, 4CH
21 INT 21H
22 MAIN ENDP
23 END MAIN
```

Figure 24: Assembly Code for "Hello World" string display

Output

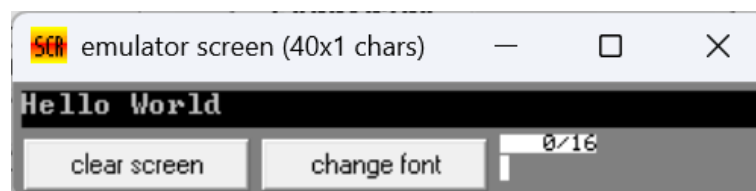


Figure 25: Emulator Output Window showing "Hello World"

LAB 3

Registers

This lab introduces the internal architecture of the **8086 processor**. You will move beyond theory to practice by writing code that manipulates **general-purpose registers**, defines **memory segments**, and executes **DOS interrupts**. The goal is to understand how the CPU stores temporary data and how to control program execution flow.

8086 Register Architecture

Registers are high-speed storage locations within the CPU used for processing data. Unlike RAM, which is like a distant shelf, registers are like bowls on the CPU's immediate worktable.

General Purpose Registers

These 16-bit registers are the primary tools for data manipulation.

Register	Name	Primary Function
AX	Accumulator	Used for arithmetic, logic, I/O, multiplication, and division.
BX	Base	Holds memory addresses that point to variables.
CX	Counter	Acts as a counter for loops; automatically decrements during iterations.
DX	Data	Used in multiplication/division and strictly required for I/O operations.

Table 1: 8086 General-Purpose Register (GPR) Set Overview

Segment Registers

These registers define specific areas of memory for your program.

Segment	Function
CS (Code)	Stores the executable instructions (the program logic).
DS (Data)	Stores initialized variables and constants.
SS (Stack)	Defines the memory area used for the stack (temporary storage).
ES (Extra)	Additional data segment used for string operations.

Table 2: Memory Segment and Register Functionality Diagram

Program Execution Flow

Assembly programs follow a specific lifecycle. The chart below illustrates the flow you will implement in this lab, from defining data to exiting via DOS interrupts.

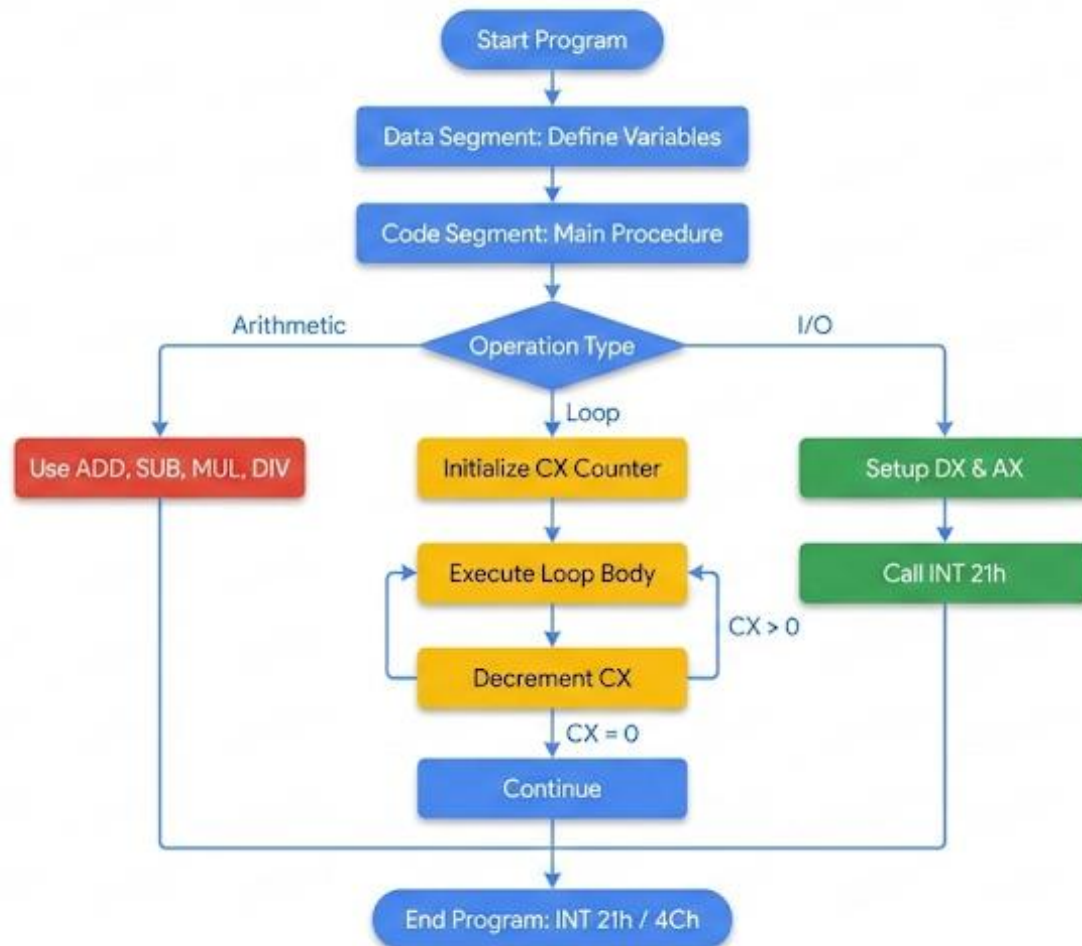


Figure 26: x86 Program Execution Lifecycle Flowchart

Key Components of the Flow

- Initialization: Setting up the Data Segment (.DATA) to store constants like messages or numbers.
- Logic Execution: The Code Segment (.CODE) where instructions like MOV and ADD change register values.
- Termination: Using MOV AH, 4Ch followed by INT 21h to safely exit the program and return control to DOS.

Essential Commands & Concepts

In this lab, you will utilize the following instructions to solve the exercises.

Data Movement (MOV)

Copies data from a source to a destination.

Example: MOV AX, 10 puts the value 10 into the Accumulator.

Arithmetic (ADD, SUB, INC, DEC)

Performs basic math on register values.

Example: INC AX increases the value in AX by 1.

Comparison (CMP)

Compare two values (like subtraction) to set flags without changing the data.

Used to decide if values are Equal, Not Equal, Greater, etc.

Interrupts (INT 21h)

Accesses DOS services.

Commonly used to print strings (Function 09h) or exit (Function 4Ch).

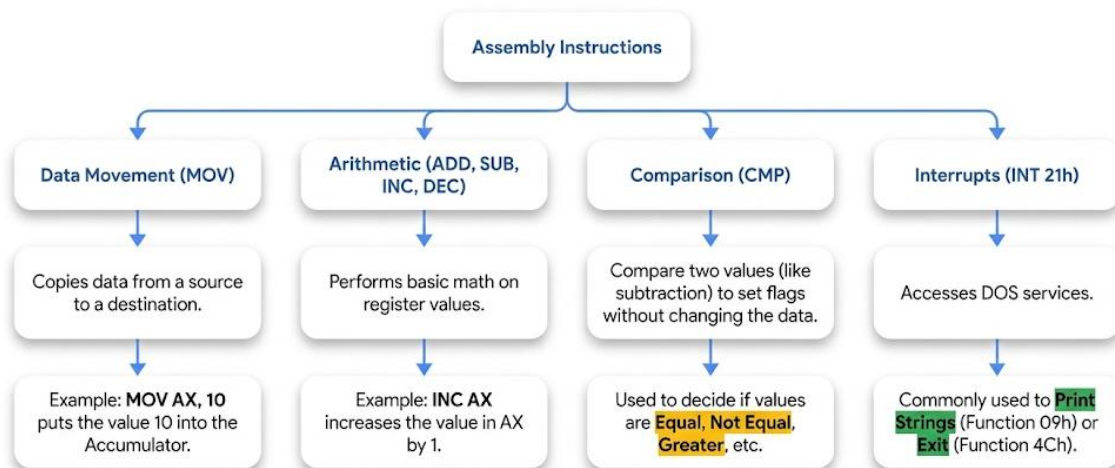
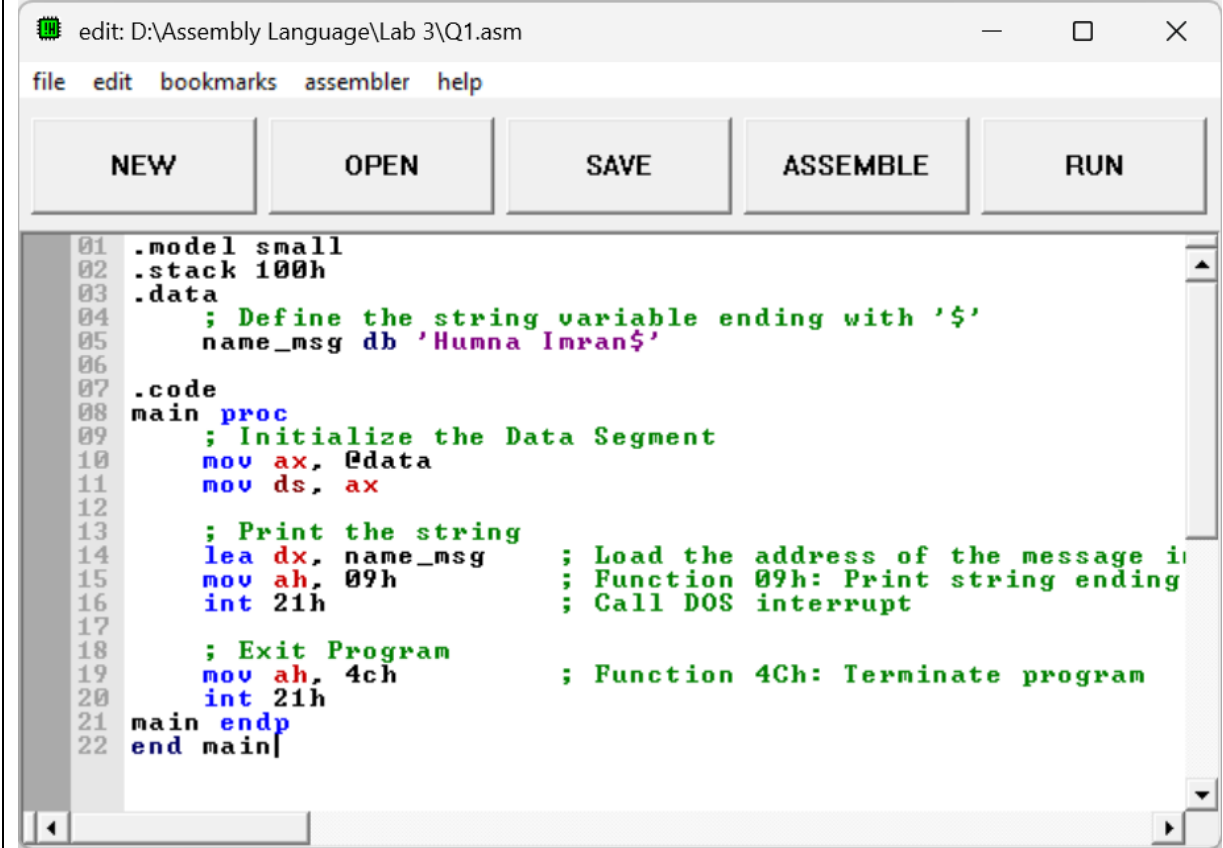


Figure 27: x86 Arithmetic Instruction Set Summary

Question no 1

Task: Write a program to display your full name on the screen.

Print Your Name



The screenshot shows a window titled "edit: D:\Assembly Language\Lab 3\Q1.asm". The menu bar includes "file", "edit", "bookmarks", "assembler", and "help". Below the menu are five buttons: "NEW", "OPEN", "SAVE", "ASSEMBLE", and "RUN". The main text area contains the following assembly code:

```
01 .model small
02 .stack 100h
03 .data
04     ; Define the string variable ending with '$'
05     name_msg db 'Humna Imran$'
06
07 .code
08 main proc
09     ; Initialize the Data Segment
10     mov ax, @data
11     mov ds, ax
12
13     ; Print the string
14     lea dx, name_msg      ; Load the address of the message in
15     mov ah, 09h          ; Function 09h: Print string ending in
16     int 21h              ; Call DOS interrupt
17
18     ; Exit Program
19     mov ah, 4ch           ; Function 4Ch: Terminate program
20     int 21h
21 main endp
22 end main
```

Figure 28: Q1.asm

Output

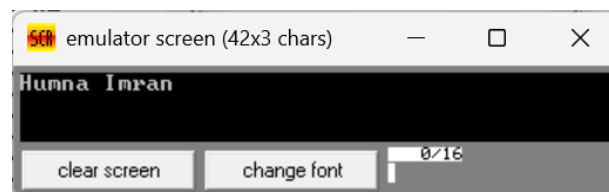
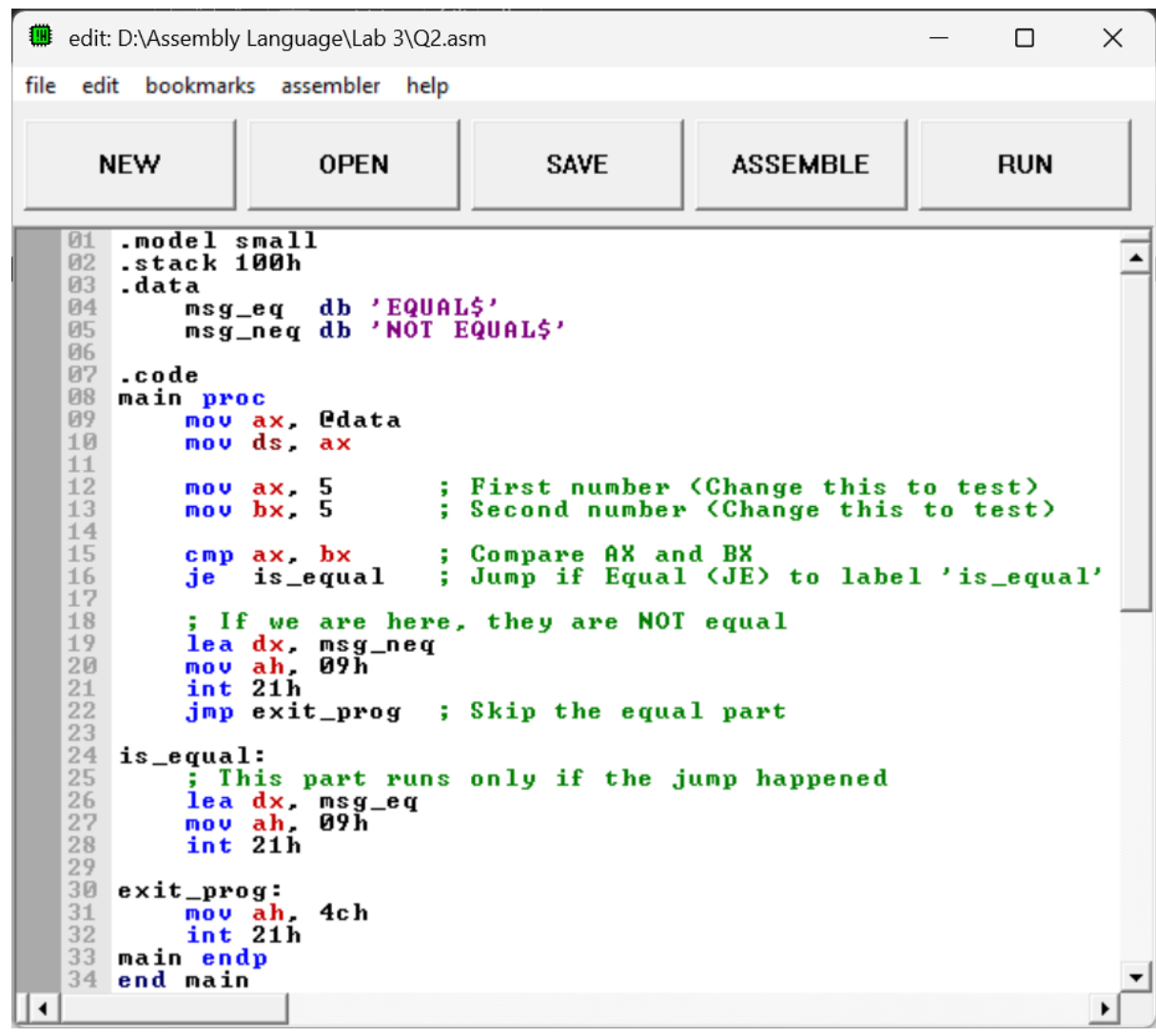


Figure 29: Output screen

Question no 2

Task: Compare two numbers. If equal print "EQUAL", otherwise print "NOT EQUAL".

Compare Two Numbers



```

01 .model small
02 .stack 100h
03 .data
04     msg_eq    db 'EQUAL$'
05     msg_neq   db 'NOT EQUAL$'
06
07 .code
08 main proc
09     mov ax, @data
10     mov ds, ax
11
12     mov ax, 5      ; First number <Change this to test>
13     mov bx, 5      ; Second number <Change this to test>
14
15     cmp ax, bx     ; Compare AX and BX
16     je  is_equal   ; Jump if Equal <JE> to label 'is_equal'
17
18     ; If we are here, they are NOT equal
19     lea dx, msg_neq
20     mov ah, 09h
21     int 21h
22     jmp exit_prog  ; Skip the equal part
23
24 is_equal:
25     ; This part runs only if the jump happened
26     lea dx, msg_eq
27     mov ah, 09h
28     int 21h
29
30 exit_prog:
31     mov ah, 4ch
32     int 21h
33 main endp
34 end main
  
```

Figure 30: Q2.asm

Output

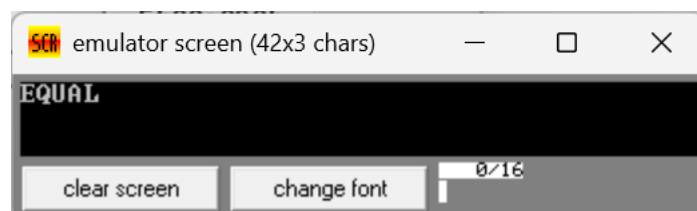
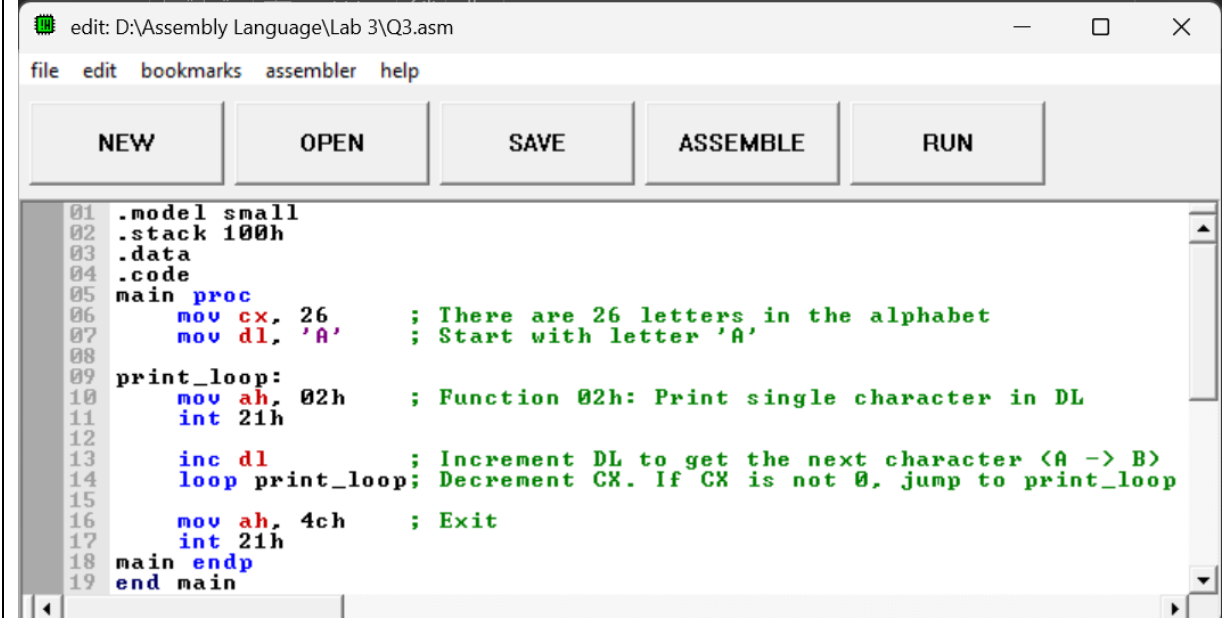


Figure 31: Output screen

Question no 3

Task: Write a program to print all uppercase alphabets A-Z using a loop.

Print A to Z



```
edit: D:\Assembly Language\Lab 3\Q3.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 .model small
02 .stack 100h
03 .data
04 .code
05 main proc
06     mov cx, 26      ; There are 26 letters in the alphabet
07     mov dl, 'A'     ; Start with letter 'A'
08
09 print_loop:
10     mov ah, 02h     ; Function 02h: Print single character in DL
11     int 21h
12
13     inc dl          ; Increment DL to get the next character (A -> B)
14     loop print_loop; Decrement CX. If CX is not 0, jump to print_loop
15
16     mov ah, 4ch     ; Exit
17     int 21h
18 main endp
19 end main
```

Figure 32: Q3.asm

Output

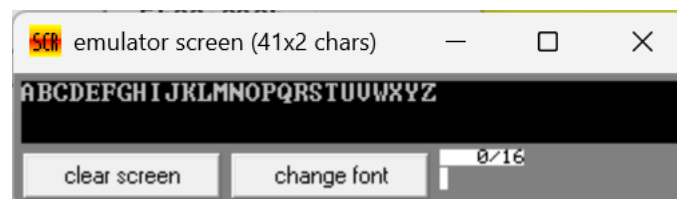
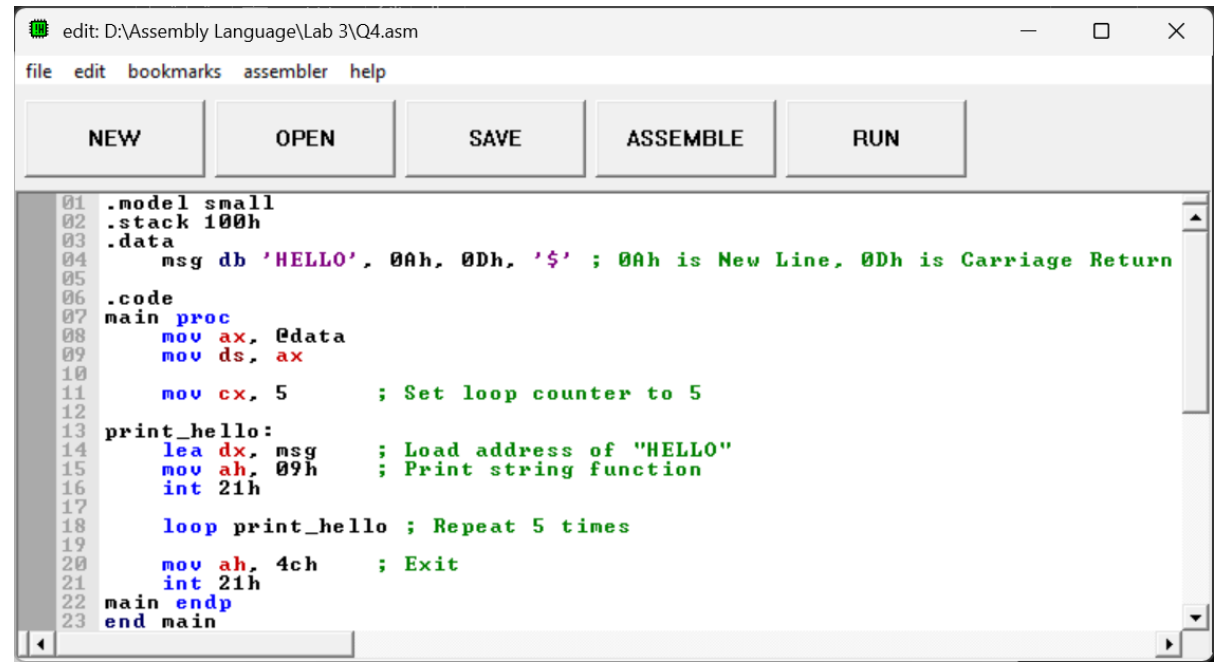


Figure 33: Output screen

Question no 4

Task: Print "HELLO" five times using a loop.

Print a Message 5 Times



```
01 .model small
02 .stack 100h
03 .data
04     msg db 'HELLO', 0Ah, 0Dh, '$' ; 0Ah is New Line, 0Dh is Carriage Return
05
06 .code
07 main proc
08     mov ax, @data
09     mov ds, ax
10
11     mov cx, 5        ; Set loop counter to 5
12
13 print_hello:
14     lea dx, msg      ; Load address of "HELLO"
15     mov ah, 09h      ; Print string function
16     int 21h
17
18     loop print_hello ; Repeat 5 times
19
20     mov ah, 4ch      ; Exit
21     int 21h
22 main endp
23 end main
```

Figure 34: Q4.asm

Output



```
emulator screen (40x5 chars)
HELLO
HELLO
HELLO
HELLO
HELLO
HELLO
```

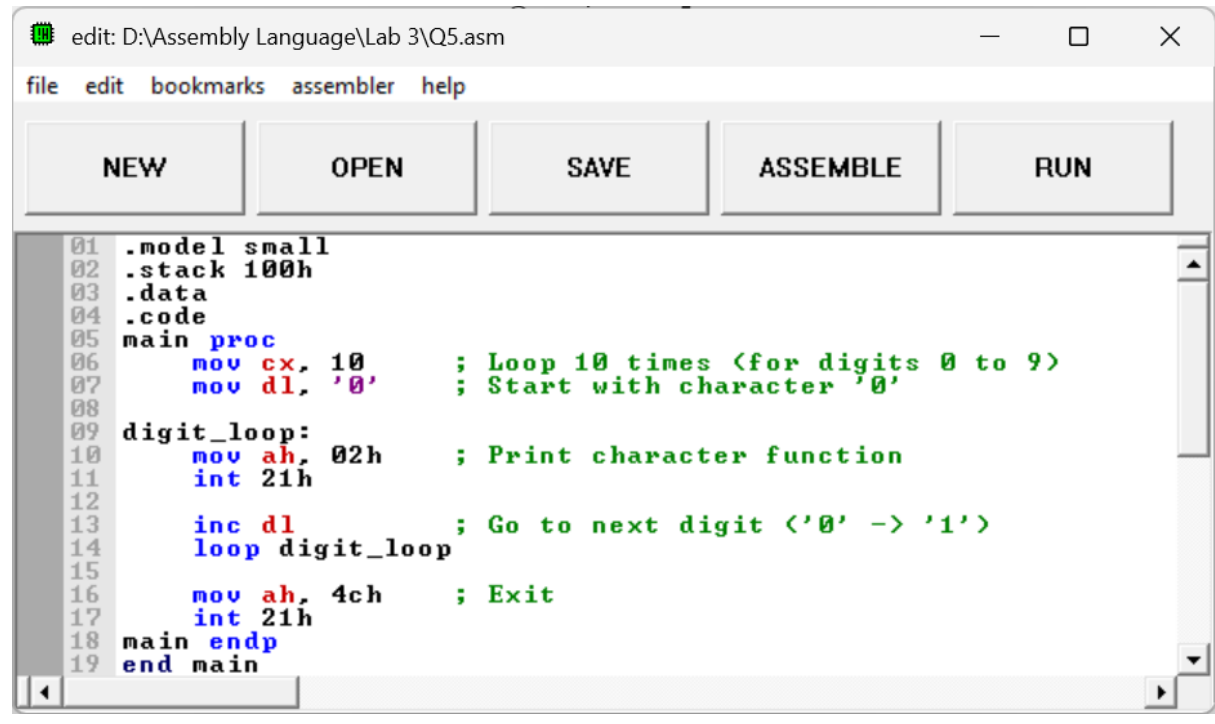
clear screen change font 0/16

Figure 35: Output screen

Question no 5

Task: Display all digits from 0 to 9 in one line.

Print 0-9 (Digits)



```
edit: D:\Assembly Language\Lab 3\Q5.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 .model small
02 .stack 100h
03 .data
04 .code
05 main proc
06     mov cx, 10      ; Loop 10 times <for digits 0 to 9>
07     mov dl, '0'     ; Start with character '0'
08
09 digit_loop:
10     mov ah, 02h     ; Print character function
11     int 21h
12
13     inc dl          ; Go to next digit <'0' -> '1'>
14     loop digit_loop
15
16     mov ah, 4ch     ; Exit
17     int 21h
18 main endp
19 end main
```

Figure 36: Q5.asm

Output

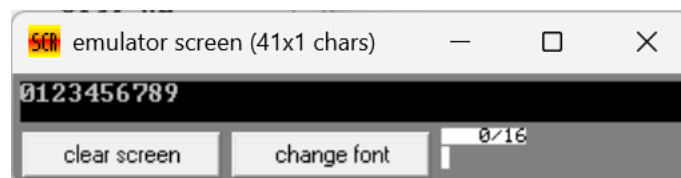
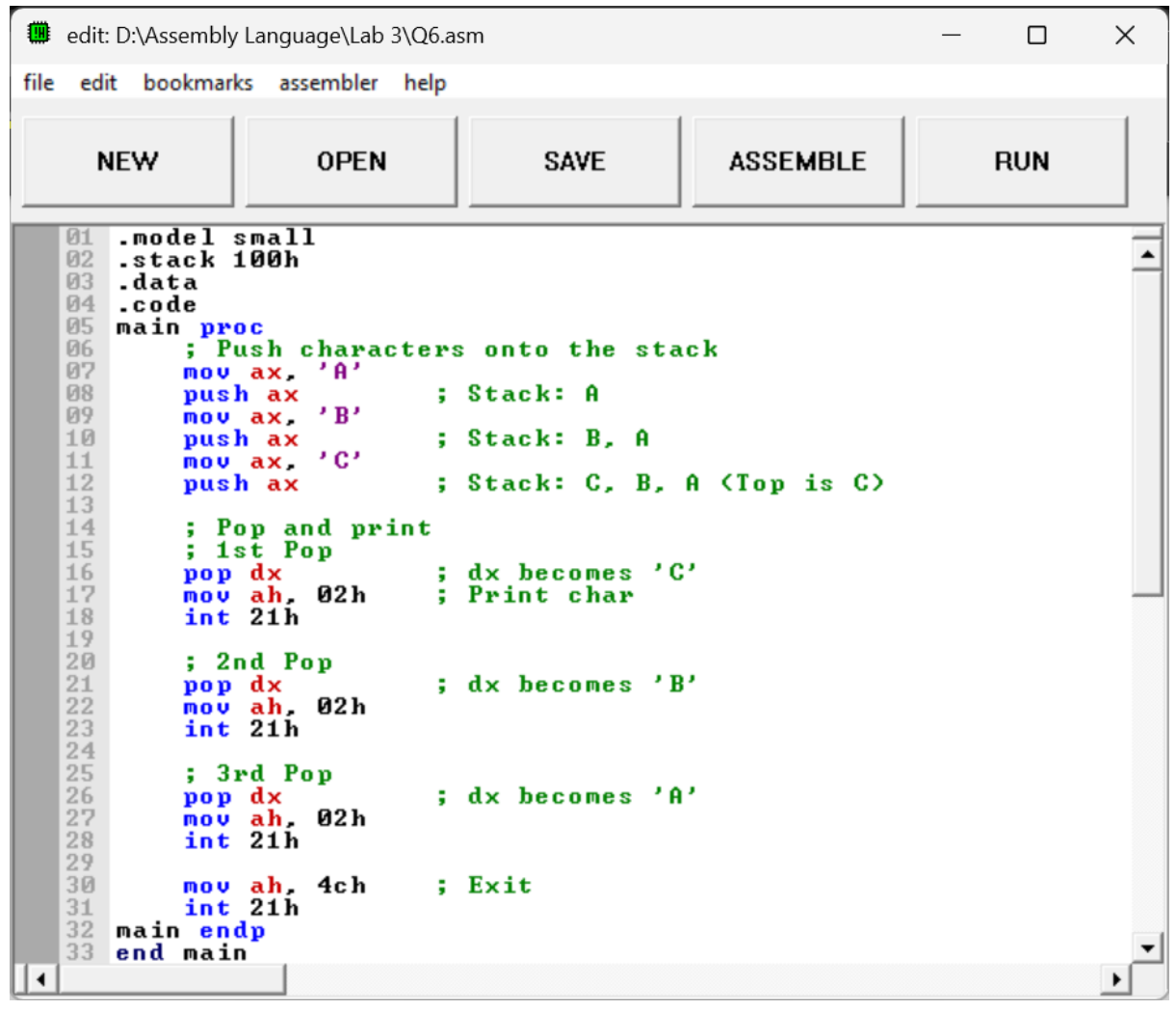


Figure 37: Output screen

Question no 6

Task: Push 'A', 'B', 'C' onto stack, pop and display them (Output: CBA).

Reverse Characters Using Stack



```

01 .model small
02 .stack 100h
03 .data
04 .code
05 main proc
06     ; Push characters onto the stack
07     mov ax, 'A'
08     push ax      ; Stack: A
09     mov ax, 'B'
10     push ax      ; Stack: B, A
11     mov ax, 'C'
12     push ax      ; Stack: C, B, A (Top is C)
13
14     ; Pop and print
15     ; 1st Pop
16     pop dx       ; dx becomes 'C'
17     mov ah, 02h   ; Print char
18     int 21h
19
20     ; 2nd Pop
21     pop dx       ; dx becomes 'B'
22     mov ah, 02h   ; Print char
23     int 21h
24
25     ; 3rd Pop
26     pop dx       ; dx becomes 'A'
27     mov ah, 02h   ; Print char
28     int 21h
29
30     mov ah, 4ch   ; Exit
31     int 21h
32 main endp
33 end main
  
```

Figure 38: Q6.asm

Output

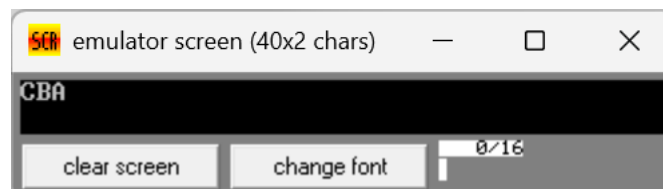
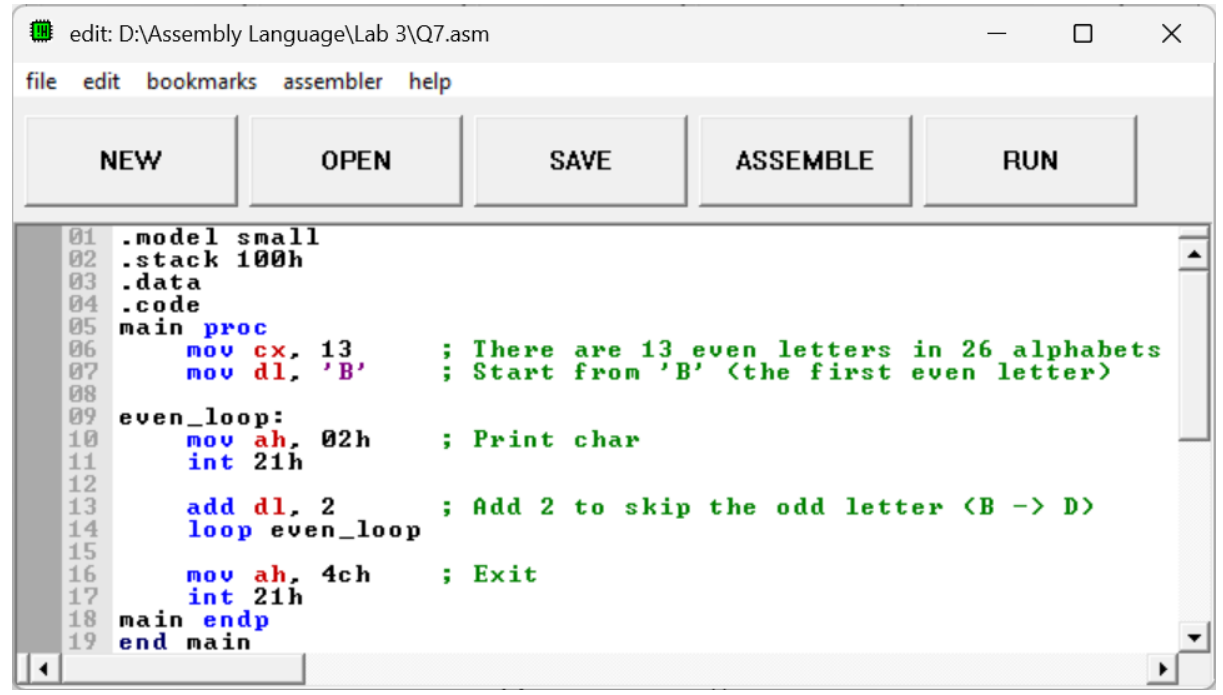


Figure 39: Output screen

Question no 7

Task: Print B, D, F... Z.

Print Even Letters Only



```
01 .model small
02 .stack 100h
03 .data
04 .code
05 main proc
06     mov cx, 13      ; There are 13 even letters in 26 alphabets
07     mov dl, 'B'     ; Start from 'B' (the first even letter)
08
09 even_loop:
10     mov ah, 02h     ; Print char
11     int 21h
12
13     add dl, 2       ; Add 2 to skip the odd letter (B -> D)
14     loop even_loop
15
16     mov ah, 4ch     ; Exit
17     int 21h
18 main endp
19 end main
```

Figure 40: Q7.asm

Output

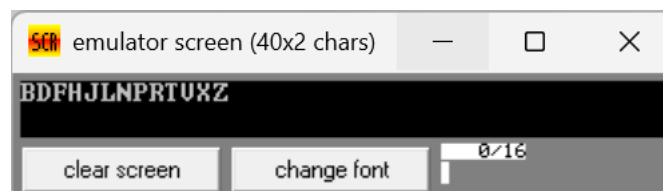
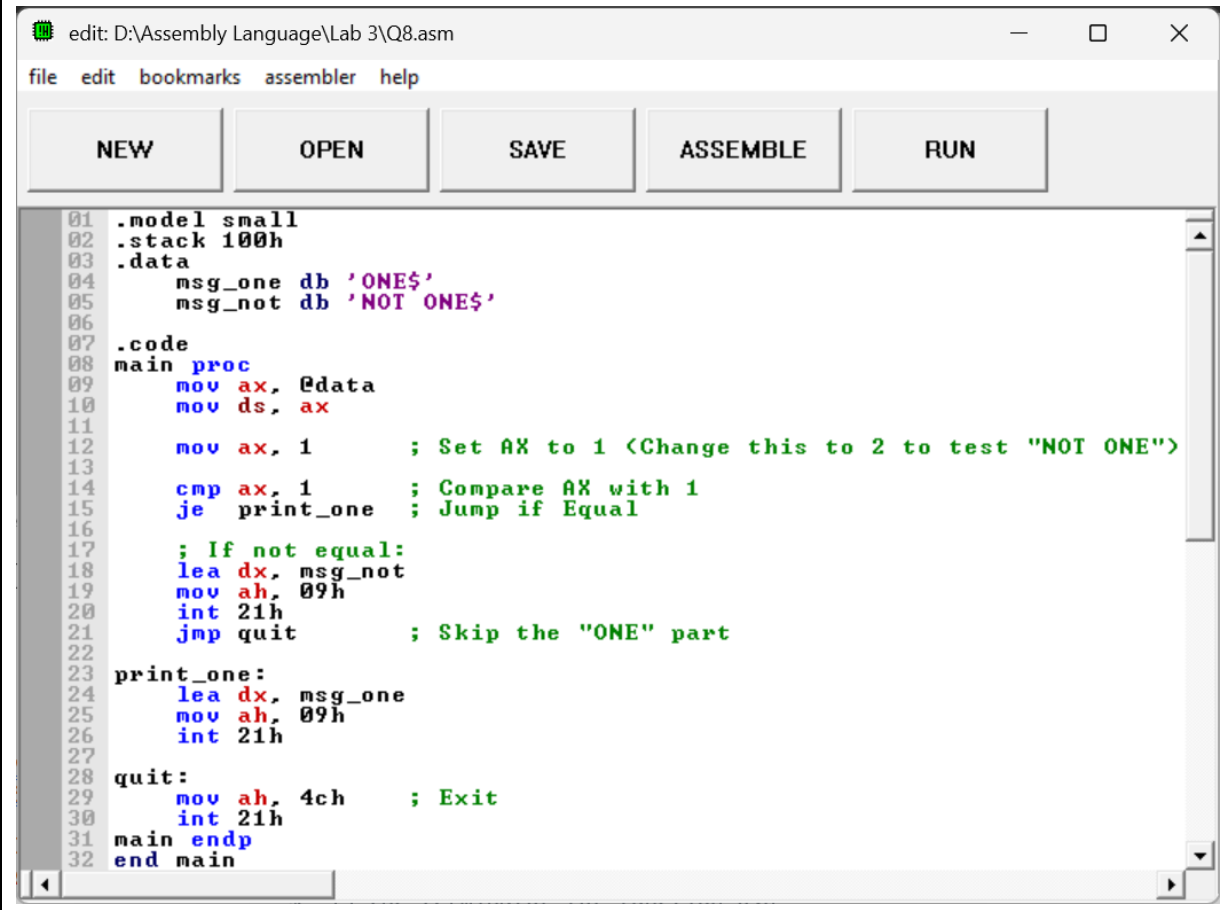


Figure 41: Output screen

Question no 8

Task: If AX is 1 print "ONE", otherwise print "NOT ONE".

Jump Example (Check AX=1)



```

01 .model small
02 .stack 100h
03 .data
04     msg_one db 'ONE$'
05     msg_not db 'NOT ONE$'
06
07 .code
08 main proc
09     mov ax, @data
10     mov ds, ax
11
12     mov ax, 1      ; Set AX to 1 (Change this to 2 to test "NOT ONE")
13
14     cmp ax, 1      ; Compare AX with 1
15     je  print_one  ; Jump if Equal
16
17     ; If not equal:
18     lea dx, msg_not
19     mov ah, 09h
20     int 21h
21     jmp quit       ; Skip the "ONE" part
22
23 print_one:
24     lea dx, msg_one
25     mov ah, 09h
26     int 21h
27
28 quit:
29     mov ah, 4ch    ; Exit
30     int 21h
31 main endp
32 end main
  
```

Figure 42: Q8.asm

Output

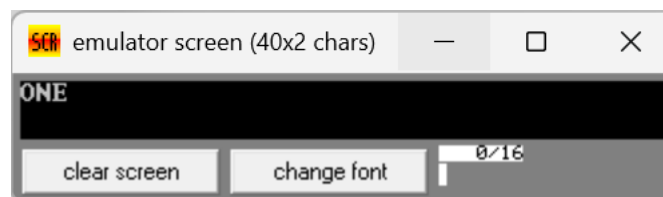
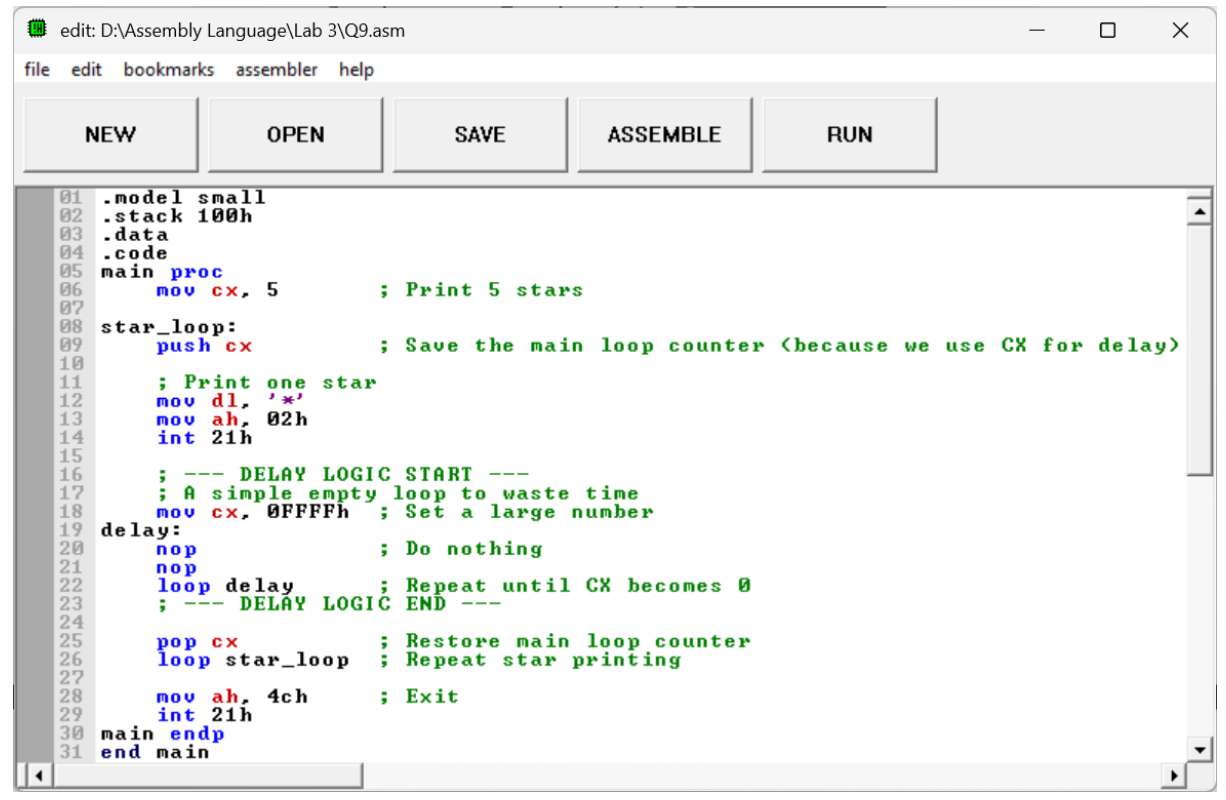


Figure 43: Output screen

Question no 9

Task: Print ***** with a small pause.

Loop with Delay



```
01 .model small
02 .stack 100h
03 .data
04 .code
05 main proc
06     mov cx, 5          ; Print 5 stars
07
08 star_loop:
09     push cx            ; Save the main loop counter (because we use CX for delay)
10
11     ; Print one star
12     mov dl, '*'
13     mov ah, 02h
14     int 21h
15
16     ; --- DELAY LOGIC START ---
17     ; A simple empty loop to waste time
18     mov cx, 0FFFFh    ; Set a large number
19
20 delay:
21     nop                ; Do nothing
22     nop
23     loop delay         ; Repeat until CX becomes 0
24     ; --- DELAY LOGIC END ---
25
26     pop cx             ; Restore main loop counter
27     loop star_loop    ; Repeat star printing
28
29     mov ah, 4ch        ; Exit
30     int 21h
31 main endp
end main
```

Figure 44: Q9.asm

Output

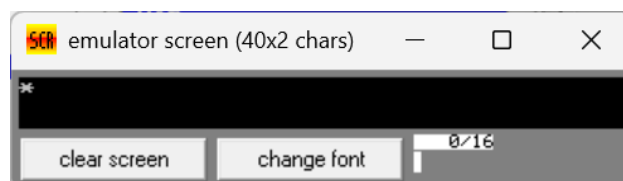
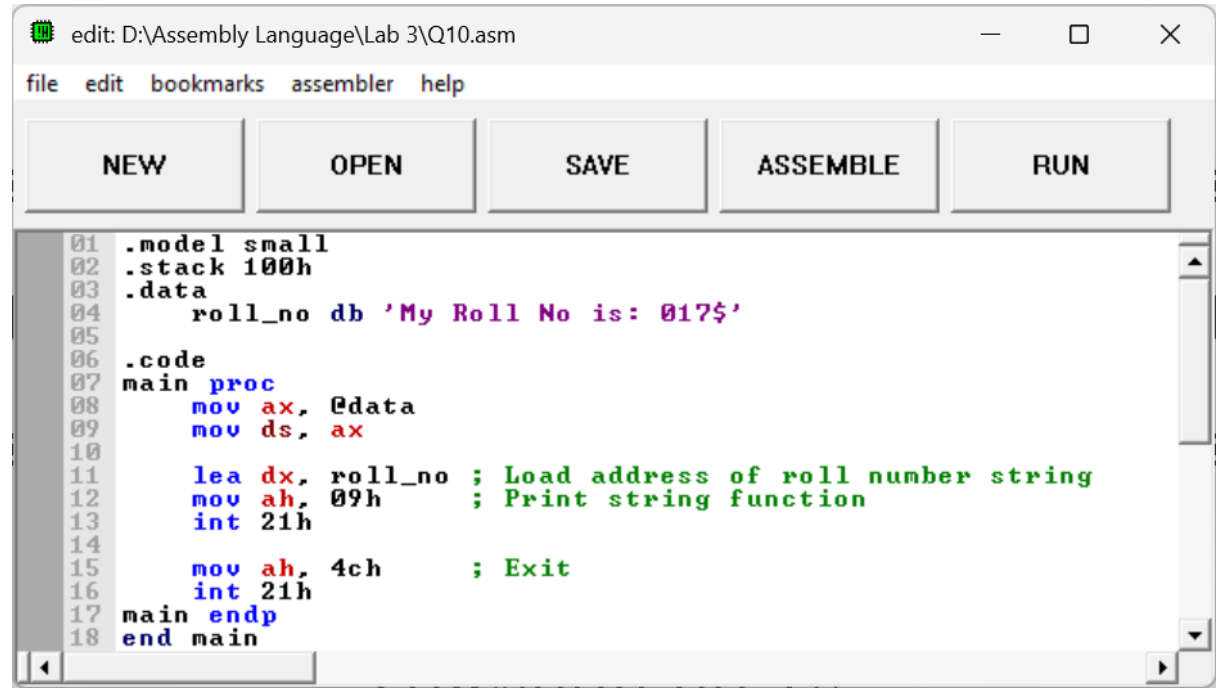


Figure 45: Output screen

Question no 10

Task: Print "017".

Print Your Roll Number



```
edit: D:\Assembly Language\Lab 3\Q10.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 .model small
02 .stack 100h
03 .data
04     roll_no db 'My Roll No is: 017$'
05
06 .code
07 main proc
08     mov ax, @data
09     mov ds, ax
10
11     lea dx, roll_no ; Load address of roll number string
12     mov ah, 09h     ; Print string function
13     int 21h
14
15     mov ah, 4ch      ; Exit
16     int 21h
17 main endp
18 end main
```

Figure 46: Q10.asm

Output

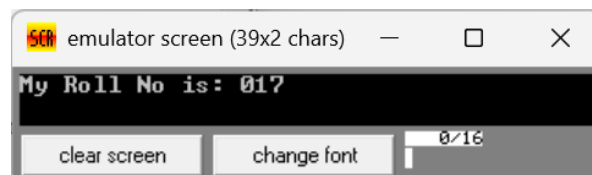


Figure 47: Output screen

LAB 4

Arithmetic Register

This lab explores the fundamental arithmetic operations essential for computation in **x86 architecture**. You will write and execute Assembly programs to perform addition, subtraction, multiplication, and division using **16-bit general-purpose registers** like AX, BX, CX, and DX.

Objectives

By completing this lab, you will achieve the following:

- Perform basic **Addition (ADD)** and **Subtraction (SUB)** on 16-bit operands.
- Execute unsigned **Multiplication (MUL)** and manage 32-bit results stored across register pairs.
- Execute unsigned **Division (DIV)** to calculate quotients and remainders.

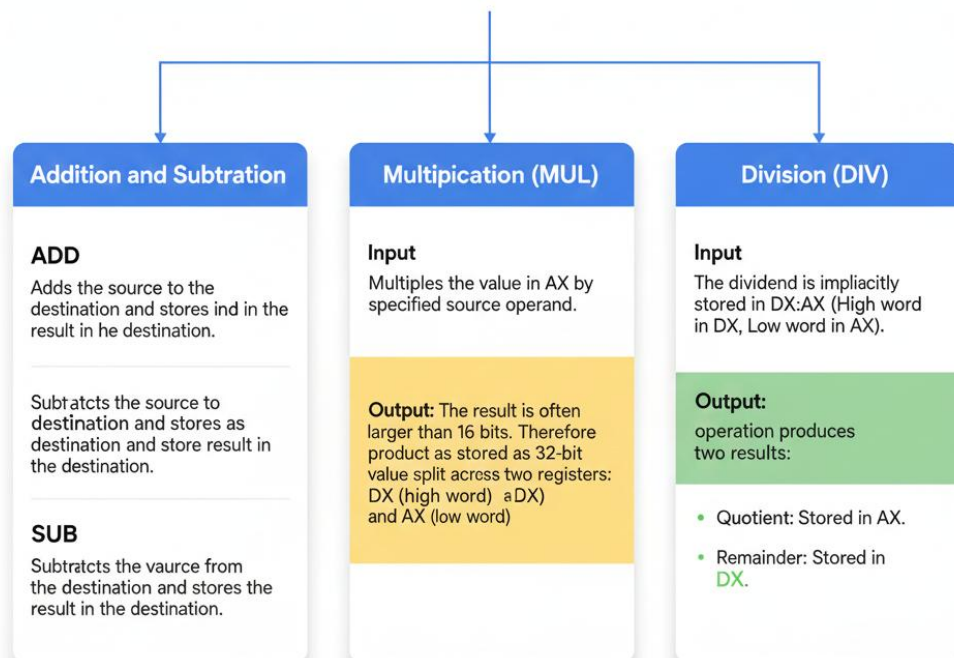
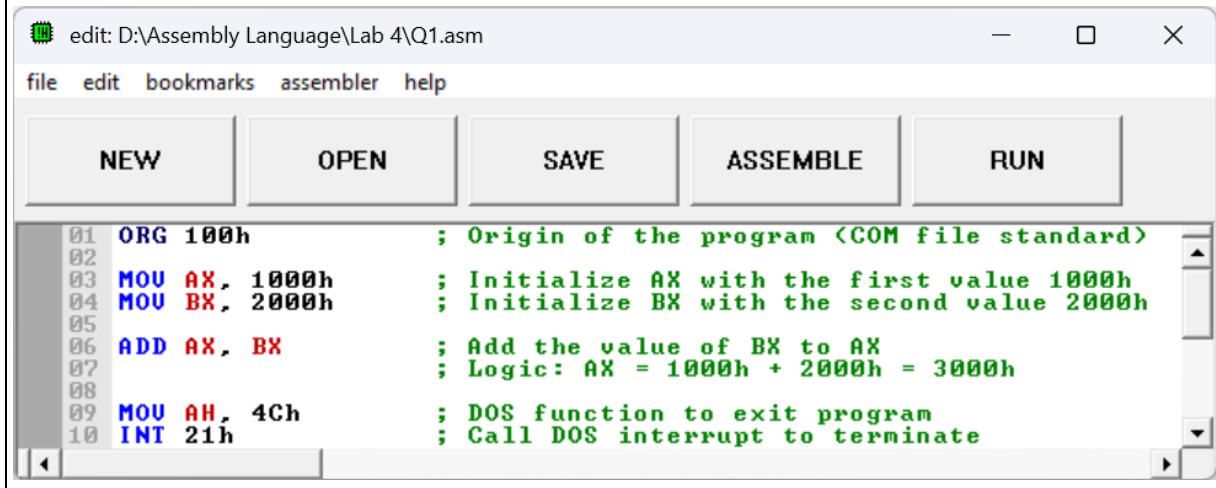


Figure 48: Logical Shift and Comparison Logic Flow

Question no 1

Task: The value 1000h is added to 2000h. The result is stored in AX.

Simple Addition in AX



```

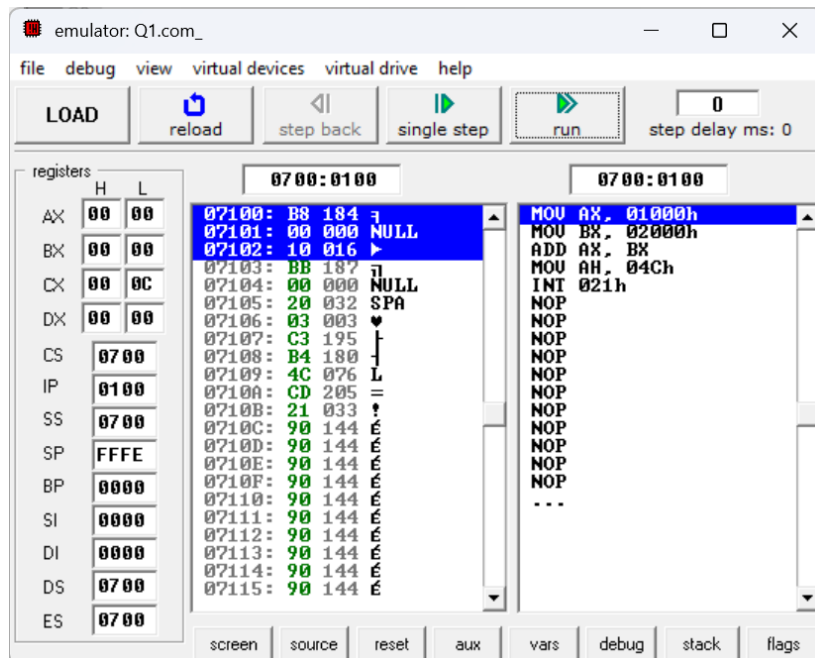
edit: D:\Assembly Language\Lab 4\Q1.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 ORG 100h           ; Origin of the program (COM file standard)
02
03 MOV AX, 1000h      ; Initialize AX with the first value 1000h
04 MOV BX, 2000h      ; Initialize BX with the second value 2000h
05
06 ADD AX, BX         ; Add the value of BX to AX
07                     ; Logic: AX = 1000h + 2000h = 3000h
08
09 MOV AH, 4Ch        ; DOS function to exit program
10 INT 21h           ; Call DOS interrupt to terminate
  
```

Figure 49: Assembly Code for Simple Addition

Output



emulator: Q1.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	00
BX	00	00
CX	00	0C
DX	00	00
CS	07	00
IP	01	00
SS	07	00
SP	FF	FE
BP	00	00
SI	00	00
DI	00	00
DS	07	00
ES	07	00

0700:0100

0700:0100

```

07100: B8 184  MOV AX, 01000h
07101: 00 000  NULL
07102: 10 016  MOV BX, 02000h
07103: B8 187  ADD AX, BX
07104: 00 000  NULL
07105: 20 032  MOV AH, 04Ch
07106: 03 003  INT 021h
07107: C3 195  SPA
07108: B4 180  NOP
07109: 4C 076  NOP
0710A: CD 205  NOP
0710B: 21 033  NOP
0710C: 90 144  NOP
0710D: 90 144  NOP
0710E: 90 144  NOP
0710F: 90 144  NOP
07110: 90 144  NOP
07111: 90 144  NOP
07112: 90 144  NOP
07113: 90 144  NOP
07114: 90 144  NOP
07115: 90 144  NOP
  
```

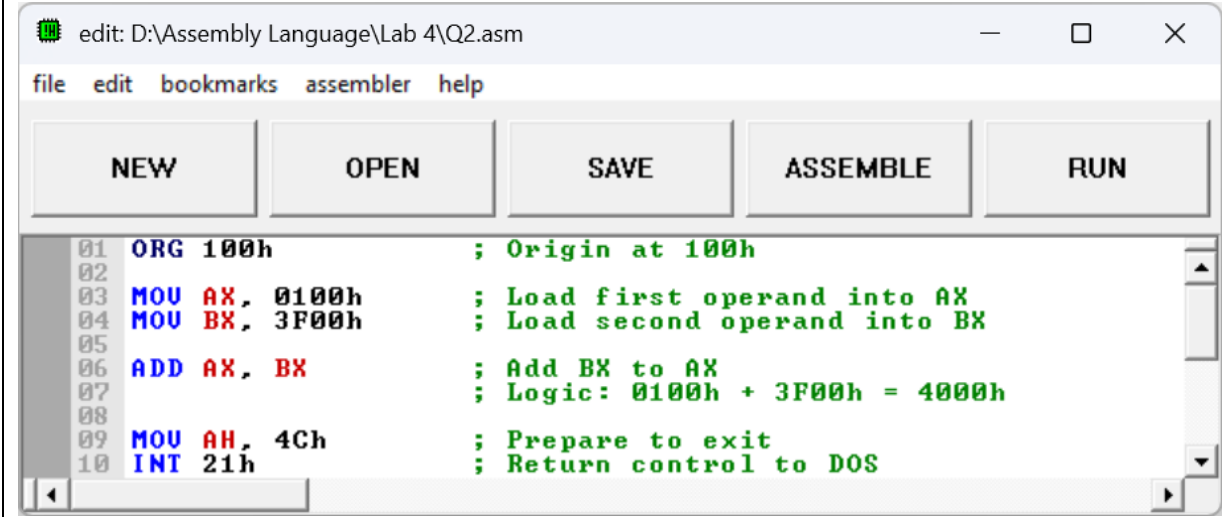
screen source reset aux vars debug stack flags

Figure 50: Result of Adding 1000h and 2000h

Question no 2:

Task: The value 0100h is added to 3F00h. The result is stored in AX.

Addition with Carry Potential



```

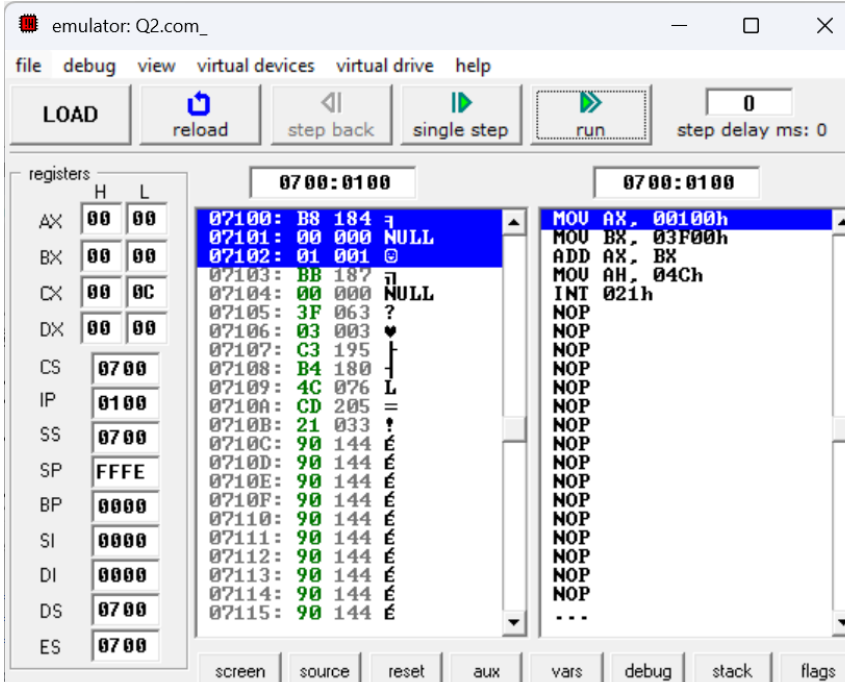
edit: D:\Assembly Language\Lab 4\Q2.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 ORG 100h           ; Origin at 100h
02
03 MOV AX, 0100h      ; Load first operand into AX
04 MOV BX, 3F00h      ; Load second operand into BX
05
06 ADD AX, BX         ; Add BX to AX
07                   ; Logic: 0100h + 3F00h = 4000h
08
09 MOV AH, 4Ch        ; Prepare to exit
10 INT 21h           ; Return control to DOS
  
```

Figure 51: Code for Adding 0100h and 3F00h

Output



emulator: Q2.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	00
BX	00	00
CX	00	0C
DX	00	00
CS	0700	
IP	0100	
SS	0700	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

0700:0100

07100:	B8	184	?	MOV AX, 00100h
07101:	00	000	NULL	MOV BX, 03F00h
07102:	01	001	0	ADD AX, BX
07103:	BB	187	?	MOV AH, 04Ch
07104:	00	000	?	INT 021h
07105:	3F	063	?	NOP
07106:	03	003	?	NOP
07107:	C3	195	?	NOP
07108:	B4	180	?	NOP
07109:	4C	076	L	NOP
0710A:	CD	205	=	NOP
0710B:	21	033	?	NOP
0710C:	90	144	E	NOP
0710D:	90	144	E	NOP
0710E:	90	144	E	NOP
0710F:	90	144	E	NOP
07110:	90	144	E	NOP
07111:	90	144	E	NOP
07112:	90	144	E	NOP
07113:	90	144	E	NOP
07114:	90	144	E	NOP
07115:	90	144	E	...

screen source reset aux vars debug stack flags

Figure 52: Emulator Showing AX=4000h Result

Question no 3

Task: 0050h is subtracted from 00A0h. The result is stored in BX.

Subtraction using BX Register

```

edit: D:\Assembly Language\Lab 4\Q3.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 ; Name: Humna Imran
02 ; Roll Number: 2023-BS-AI-017
03 ; Lab 4 - Question 3
04 ; Task: Subtract 0050h from 00A0h, result in BX
05
06 ORG 100h           ; Standard COM file start
07
08 MOV BX, 00A0h      ; Load the Minuend (Total) into BX
09 MOV CX, 0050h      ; Load the Subtrahend (Amount to remove) into CX
10
11 SUB BX, CX         ; Subtract CX from BX
12                   ; Logic: BX = 00A0h - 0050h = 0050h
13
14 MOV AH, 4Ch        ; Function to terminate
15 INT 21h           ; Execute interrupt
  
```

Figure 53: Subtraction Logic using BX Register

Output:

emulator: Q3.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	00
BX	00	50
CX	00	50
DX	00	00
CS	0700	
IP	0100	
SS	0700	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

0700:0100 0700:0100

Address	Hex	ASCII	Instruction
07100	BB 187	j	MOV BX, 00A0h
07101	A0 160	a	MOV CX, 0050h
07102	00 000	NULL	SUB BX, CX
07103	B9 185	j	MOV AH, 04Ch
07104	50 080	P	INT 021h
07105	00 000	NULL	NOP
07106	2B 043	+	NOP
07107	D9 217	j	NOP
07108	B4 180	j	NOP
07109	4C 076	L	NOP
0710A	CD 205	=	NOP
0710B	21 033	!	NOP
0710C	90 144	E	NOP
0710D	90 144	E	NOP
0710E	90 144	E	NOP
0710F	90 144	E	NOP
07110	90 144	E	NOP
07111	90 144	E	NOP
07112	90 144	E	NOP
07113	90 144	E	NOP
07114	90 144	E	NOP
07115	90 144	E	...

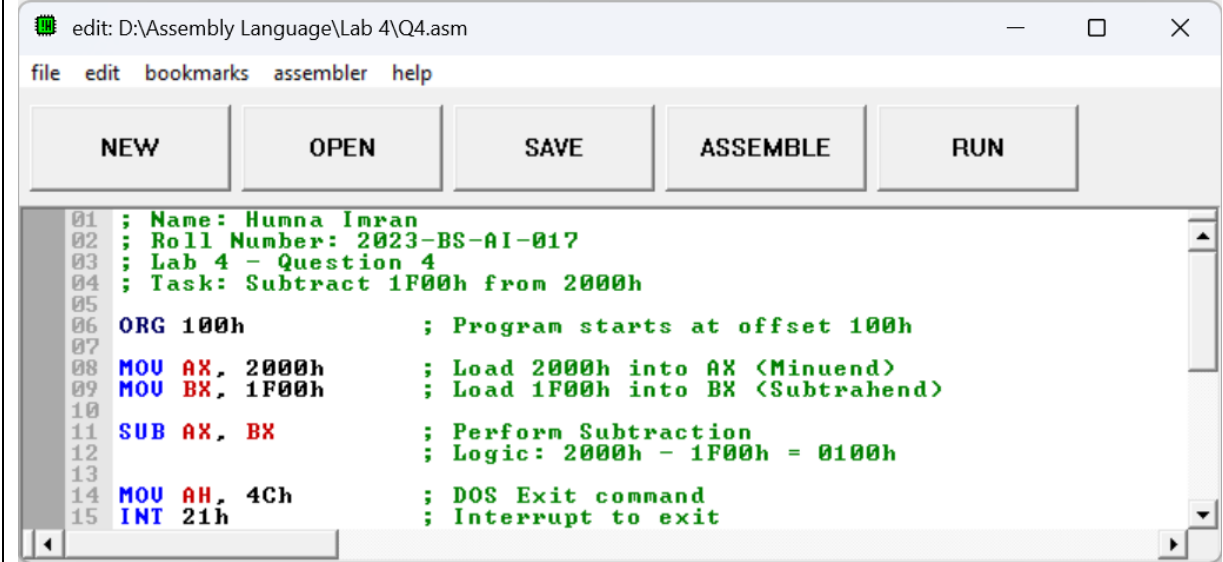
screen source reset aux vars debug stack flags

Figure 54: Result of 00A0h minus 0050h

Question no 4

Task: 1F00h is subtracted from 2000h. The result is stored in AX.

Standard Subtraction in AX



```

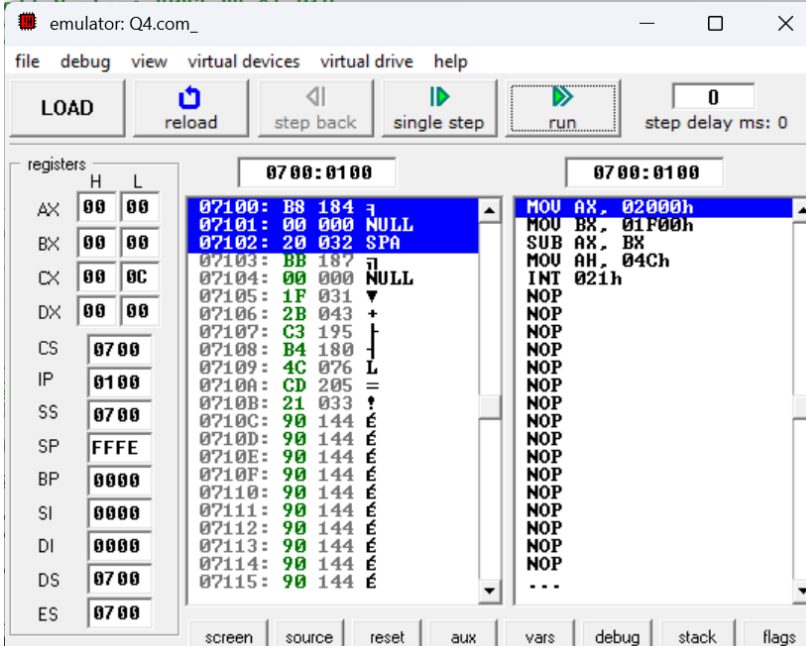
edit: D:\Assembly Language\Lab 4\Q4.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 ; Name: Humna Imran
02 ; Roll Number: 2023-BS-AI-017
03 ; Lab 4 - Question 4
04 ; Task: Subtract 1F00h from 2000h
05
06 ORG 100h           ; Program starts at offset 100h
07
08 MOV AX, 2000h      ; Load 2000h into AX (Minuend)
09 MOV BX, 1F00h      ; Load 1F00h into BX (Subtrahend)
10
11 SUB AX, BX         ; Perform Subtraction
12                     ; Logic: 2000h - 1F00h = 0100h
13
14 MOV AH, 4Ch        ; DOS Exit command
15 INT 21h           ; Interrupt to exit
  
```

Figure 55: Subtraction Code for 2000h and 1F00h

Output



emulator: Q4.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	00
BX	00	00
CX	00	0C
DX	00	00
CS	0700	
IP	0100	
SS	0700	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

0700:0100 0700:0100

```

07100: B8 184  MOV AX, 02000h
07101: 00 000  MOV BX, 01F00h
07102: 20 032  SUB AX, BX
07103: BB 187  MOV AH, 04Ch
07104: 00 000  INT 021h
07105: 1F 031  NOP
07106: 2B 043  NOP
07107: C3 195  NOP
07108: B4 180  NOP
07109: 4C 076  NOP
0710A: CD 205  NOP
0710B: 21 033  NOP
0710C: 90 144  NOP
0710D: 90 144  NOP
0710E: 90 144  NOP
0710F: 90 144  NOP
07110: 90 144  NOP
07111: 90 144  NOP
07112: 90 144  NOP
07113: 90 144  NOP
07114: 90 144  NOP
07115: 90 144  NOP
...
  
```

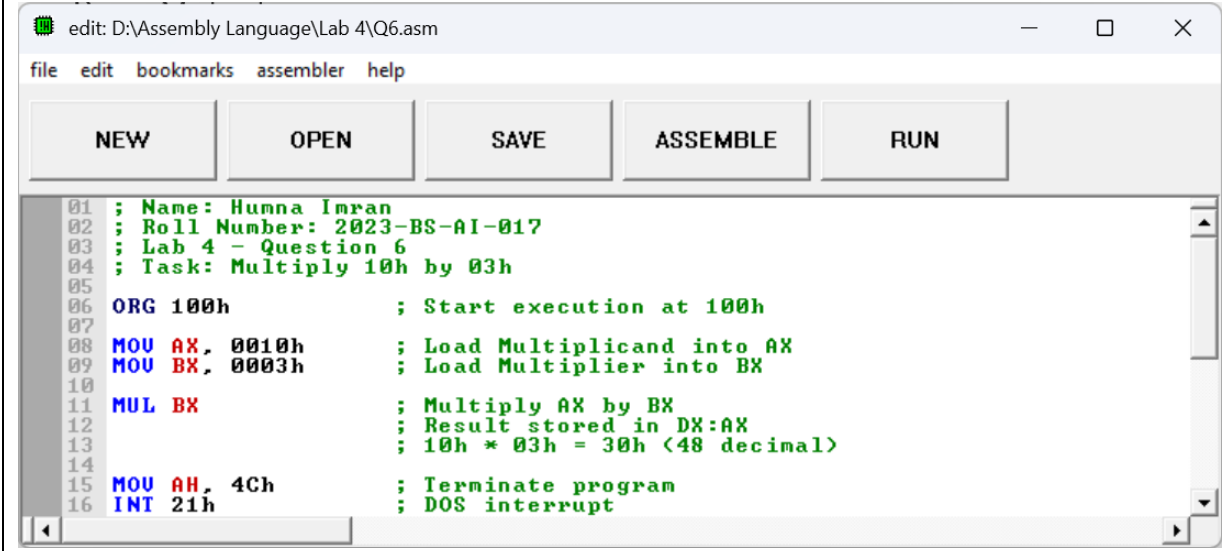
screen source reset aux vars debug stack flags

Figure 56: Output Window Showing AX=0100h

Question no 6

Task: The number 10h is multiplied by 03h. The product is stored in DX:AX.

Basic Multiplication



```

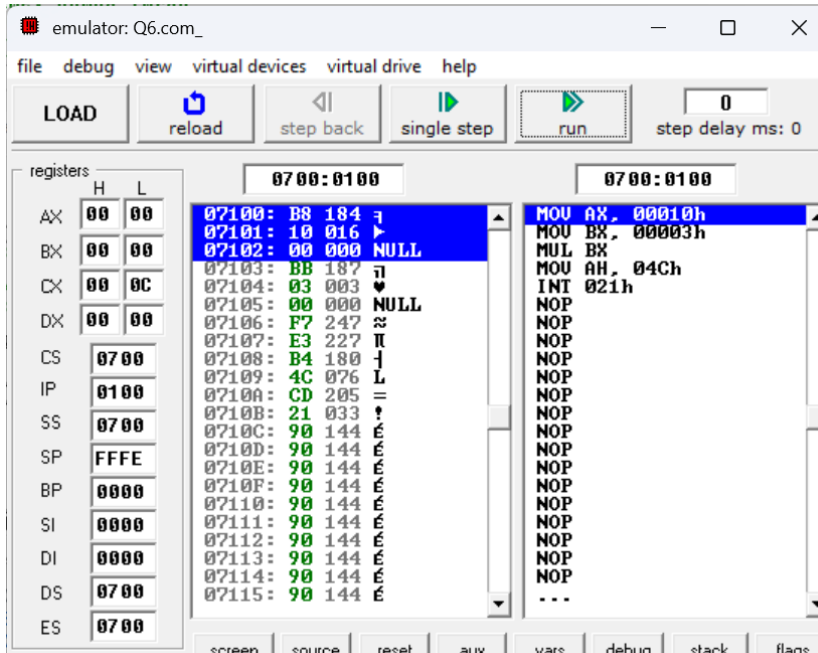
edit: D:\Assembly Language\Lab 4\Q6.asm
file  edit  bookmarks  assembler  help

NEW  OPEN  SAVE  ASSEMBLE  RUN

01 ; Name: Humna Imran
02 ; Roll Number: 2023-BS-AI-017
03 ; Lab 4 - Question 6
04 ; Task: Multiply 10h by 03h
05
06 ORG 100h           ; Start execution at 100h
07
08 MOV AX, 0010h      ; Load Multiplicand into AX
09 MOV BX, 0003h      ; Load Multiplier into BX
10
11 MUL BX             ; Multiply AX by BX
12                     ; Result stored in DX:AX
13                     ; 10h * 03h = 30h (48 decimal)
14
15 MOV AH, 4Ch        ; Terminate program
16 INT 21h            ; DOS interrupt
  
```

Figure 59: Multiplication Code (10h * 03h)

Output



emulator: Q6.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	00
BX	00	00
CX	00	0C
DX	00	00
CS	0700	
IP	0100	
SS	0700	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

0700:0100 0700:0100

Address	Hex	Dec	Symbol
07100	B8	184	MOV AX, 00010h
07101	10	016	MOV BX, 00003h
07102	00	000	MUL BX
07103	BB	187	MOV AH, 04Ch
07104	03	003	INT 021h
07105	00	000	NOP
07106	F7	247	NOP
07107	E3	227	NOP
07108	B4	180	NOP
07109	4C	076	NOP
0710A	CD	205	NOP
0710B	21	033	NOP
0710C	90	144	NOP
0710D	90	144	NOP
0710E	90	144	NOP
0710F	90	144	NOP
07110	90	144	NOP
07111	90	144	NOP
07112	90	144	NOP
07113	90	144	NOP
07114	90	144	NOP
07115	90	144	NOP

screen source reset aux vars debug stack flags

Figure 60: Multiplication Result in DX:AX

Question no 7

Task: The number 100 decimal (64h) is multiplied by 4. The product is stored in DX:AX.

Decimal to Hex Multiplication

```

edit: D:\Assembly Language\Lab 4\Q7.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 ; Name: Humna Imran
02 ; Roll Number: 2023-BS-AI-017
03 ; Lab 4 - Question 7
04 ; Task: Multiply 100 decimal (64h) by 4
05
06 ORG 100h           ; Standard start
07
08 MOV AX, 0064h      ; Load 100 decimal (which is 64h) into AX
09 MOV BX, 0004h      ; Load 4 decimal (04h) into BX
10
11 MUL BX             ; Multiply AX by BX (100 * 4 = 400)
12                   ; 400 in Hex is 190h
13                   ; Result: DX=0000h, AX=0190h
14
15 MOV AH, 4Ch        ; Exit code
16 INT 21h           ; Interrupt
  
```

Figure 61: Code Converting Decimal 100 to Hex and Multiplying

Output

emulator: Q7.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	00
BX	00	00
CX	00	0C
DX	00	00
CS	0700	
IP	0100	
SS	0700	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

0700:0100

Address	Hex	Decimal	Symbol
07100	B8	184	j
07101	64	100	d
07102	00	000	NULL
07103	BB	187	j
07104	04	004	♦
07105	00	000	NULL
07106	F7	247	≈
07107	E3	227	π
07108	B4	180	j
07109	4C	076	L
0710A	CD	205	=
0710B	21	033	!
0710C	90	144	é
0710D	90	144	é
0710E	90	144	é
0710F	90	144	é
07110	90	144	é
07111	90	144	é
07112	90	144	é
07113	90	144	é
07114	90	144	é
07115	90	144	é

0700:0100

```

MOV AX, 0064h
MOV BX, 0004h
MUL BX
MOV AH, 04Ch
INT 021h
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
...
  
```

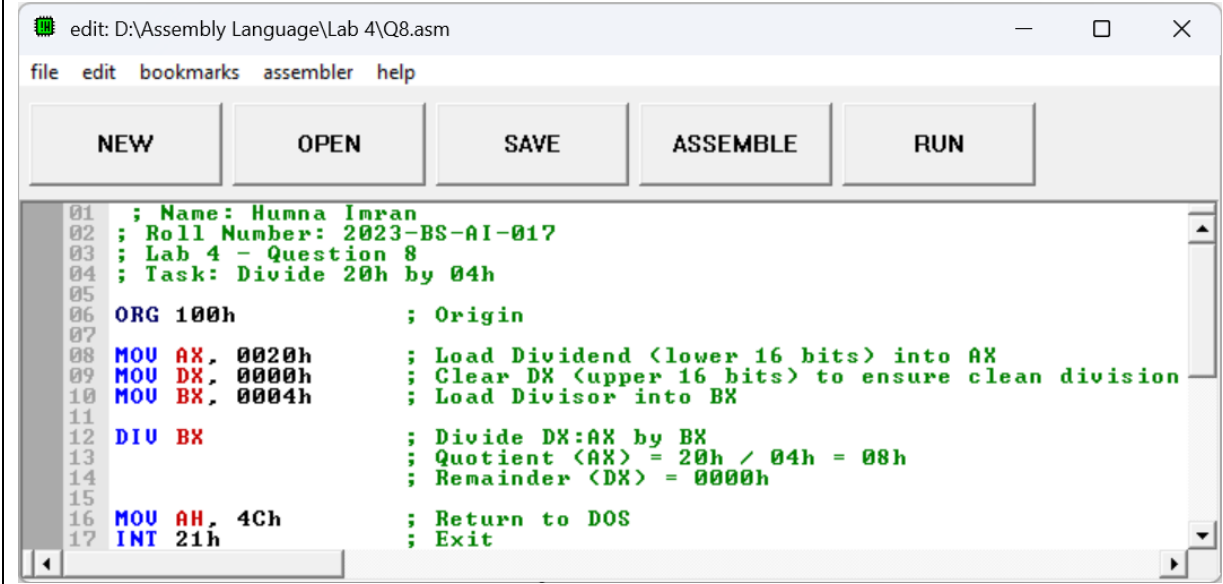
screen source reset aux vars debug stack flags

Figure 62: Result showing 400 (Decimal) as 0190h

Question no 8

Task: 20h is divided by 04h. Quotient stored in AX, Remainder in DX.

Simple Division



```

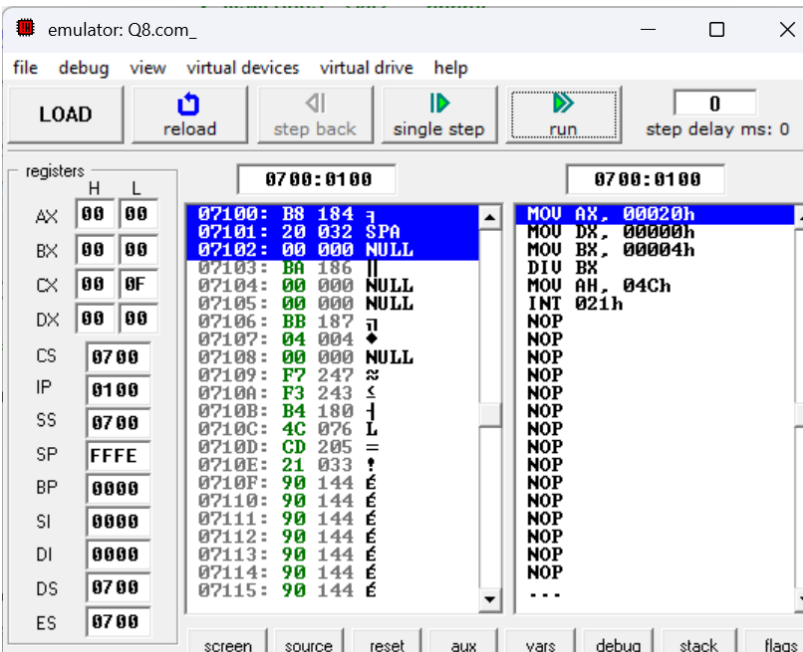
edit: D:\Assembly Language\Lab 4\Q8.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 ; Name: Humna Imran
02 ; Roll Number: 2023-BS-AI-017
03 ; Lab 4 - Question 8
04 ; Task: Divide 20h by 04h
05
06 ORG 100h ; Origin
07
08 MOV AX, 0020h ; Load Dividend (lower 16 bits) into AX
09 MOV DX, 0000h ; Clear DX (upper 16 bits) to ensure clean division
10 MOV BX, 0004h ; Load Divisor into BX
11
12 DIV BX ; Divide DX:AX by BX
13 ; Quotient (AX) = 20h / 04h = 08h
14 ; Remainder (DX) = 0000h
15
16 MOV AH, 4Ch ; Return to DOS
17 INT 21h ; Exit
  
```

Figure 63: Division Code (20h / 04h)

Output



emulator: Q8.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	00
BX	00	00
CX	00	0F
DX	00	00
CS	07	00
IP	01	00
SS	07	00
SP	FF	FE
BP	00	00
SI	00	00
DI	00	00
DS	07	00
ES	07	00

0700:0100 0700:0100

07100: B8 184 j
 07101: 20 032 SP0
 07102: 00 000 NULL
 07103: BA 186 j
 07104: 00 000 NULL
 07105: 00 000 NULL
 07106: BB 187 j
 07107: 04 004 j
 07108: 00 000 NULL
 07109: F7 247 s
 0710A: F3 243 s
 0710B: B4 180 j
 0710C: 4C 076 L
 0710D: CD 205 =
 0710E: 21 033 j
 0710F: 90 144 e
 07110: 90 144 e
 07111: 90 144 e
 07112: 90 144 e
 07113: 90 144 e
 07114: 90 144 e
 07115: 90 144 e
 ...

MOV AX, 00020h
 MOV DX, 00000h
 MOV BX, 00004h
 DIV BX
 MOV AH, 04Ch
 INT 021h
 NOP
 NOP
 NOP
 NOP
 NOP
 NOP
 NOP
 NOP
 NOP
 NOP
 ...

screen source reset aux vars debug stack flags

Figure 64: Quotient and Remainder Verification

Question no 9

Task: 1Ah (26 decimal) is divided by 05h (5 decimal). Quotient in AX, Remainder in DX.

Division with Remainder

```

edit: D:\Assembly Language\Lab 4\Q9.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 ; Name: Humna Imran
02 ; Roll Number: 2023-BS-AI-017
03 ; Lab 4 - Question 9
04 ; Task: Divide 1Ah (26) by 05h (5)
05
06 ORG 100h ; Program start
07
08 MOV AX, 001Ah ; Load 26 decimal (1Ah) into AX
09 MOV DX, 0000h ; Clear high-word register DX
10 MOV BX, 0005h ; Load 5 decimal (05h) into BX
11
12 DIV BX ; Perform Division
13 ; 26 / 5 = 5 with remainder 1
14 ; AX (Quotient) = 0005h
15 ; DX (Remainder) = 0001h
16
17 MOV AH, 4Ch ; Exit setup
18 INT 21h ; Execute exit
  
```

Figure 65: Code for Dividing 26 (Decimal) by 5

Output

emulator: Q9.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	05
BX	00	00
CX	00	0F
DX	00	01
CS	0700	
IP	0100	
SS	0700	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

0700:0100 0700:0100

Address	Hex	Dec	Symbol
07100	B8	184	MOV AX, 0001Ah
07101	1A	26	MOV DX, 0000h
07102	00	000	MOV BX, 0005h
07103	00	000	DIV BX
07104	00	000	MOV AH, 04Ch
07105	00	000	INT 021h
07106	BB	187	NOP
07107	05	005	NOP
07108	00	000	NOP
07109	F7	247	NOP
0710A	F3	243	NOP
0710B	B4	180	NOP
0710C	4C	076	NOP
0710D	CD	205	NOP
0710E	21	033	NOP
0710F	90	144	NOP
07110	90	144	NOP
07111	90	144	NOP
07112	90	144	NOP
07113	90	144	NOP
07114	90	144	NOP
07115	90	144	NOP

screen source reset aux vars debug stack flags

Figure 66: Result showing Quotient 5 and Remainder 1

Question no 10

Task: 1000h is divided by 10h. Quotient in AX, Remainder in DX.

Large Number Division

```

edit: D:\Assembly Language\Lab 4\Q10.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 ; Name: Humna Imran
02 ; Roll Number: 2023-BS-AI-017
03 ; Lab 4 - Question 10
04 ; Task: Divide 1000h by 10h
05
06 ORG 100h          ; Origin 100h
07
08 MOV AX, 1000h      ; Load Dividend 1000h into AX
09 MOV DX, 0000h      ; Clear DX (Essential for accurate 16-bit division)
10 MOV BX, 0010h      ; Load Divisor 10h into BX
11
12 DIV BX             ; Divide DX:AX by BX
13                   ; 1000h / 10h = 100h
14                   ; AX (Quotient) = 0100h
15                   ; DX (Remainder) = 0000h
16
17 MOV AH, 4Ch        ; Exit function
18 INT 21h           ; DOS Interrupt
  
```

Figure 67: Division Code for Large Hex Number (1000h)

Output

emulator: Q10.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	00
BX	00	00
CX	00	0F
DX	00	00
CS	0700	
IP	0100	
SS	0700	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

0700:0100 0700:0100

```

07100: B8 184  MOV AX, 01000h
07101: 00 000  MOV DX, 00000h
07102: 10 016  MOV BX, 00010h
07103: BA 186  DIV BX
07104: 00 000  MOV AH, 04Ch
07105: 00 000  INT 021h
07106: BB 187  NOP
07107: 10 016  NOP
07108: 00 000  NOP
07109: F7 247  NOP
0710A: F3 243  NOP
0710B: B4 180  NOP
0710C: 4C 076  NOP
0710D: CD 205  NOP
0710E: 21 033  NOP
0710F: 90 144  NOP
07110: 90 144  NOP
07111: 90 144  NOP
07112: 90 144  NOP
07113: 90 144  NOP
07114: 90 144  NOP
07115: 90 144  ...
  
```

screen source reset aux vars debug stack flags

Figure 68: Final Division Output in Registers

LAB 5

Addressing Modes

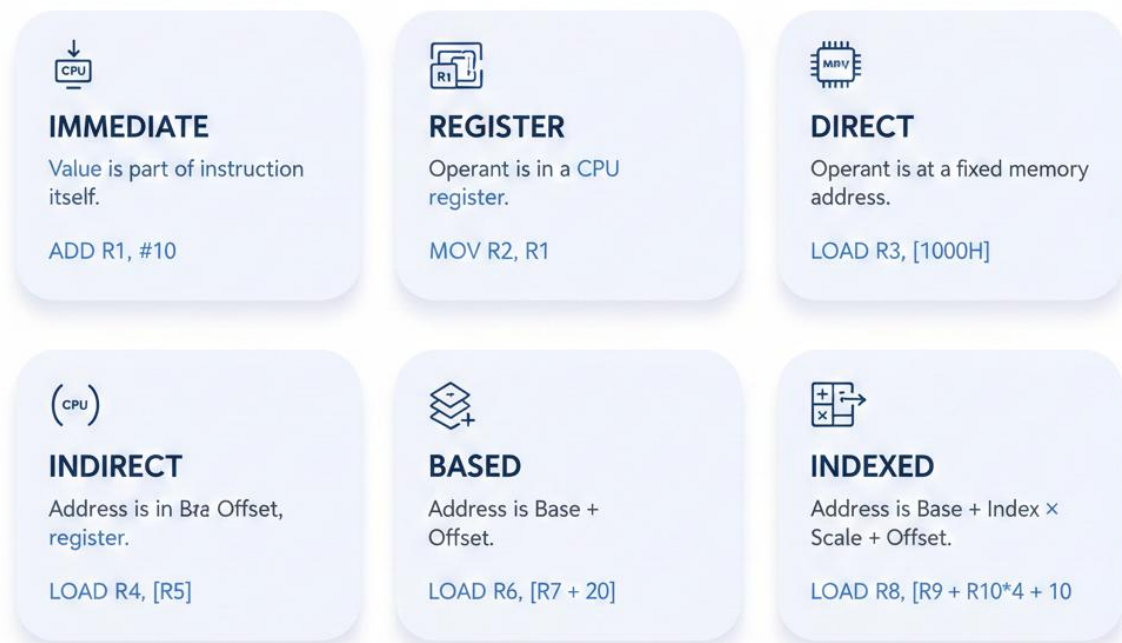


Figure 69: CPU Addressing Modes Overview

Immediate Addressing Mode

In this mode, the operand is an **integral part of the instruction itself**. No memory or register lookup is required to find the data.

- **Internal Process:** The value is fetched during the instruction fetch phase from the code segment, not the data segment.
- **Use Case:** Defining constants, flags, or bitmasks.

Example

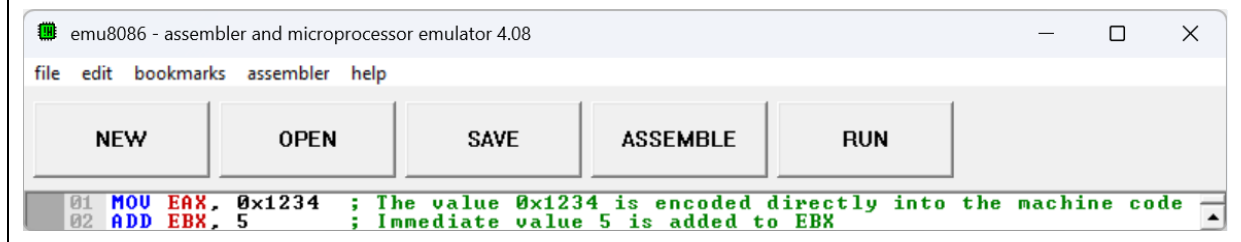


Figure 70: Immediate Addressing Mode Code Example

Output: If you inspect the hex code of the executable, you will see 34 12 (in little-endian) embedded right after the opcode for MOV.

Register Addressing Mode

The operand is located in one of the **Internal CPU Registers**.

- **Internal Process:** The CPU accesses its internal register file. This is the "gold standard" for speed because it avoids the system bus entirely.
- **Use Case:** Intermediate arithmetic and high-speed data manipulation.

Example

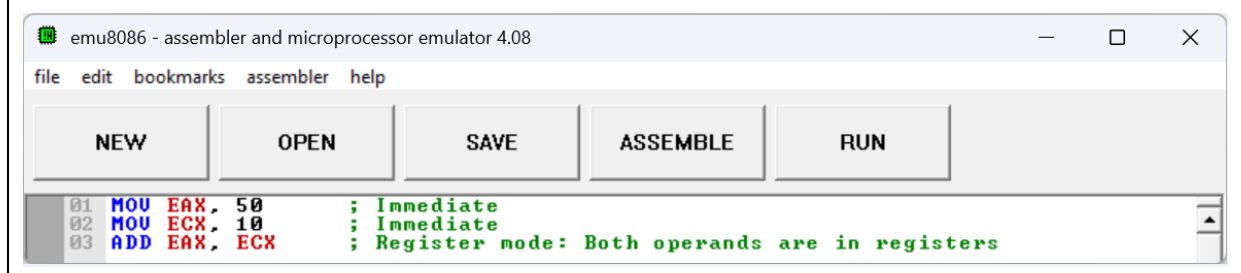


Figure 71: Register Addressing Mode Code Example

Output: After execution, EAX will hold 60. The operation is nearly instantaneous (usually 1 clock cycle).

Direct (Absolute) Addressing Mode

The instruction encodes the **Effective Address (EA)** of the operand. The CPU goes directly to that memory location.

- **Internal Process:** The CPU reads the address from the instruction, places it on the Address Bus, and reads the data from the Data Bus.

Example



```

01 section .data
02     status_flag db 1    ; A single byte initialized to 1
03
04 section .text
05     MOV AL, [status_flag] ; Directly access the memory address of status_flag
  
```

Figure 72: Direct (Absolute) Addressing Mode Code Example

Output: AL will contain 01. If the memory address of status_flag is 0x403000, the instruction effectively tells the CPU: "Go to 0x403000 and bring me the byte there."

Register Indirect Addressing Mode

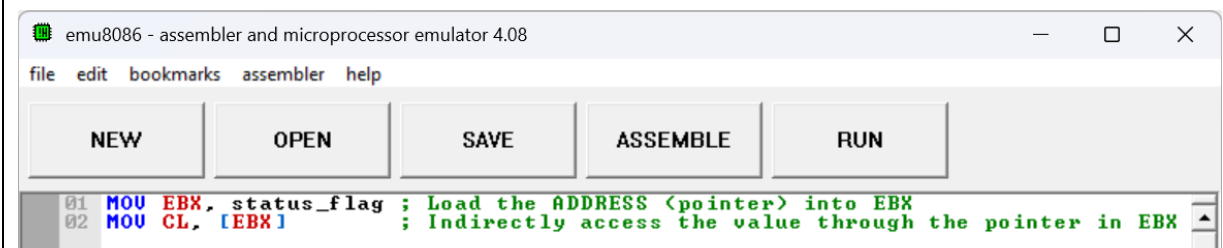
The register doesn't hold the data; it holds a **pointer** (the memory address) to where the data is located.

Internal Process

1. Look inside the register to find the address.
2. Use that address to access RAM.

Use Case: Handling strings and passing large structures by reference.

Example



```

01 MOV EBX, status_flag ; Load the ADDRESS <pointer> into EBX
02 MOV CL, [EBX]        ; Indirectly access the value through the pointer in EBX
  
```

Figure 73: Register Indirect Addressing Mode Code Example

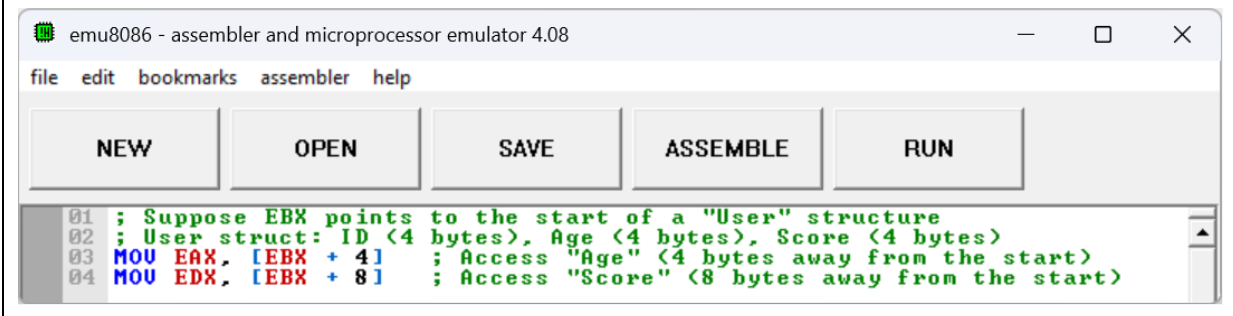
Output: EBX contains the address (e.g., 0x403000), and CL will contain the value stored there.

Displacement (Based) Addressing Mode

This combines a register with a constant value (displacement) to find the address.

- **Formula:** Effective\ Address = [Register] + Constant
- **Use Case:** Accessing fields within a struct or a Record.

Example



```
01 ; Suppose EBX points to the start of a "User" structure
02 ; User struct: ID (4 bytes), Age (4 bytes), Score (4 bytes)
03 MOV EAX, [EBX + 4] ; Access "Age" (4 bytes away from the start)
04 MOV EDI, [EBX + 8] ; Access "Score" (8 bytes away from the start)
```

Figure 74: Displacement (Based) Addressing Mode Code Example

Scaled Indexed Addressing Mode

Unique to x86, this mode allows you to multiply an index register by a scale factor (1, 2, 4, or 8) and add it to a base.

- **Formula:** Effective\ Address = Base + (Index \times Scale) + Displacement
- **Use Case:** Iterating through arrays of integers (4 bytes) or doubles (8 bytes).

Example

```

01 section .data
02     my_array dd 10, 20, 30, 40, 50
03
04 section .text
05     MOV EBX, my_array ; Base address
06     MOV ESI, 2        ; We want the 3rd element (index 2)
07     MOV EAX, [EBX + ESI*4] ; Scale by 4 because each 'dd' is 4 bytes
  
```

Figure 75: Scaled Indexed Addressing Mode Code Example

Output: EAX will contain 30. The CPU does the math (Base + 2 \times 4) automatically in one instruction.

Summary for Lab Submission

Mode	Effective Address (EA) Calculation	Example Syntax
Immediate	Data is in instruction	MOV EAX, 10
Register	Data is in register	MOV EAX, EBX
Direct	EA = Constant	MOV EAX, [0x4000]
Indirect	EA = [Register]	MOV EAX, [EBX]
Based	EA = [Base] + Disp	MOV EAX, [EBP - 8]
Indexed	EA = [Base] + [Index \times Scale] + Disp	MOV EAX, [EBX + ESI*4 + 12]

Figure 76: Summary Table of Addressing Modes and Effective Address Calculations

Question No 1

Task: Multiply 03h by 4 using SHL, then add 02h to the result and store it in AL

Bitwise Multiplication and Addition

```

edit: D:\Assembly Language\Lab 7\Q1.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 org 100h
02
03 MOV AL, 03h      ; Load 03h into AL
04 MOV CL, 2       ; Set shift count to 2 (Shifting left by 2 multiplies by 4)
05
06 SHL AL, CL      ; Multiply: 03h * 4 = 0Ch (12 decimal)
07
08 ADD AL, 02h     ; Add 02h: 0Ch + 02h = 0Eh (14 decimal)
09
10 MOV AH, 4Ch     ; Return control to OS
11 INT 21h
  
```

Figure 77: Code for Bitwise Multiplication

Output

emulator: Q1.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	00
BX	00	00
CX	00	0C
DX	00	00
CS	0700	
IP	0100	
SS	0700	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

0700:0100 0700:0100

Address	Hex	Dec	Symbol
07100	B0	176	
07101	03	003	
07102	B1	177	
07103	02	002	
07104	D2	210	
07105	E0	224	
07106	04	004	
07107	02	002	
07108	B4	180	
07109	4C	076	
0710A	CD	205	
0710B	21	033	
0710C	90	144	
0710D	90	144	
0710E	90	144	
0710F	90	144	
07110	90	144	
07111	90	144	
07112	90	144	
07113	90	144	
07114	90	144	
07115	90	144	

MOV AL, 03h
MOV CL, 02h
SHL AL, CL
ADD AL, 02h
MOV AH, 04Ch
INT 021h
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
...

screen source reset aux vars debug stack flags

Figure 78: Register Window Showing AL Result 0E

Question No 2

Task: Divide 40h by 2 using SHR, then compare the result with 15h and jump to a label if it is greater.

Division and Conditional Jump



```

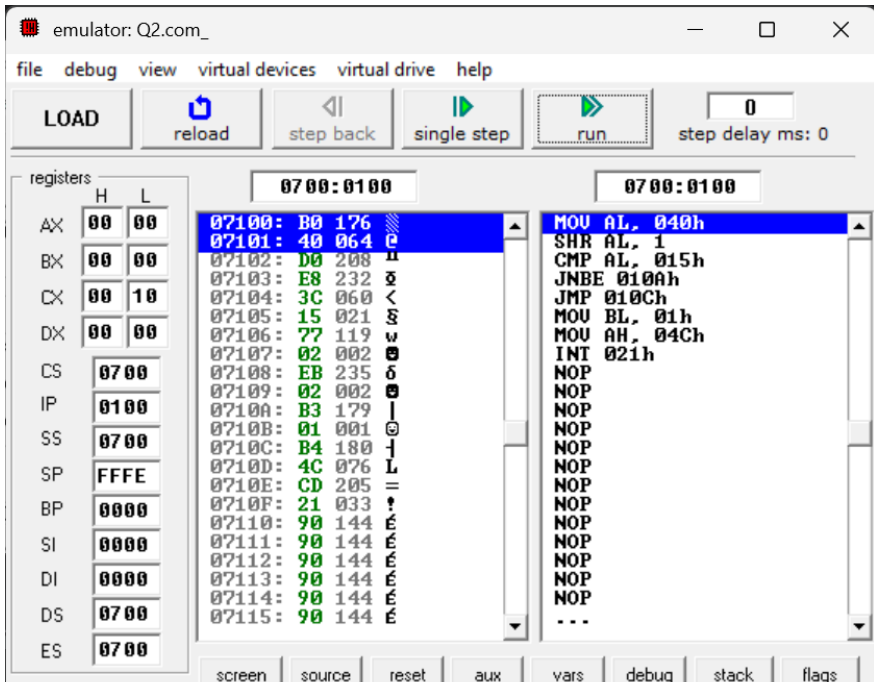
edit: D:\Assembly Language\Lab 7\Q2.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 org 100h
02
03 MOV AL, 40h      ; Load 40h into AL
04 SHR AL, 1        ; Divide by 2 (Shift Right by 1): 40h / 2 = 20h
05
06 CMP AL, 15h      ; Compare result (20h) with 15h
07 JA ItIsGreater   ; Jump if Above (Greater) because 20h > 15h
08
09 JMP Stop         ; Skip if not greater
10
11 ItIsGreater:
12 MOV BL, 01h      ; Marker: Move 1 to BL to show we jumped successfully
13
14 Stop:
15 MOV AH, 4Ch      ; Exit program
16 INT 21h
  
```

Figure 79: Code for Division and Comparison

Output



emulator: Q2.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	00
BX	00	00
CX	00	10
DX	00	00
CS	0700	
IP	0100	
SS	0700	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

0700:0100

07100:	B0	176	
07101:	40	064	
07102:	D0	208	
07103:	E8	232	
07104:	3C	060	
07105:	15	021	
07106:	77	119	
07107:	02	002	
07108:	EB	235	
07109:	02	002	
0710A:	B3	179	
0710B:	01	001	
0710C:	B4	180	
0710D:	4C	076	
0710E:	CD	205	
0710F:	21	033	
07110:	90	144	
07111:	90	144	
07112:	90	144	
07113:	90	144	
07114:	90	144	
07115:	90	144	

0700:0100

```

MOV AL, 040h
SHR AL, 1
CMP AL, 015h
JNB 010Ah
JMP 010Ch
MOV BL, 01h
MOV AH, 04Ch
INT 021h
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
...
  
```

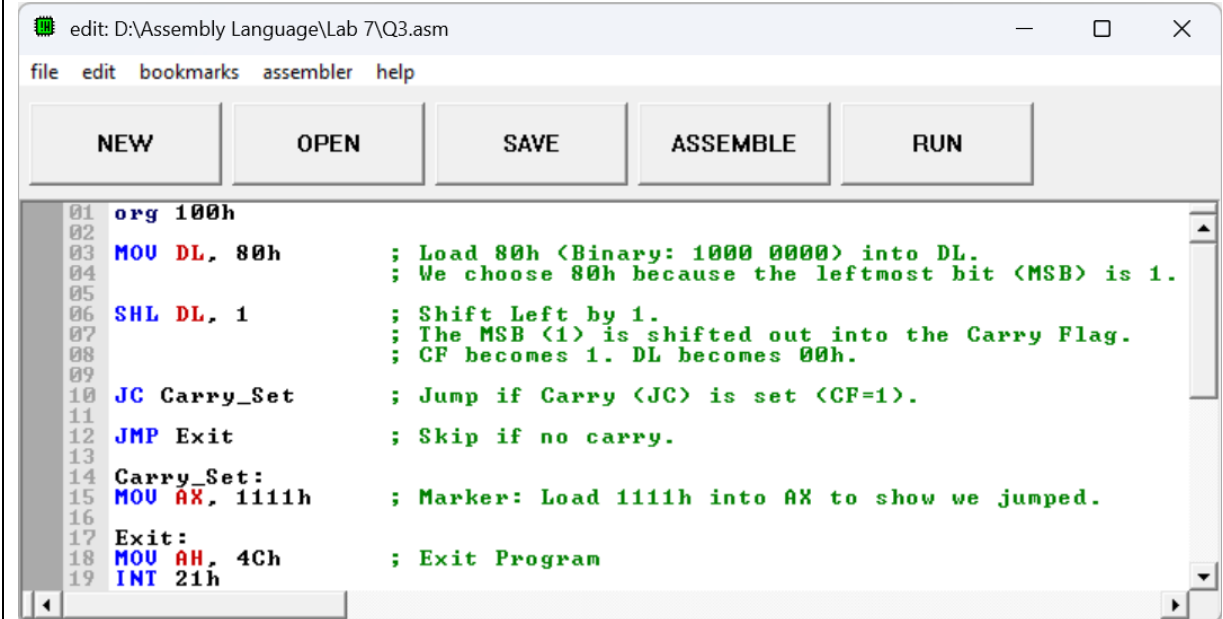
screen source reset aux vars debug stack flags

Figure 80: Result Showing Successful Jump execution

Question no 3

Task: Shift DL left by 1 and jump to a label if the Carry Flag (CF) is set.

Testing the Carry Flag



```

edit: D:\Assembly Language\Lab 7\Q3.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 org 100h
02
03 MOV DL, 80h           ; Load 80h <Binary: 1000 0000> into DL.
04                       ; We choose 80h because the leftmost bit <MSB> is 1.
05
06 SHL DL, 1            ; Shift Left by 1.
07                       ; The MSB <1> is shifted out into the Carry Flag.
08                       ; CF becomes 1. DL becomes 00h.
09
10 JC Carry_Set         ; Jump if Carry <JC> is set <CF=1>.
11
12 JMP Exit             ; Skip if no carry.
13
14 Carry_Set:
15 MOV AX, 1111h        ; Marker: Load 1111h into AX to show we jumped.
16
17 Exit:
18 MOV AH, 4Ch          ; Exit Program
19 INT 21h
  
```

Figure 81: Code demonstrating Jump on Carry

Output

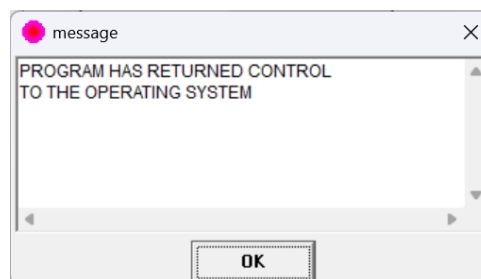


Figure 82: Emulator showing BL=FF (Carry detected)

Question no 5

Task: Shift AL left by 1. If CF=1 subtract 01h from AL and store the result; otherwise leave AL unchanged.

Conditional Subtraction

```

edit: D:\Assembly Language\Lab 7\Q5.asm
file edit bookmarks assembler help

NEW OPEN SAVE ASSEMBLE RUN

01 ORG 100h
02
03 MOV AL, 85h           ; Load 85h (Binary: 1000 0101)
04                       ; MSB is 1, so Shift Left will set Carry.
05
06 SHL AL, 1            ; Shift Left.
07                       ; New AL = 0Ah (0000 1010), Carry Flag = 1
08
09 JNC SkipSub           ; Jump if No Carry (Skip subtraction)
10
11 ; Logic: If we are here, Carry is 1
12 SUB AL, 01h           ; Subtract 1 from AL (0Ah - 01h = 09h)
13
14 SkipSub:
15 MOV AH, 4Ch
16 INT 21h
17 RET
  
```

Figure 85: Code for Conditional Subtraction logic

Output

emulator: Q5.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	00
BX	00	00
CX	00	00
DX	00	00
CS	0700	
IP	0100	
SS	0700	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

0700:0100 0700:0100

Address	Hex	Dec	Symbol
07100	B0	176	
07101	85	133	a
07102	D0	208	n
07103	E0	224	α
07104	73	115	s
07105	02	002	0
07106	2C	044	,
07107	01	001	@
07108	B4	180	l
07109	4C	076	L
0710A	CD	205	=
0710B	21	033	!
0710C	C3	195	t
0710D	90	144	e
0710E	90	144	e
0710F	90	144	e
07110	90	144	e
07111	90	144	e
07112	90	144	e
07113	90	144	e
07114	90	144	e
07115	90	144	e

MOV AL, 085h
SHL AL, 1
JNB 0108h
SUB AL, 01h
MOV AH, 04Ch
INT 021h
RET
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
NOP
...

screen source reset aux vars debug stack flags

Figure 86: Emulator state showing AL = 09

LAB 6

Display a Message

Objectives

By the end of this lab, students will be able to:

1. Understand the 16-bit Real Mode architecture and how it interacts with the operating system (DOS).
2. Define and manipulate string data in memory using proper directives.
3. Utilize **Software Interrupts (INT 21h)** to display strings and characters.
4. Write, compile, and execute functional assembly programs that generate visible output on the console.

Theoretical Background

A. Environment & Architecture

The EMU8086 emulator simulates an **Intel 8086 microprocessor** running in **16-bit Real Mode**. In this mode, the processor creates addresses using a segmented memory model. The memory is divided into segments, each handled by specific registers:

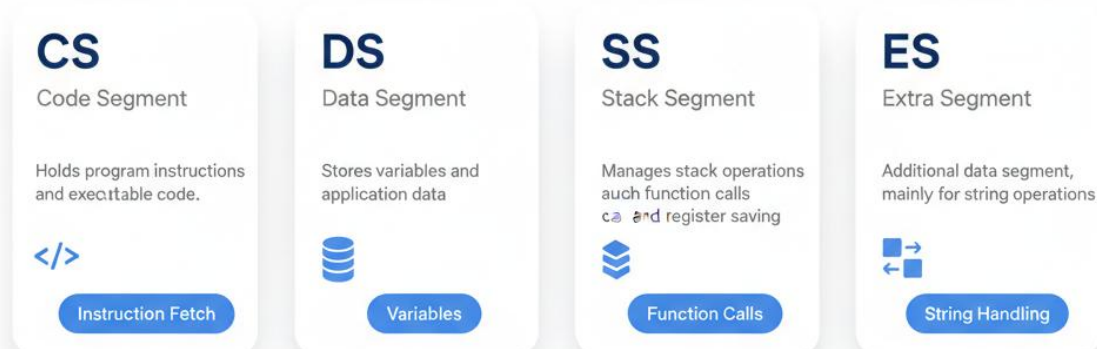


Figure 87: 8086 Memory Segment Register Roles

Unlike high-level languages (Python/C++), Assembly does not have built-in "print" functions. Instead, it relies on **Interrupt Services** provided by the Disk Operating System (DOS).

B. The Data Section

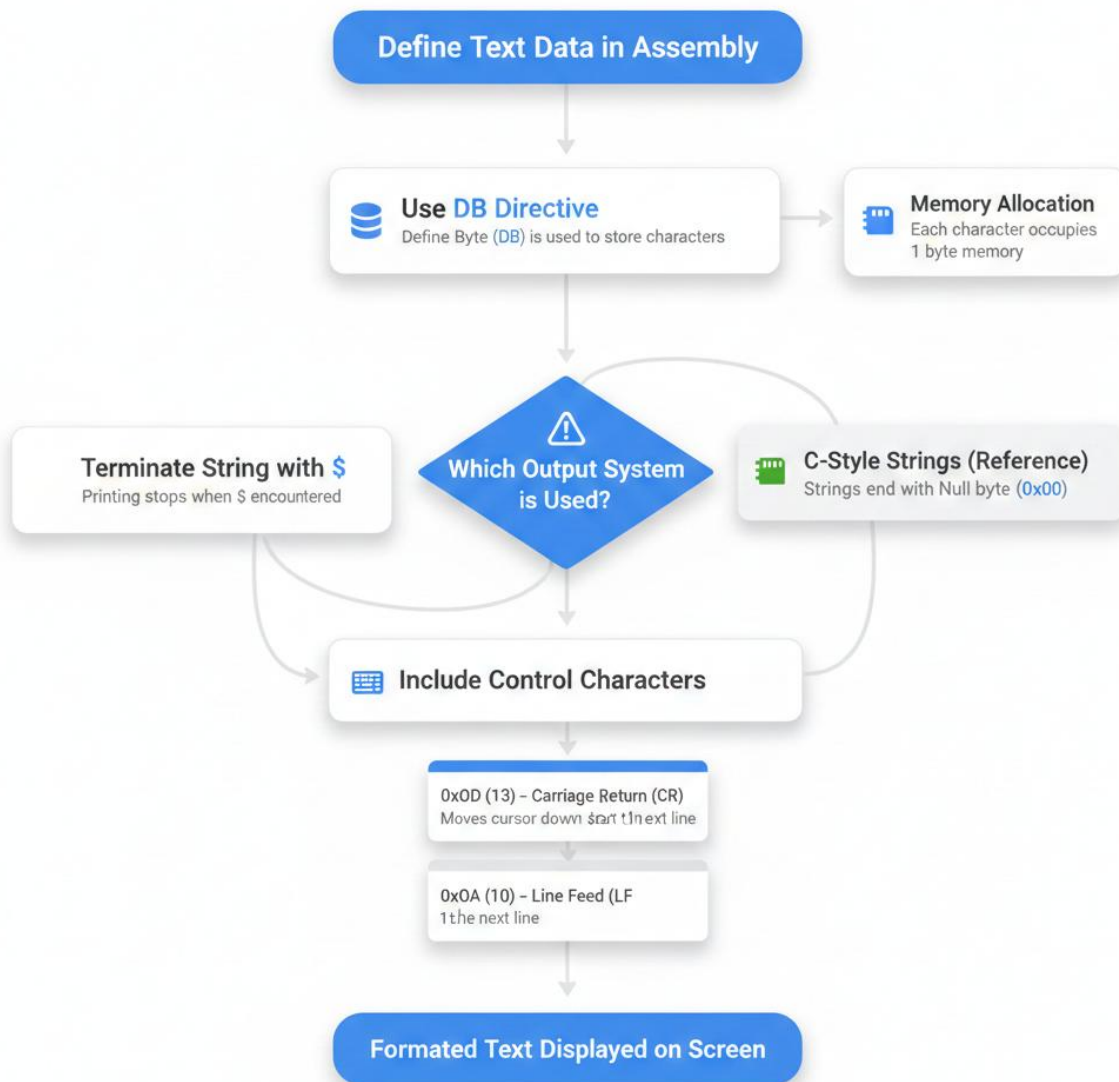


Figure 88: Text Data Definition and Formatting Process

C. System Calls & Interrupts

Communication between your program and the hardware/OS is handled via **Interrupts**.

- **INT 21h:** This is the main DOS interrupt for Input/Output operations.
- To use it, you must place specific values into CPU registers to tell the OS *which* function to perform.

Key Functions:

1. **Function 09h (Display String):** Outputs a \$ terminated string.
2. **Function 02h (Display Character):** Outputs a single character found in the DL register.
3. **Function 4Ch (Terminate Program):** Safely exits the program and returns control to the OS.

D. Registers & Workflow

The 8086 CPU has general-purpose registers (AX, BX, CX, DX) that are 16-bits wide, split into High (H) and Low (L) 8-bit portions (e.g., AH and AL).

Register	Role
 AH	Holds the Function ID (e.g., 09h for print string)
 DS	Holds the Segment Address where the string is stored
 DX	Holds the Offset Address (location of the string within the Data Segment)

Figure 89: Register Usage for DOS String Output (INT 21h)

Standard Workflow

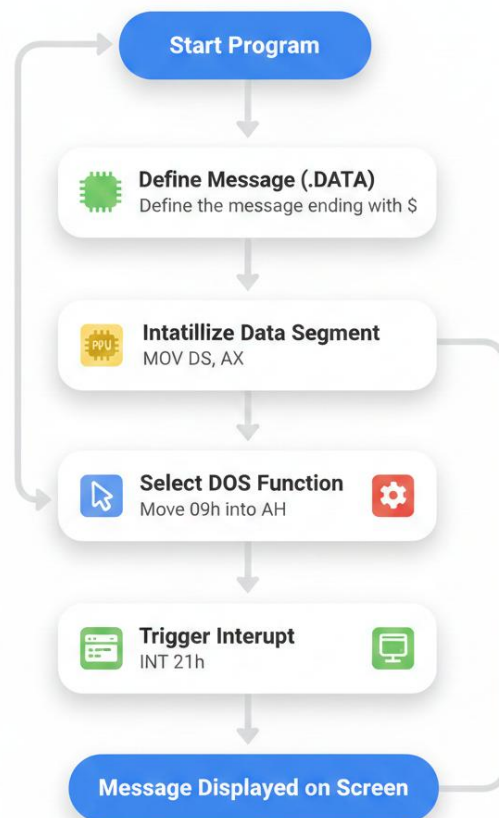


Figure 90: Standard Assembly Program Workflow for Message Display

3. Comparison: DOS (16-bit) vs. Linux (64-bit)

It is crucial to understand that assembly syntax changes based on the Operating System and Architecture.

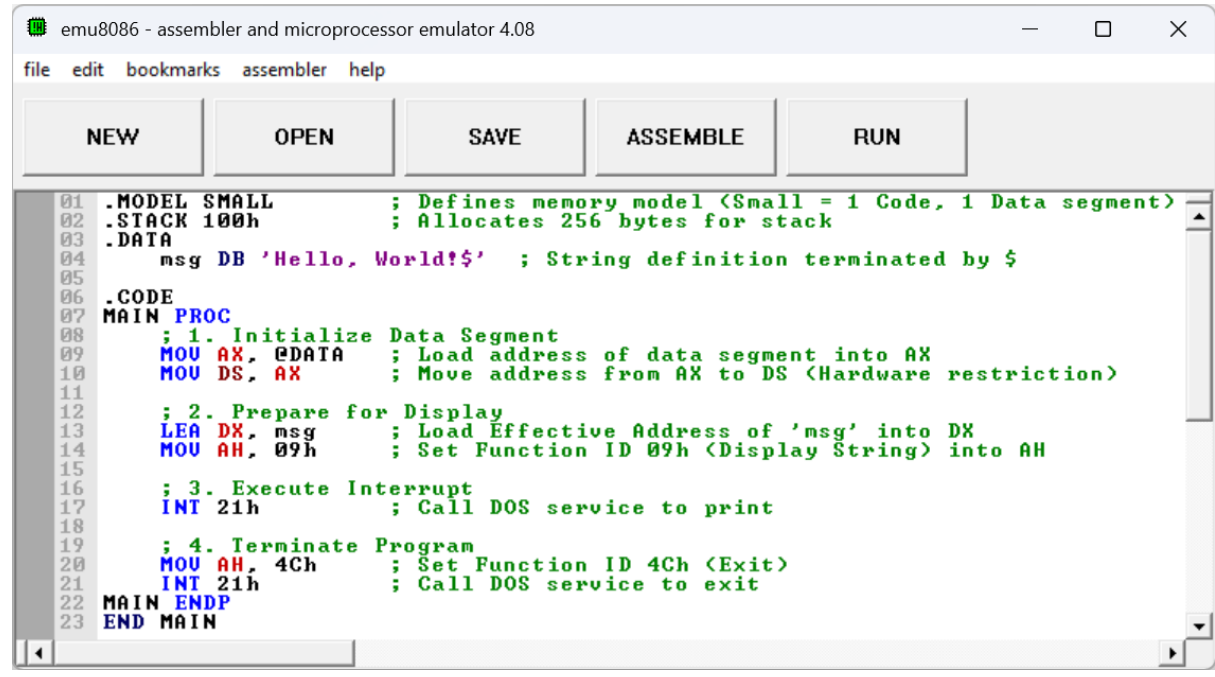
 Feature	DOS (8086 / EMU8086)	Linux (x86-64 / NASM)
 Trigger Mechanism	INT 21h	syscall
 Function ID Register	MOV AH, 09h (Display String)	MOV RAX, 1 (sys_write)
 Message Pointer Register	DX	RSI
 String Terminator	\$ (Dollar Sign)	None (Length passed in RDX)
 Output Destination	Screen (Implicit)	File Descriptor 1 (STDOUT)

Figure 91: Comparison Table: DOS (16-bit) vs. Linux (64-bit) System Calls

Question no 1

Task: Write a program to display a static string using Function 09h.

The "Hello, World!" Program



```
01 .MODEL SMALL          ; Defines memory model (Small = 1 Code, 1 Data segment)
02 .STACK 100h          ; Allocates 256 bytes for stack
03 .DATA
04     msg DB 'Hello, World!$' ; String definition terminated by $
05
06 .CODE
07 MAIN PROC
08     ; 1. Initialize Data Segment
09     MOV AX, @DATA      ; Load address of data segment into AX
10     MOV DS, AX         ; Move address from AX to DS (Hardware restriction)
11
12     ; 2. Prepare for Display
13     LEA DX, msg        ; Load Effective Address of 'msg' into DX
14     MOV AH, 09h        ; Set Function ID 09h (Display String) into AH
15
16     ; 3. Execute Interrupt
17     INT 21h           ; Call DOS service to print
18
19     ; 4. Terminate Program
20     MOV AH, 4Ch        ; Set Function ID 4Ch (Exit)
21     INT 21h           ; Call DOS service to exit
22 MAIN ENDP
23 END MAIN
```

Figure 92: Assembly Code for Static String Display

Output



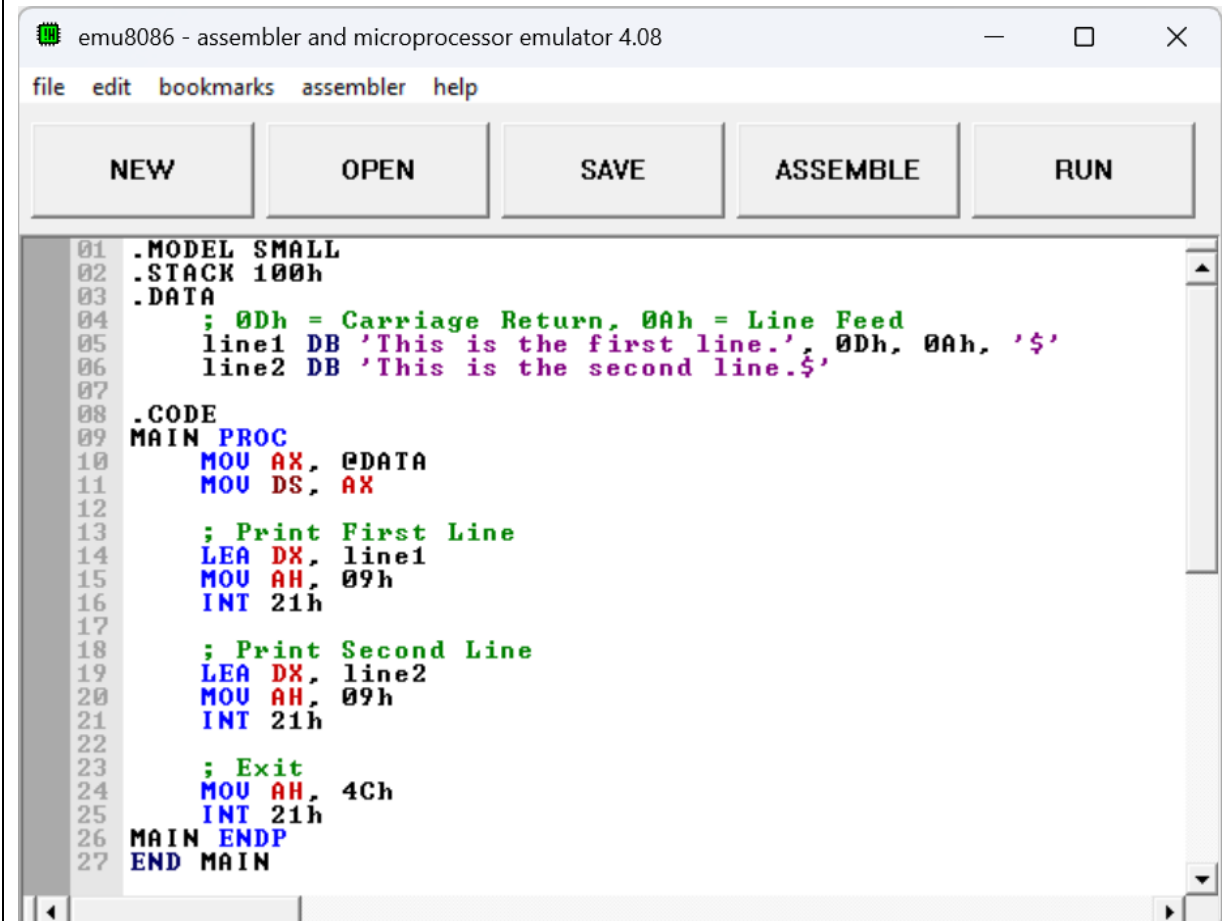
```
emulator screen (41x3 chars)
Hello, World!
clear screen  change font  0/16
```

Figure 93: Emulator Screen Output for Static String

Question no 2

Task: Display text on multiple lines using Carriage Return (0Dh) and Line Feed (0Ah).

Formatting Output (Newlines)

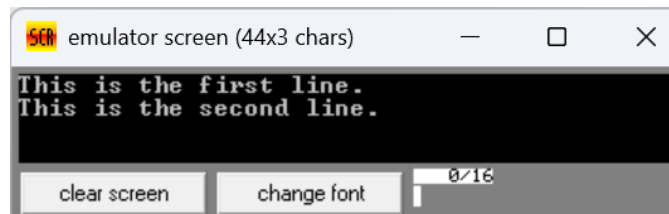


```

01 .MODEL SMALL
02 .STACK 100h
03 .DATA
04 ; 0Dh = Carriage Return, 0Ah = Line Feed
05 line1 DB 'This is the first line.', 0Dh, 0Ah, '$'
06 line2 DB 'This is the second line.$'
07
08 .CODE
09 MAIN PROC
10     MOV AX, @DATA
11     MOV DS, AX
12
13     ; Print First Line
14     LEA DX, line1
15     MOV AH, 09h
16     INT 21h
17
18     ; Print Second Line
19     LEA DX, line2
20     MOV AH, 09h
21     INT 21h
22
23     ; Exit
24     MOV AH, 4Ch
25     INT 21h
26 MAIN ENDP
27 END MAIN
  
```

Figure 94: Assembly Code for Formatting Output with Newlines

Output



```

5Ch emulator screen (44x3 chars)
This is the first line.
This is the second line.
clear screen  change font  0/16
  
```

Figure 95: Emulator Screen Output for Multiline Text

FINALS

LAB 7

Logical Operators

Objectives

By the end of this laboratory session, students will be able to:

1. Understand and implement bitwise logical operations (AND, OR, XOR, NOT, TEST).
2. Apply bitmasking techniques to extract, clear, and set specific bits within a register.
3. Analyze the impact of logical instructions on the EFLAGS register (specifically ZF, SF, PF, CF, and OF).
4. Distinguish between logical shifts, arithmetic shifts, and rotations (SHL, SHR, SAL, SAR, ROL, ROR).
5. Solve practical programming problems such as ASCII case conversion and parity checking.

Truth Table for Logical Operations					
A	B	A AND B	A OR B	A XOR B	NOT A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Figure 96: Truth Table for Fundamental Logical Operations

Pre-Lab: Theoretical Foundations

Before beginning the coding exercises, review the fundamental logic gates and how they map to assembly instructions.

Status Flags (The EFLAGS Register)

Logical operations heavily influence the processor's status flags.

- **Zero Flag (ZF):** Set to 1 if the result of the operation is zero.
- **Sign Flag (SF):** Set to 1 if the most significant bit (MSB) of the result is 1 (indicating a negative number in signed interpretation).
- **Parity Flag (PF):** Set to 1 if the least significant byte of the result contains an even number of 1s.
- **Carry Flag (CF) & Overflow Flag (OF):** Logical instructions (AND, OR, XOR) always **clear** these flags (set to 0).

Instructions: Syntax and Semantics

Core Logical Instructions

The syntax for these instructions is generally OP DESTINATION, SOURCE.

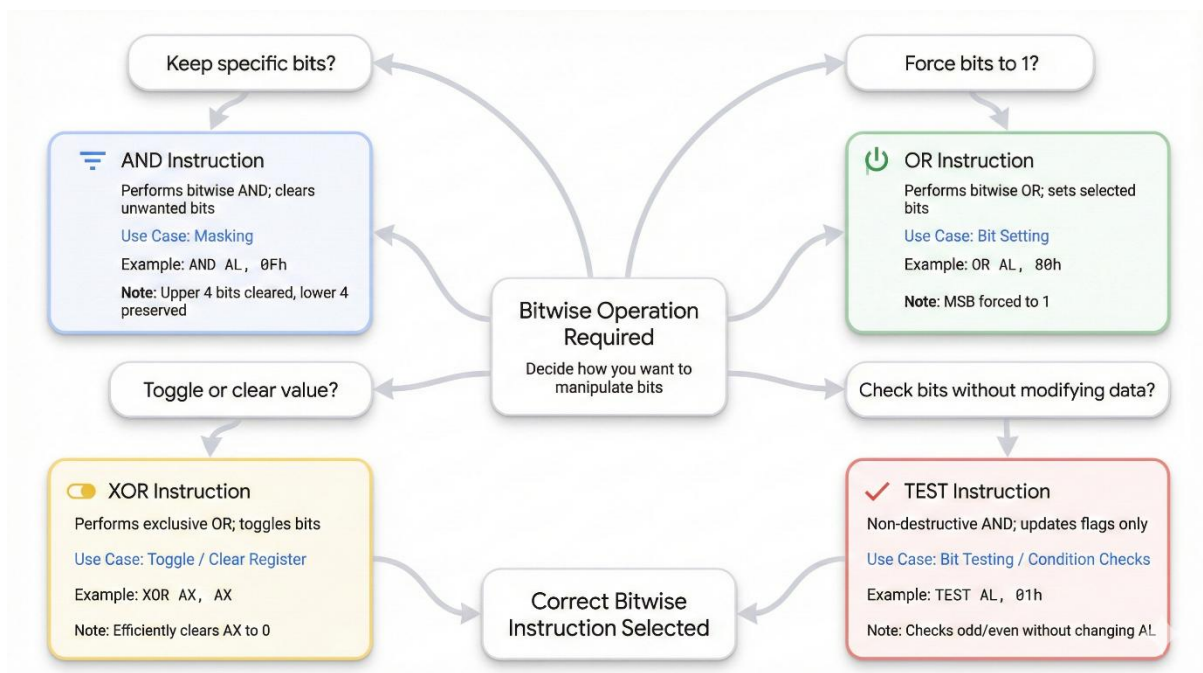


Figure 97: Bitwise Instruction Selection Decision Matrix

Shifts and Rotates

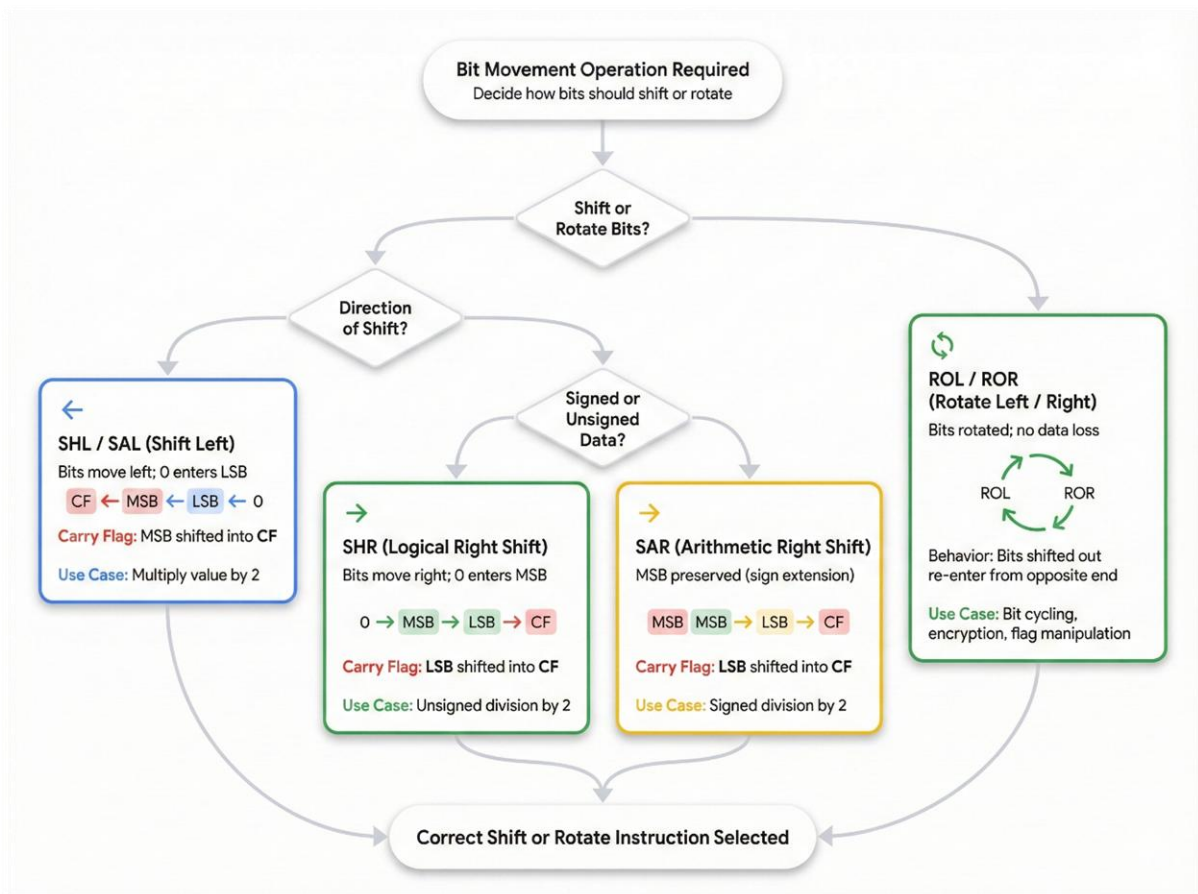
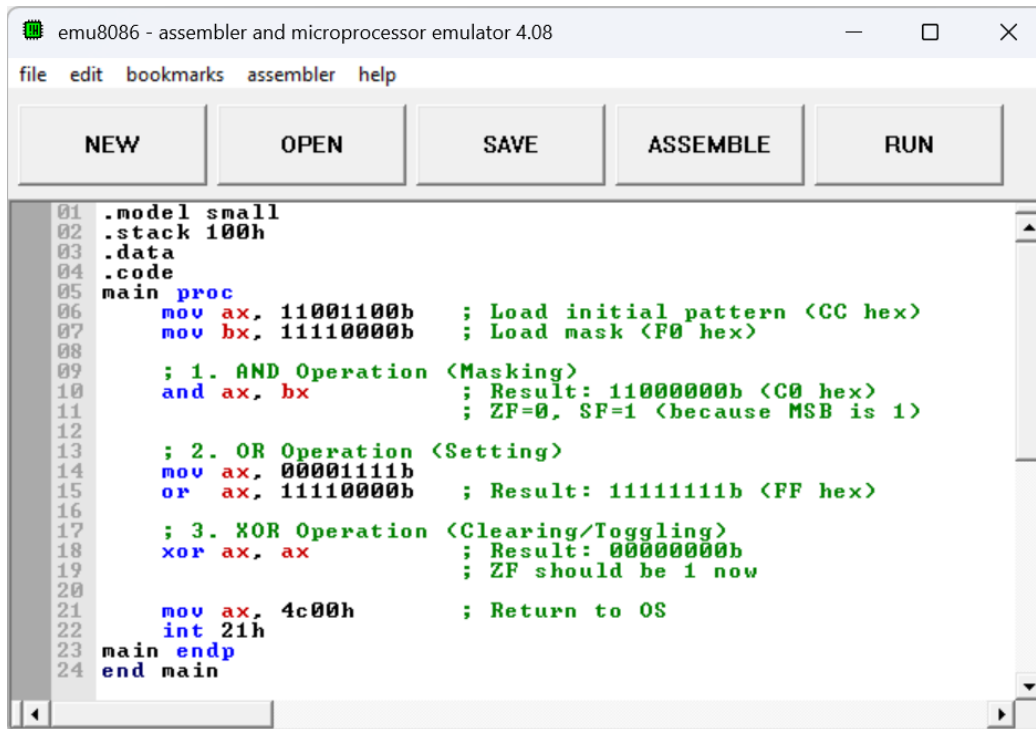


Figure 98: Shift and Rotate Operation Decision Flowchart

Question no 1

Task: Observe how AND, OR, and XOR affect the data and the Zero Flag.

Basic Logical Ops and Flag Observation

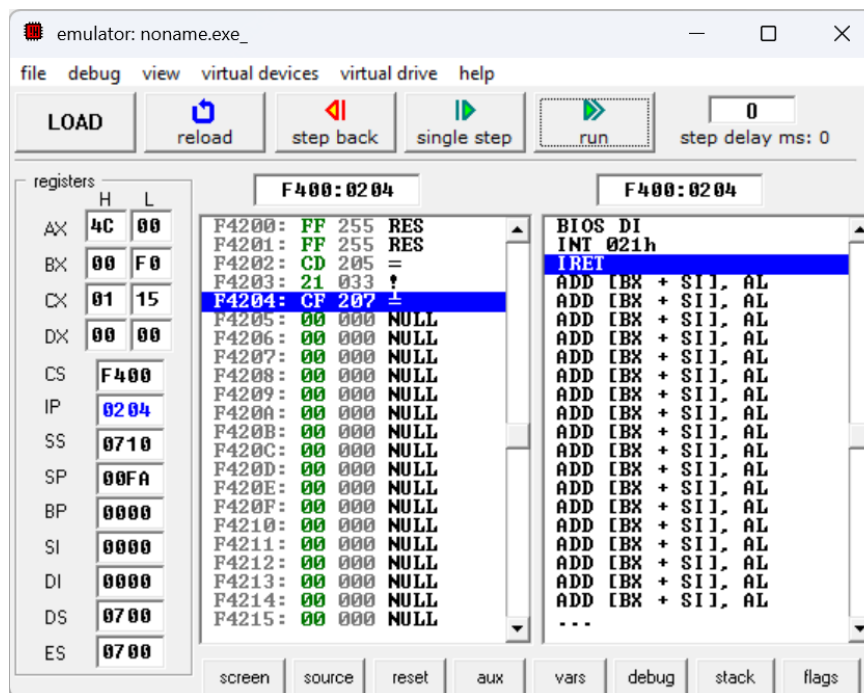


```

01 .model small
02 .stack 100h
03 .data
04 .code
05 main proc
06     mov ax, 11001100b    ; Load initial pattern <CC hex>
07     mov bx, 11110000b    ; Load mask <F0 hex>
08
09     ; 1. AND Operation <Masking>
10     and ax, bx           ; Result: 11000000b <C0 hex>
11                          ; ZF=0, SF=1 (because MSB is 1)
12
13     ; 2. OR Operation <Setting>
14     mov ax, 00001111b
15     or  ax, 11110000b    ; Result: 11111111b <FF hex>
16
17     ; 3. XOR Operation <Clearing/Toggling>
18     xor ax, ax           ; Result: 00000000b
19                          ; ZF should be 1 now
20
21     mov ax, 4c00h        ; Return to OS
22     int 21h
23 main endp
24 end main
  
```

Figure 99: Assembly Code for Logical Operation and Flag Observation

Output



The emulator window shows the final state after execution. The registers are displayed on the left, and the memory dump is on the right.

Register	Value
AX	4C 00
BX	00 F0
CX	01 15
DX	00 00
CS	F400
IP	0204
SS	0710
SP	00FA
BP	0000
SI	0000
DI	0000
DS	0700
ES	0700

The memory dump shows the following values:

Address	Value	Comment
F4200	FF 255	RES
F4201	FF 255	RES
F4202	CD 205	=
F4203	21 033	!
F4204	CF 207	±
F4205	00 000	NULL
F4206	00 000	NULL
F4207	00 000	NULL
F4208	00 000	NULL
F4209	00 000	NULL
F420A	00 000	NULL
F420B	00 000	NULL
F420C	00 000	NULL
F420D	00 000	NULL
F420E	00 000	NULL
F420F	00 000	NULL
F4210	00 000	NULL
F4211	00 000	NULL
F4212	00 000	NULL
F4213	00 000	NULL
F4214	00 000	NULL
F4215	00 000	NULL

The BIOS DI register is also shown, with the value 021h.

Figure 100: Final Emulator State and Register Values for Flag Observation

LAB 8

Array

An array is a collection of elements stored in **contiguous memory locations**, accessed using a **base address** and **index offset**. Unlike high-level languages, assembly does not provide built-in array syntax you manually manage memory addresses.

High-Level vs Assembly Arrays

Array Handling: High-Level Language vs Assembly Language		
Aspect	High-Level Language	Assembly Language
{ } Declaration	<code>int arr[5];</code>	<code>array DW 5 DUP(?)</code>
📁 Access	<code>arr[2]</code>	<code>mov ax, [array+4]</code>
✓ Bounds Checking	Automatic	Manual (none)
🔍 Abstraction	High	None – direct memory access

Figure 101: High-Level Language vs Assembly Language Array Handling Comparison Table

Why Arrays Matter in Assembly

- Efficient data storage: Process multiple values without separate variables
- Hardware-level operations: Direct memory manipulation
- Foundation for strings: Strings are byte arrays
- System programming: OS kernels, drivers use arrays extensively

Memory-Based Reality

Key Concept: In assembly, arrays don't "exist" as structures. They're just labels pointing to the first byte of consecutive memory.

Memory Organization Of Arrays

Contiguous Allocation

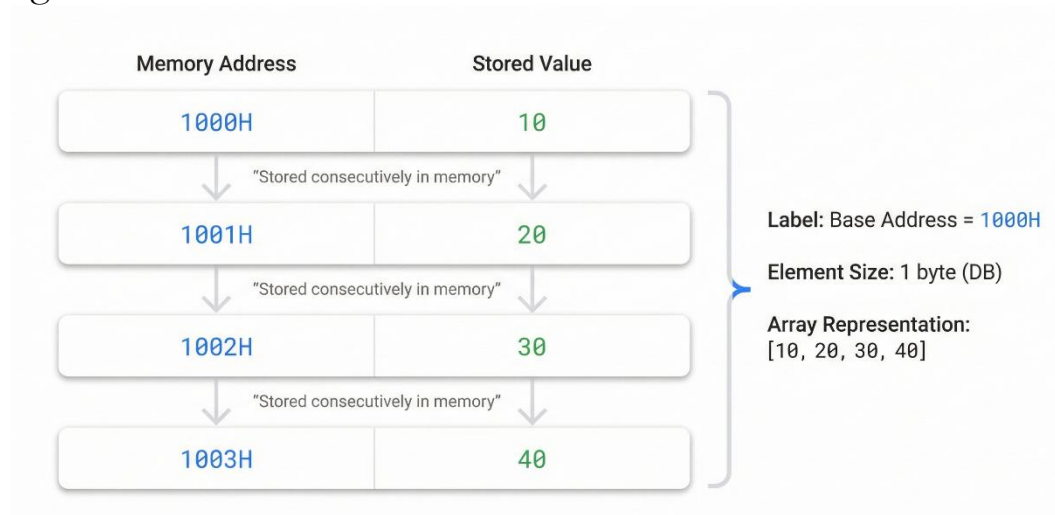


Figure 102: Diagram of Array Storage in Contiguous Memory

Base Address

The starting memory location of the array. All element addresses are calculated relative to this.

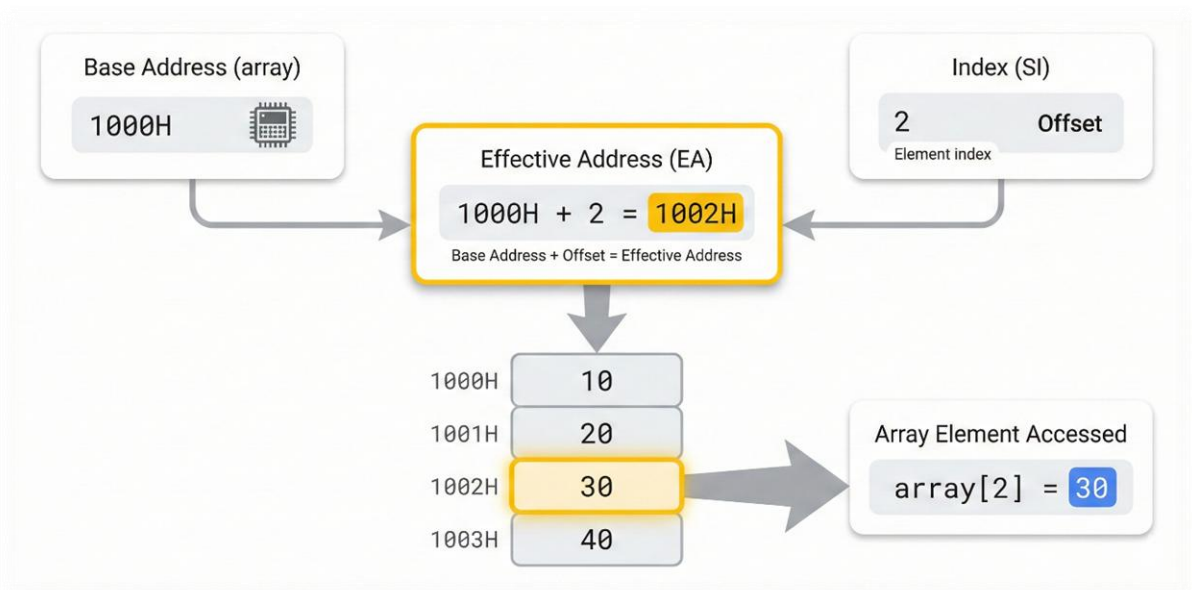


Figure 103: Diagram of Accessing Array Elements Using Base Address and Offset

Accessing Array Elements

Using Index Registers

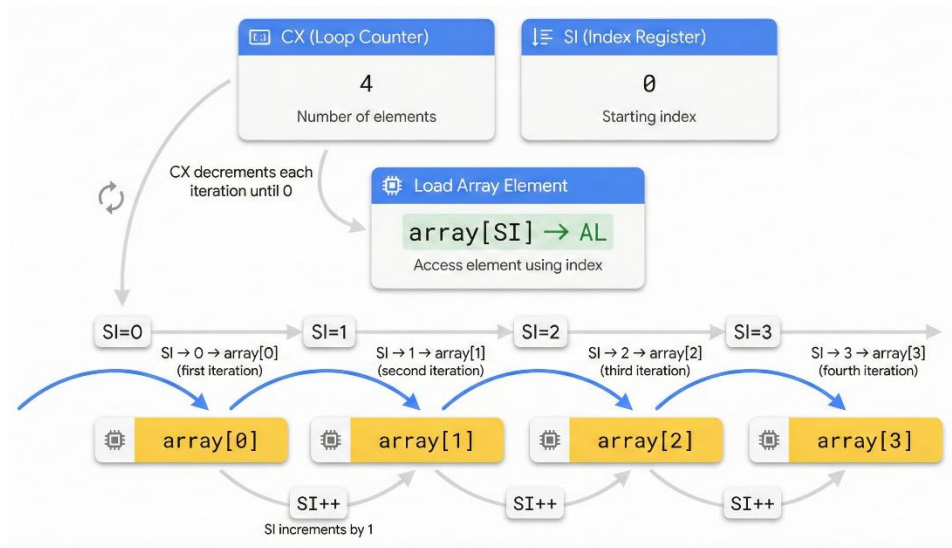


Figure 104: Diagram of Loop-Based Array Traversal Using CX and SI

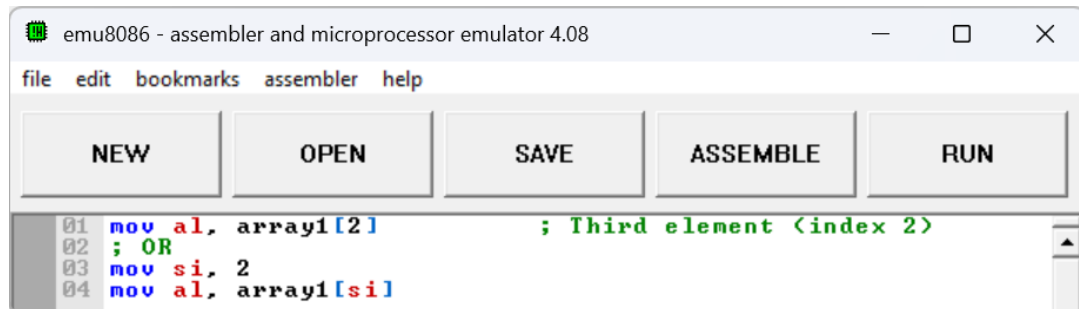
First Element

```

01 mov al, array1[0]      ; Direct indexing
02 ; OR
03 lea si, array1         ; Load base address
04 mov al, [si]           ; Access first element
  
```

Figure 105: Accessing the First Element of an Array

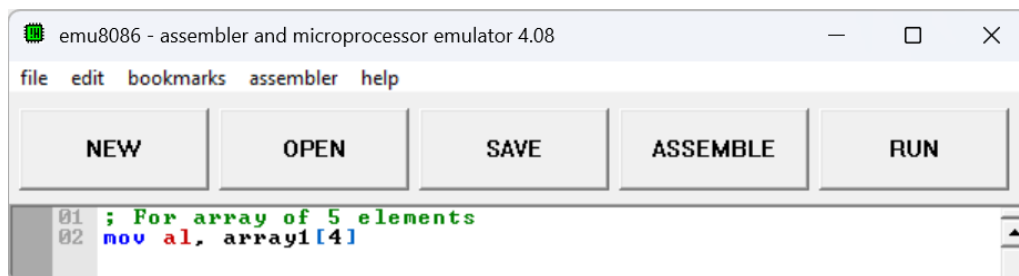
Middle Element



```
emu8086 - assembler and microprocessor emulator 4.08
file edit bookmarks assembler help
NEW OPEN SAVE ASSEMBLE RUN
01 mov al, array1[2]           ; Third element (index 2)
02 ; OR
03 mov si, 2
04 mov al, array1[si]
```

Figure 106: Accessing the Middle Element of an Array

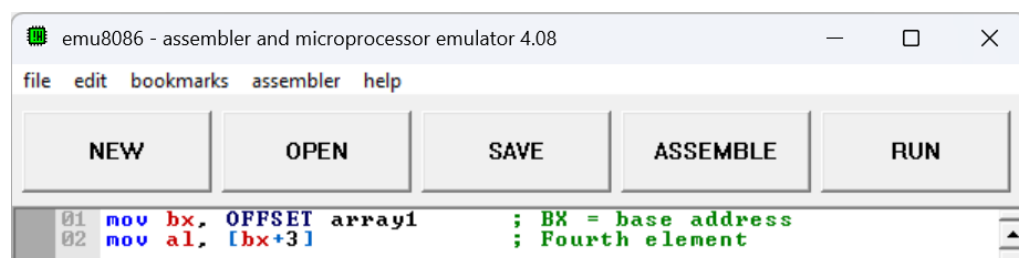
Last Element



```
emu8086 - assembler and microprocessor emulator 4.08
file edit bookmarks assembler help
NEW OPEN SAVE ASSEMBLE RUN
01 ; For array of 5 elements
02 mov al, array1[4]
```

Figure 107: Accessing the Last Element of an Array

Using Offsets



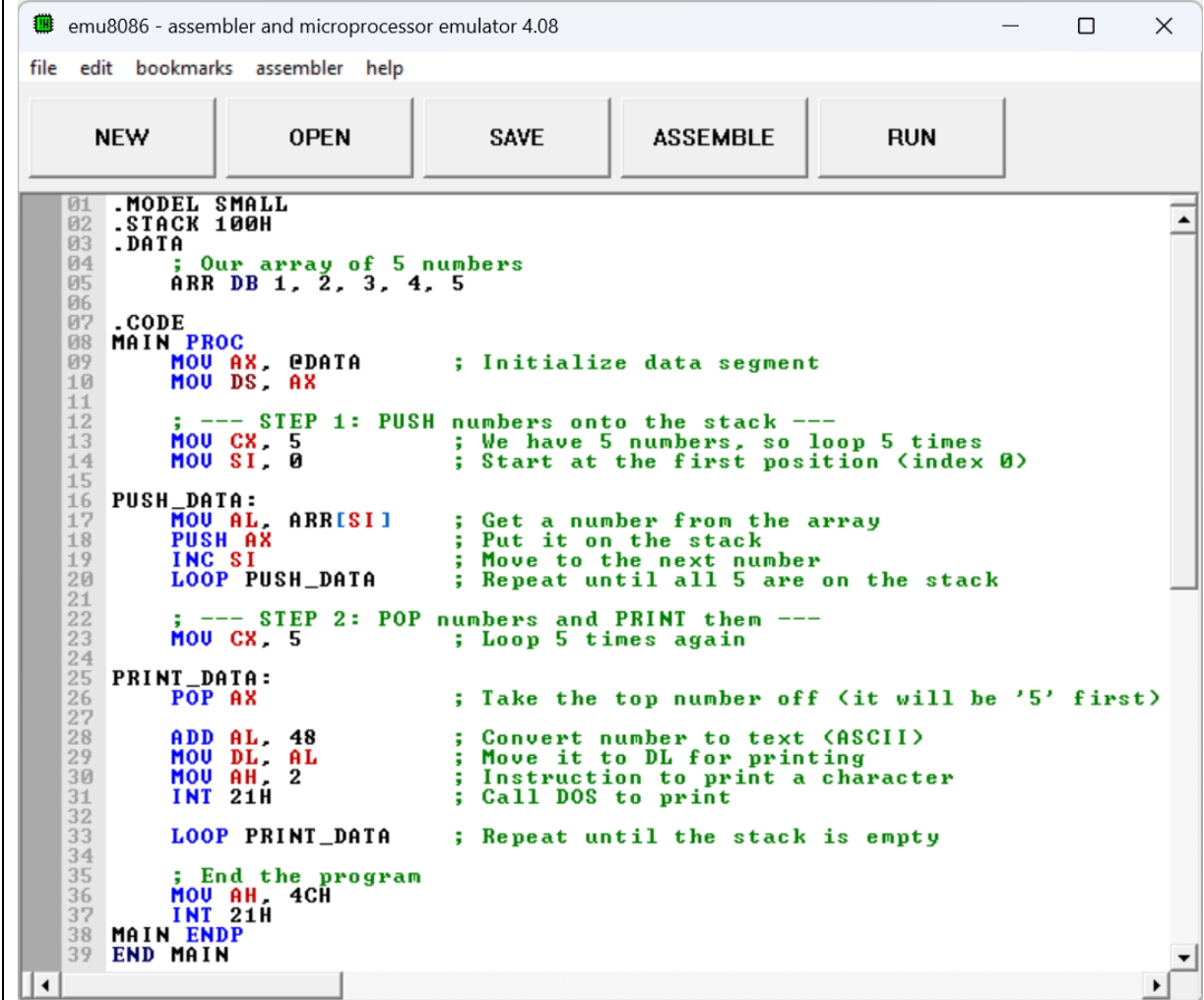
```
emu8086 - assembler and microprocessor emulator 4.08
file edit bookmarks assembler help
NEW OPEN SAVE ASSEMBLE RUN
01 mov bx, OFFSET array1      ; BX = base address
02 mov al, [bx+3]             ; Fourth element
```

Figure 108: Accessing Array Elements using OFFSETS

Question no 1

Task: Reverse an array and display its elements

Store and Display Array



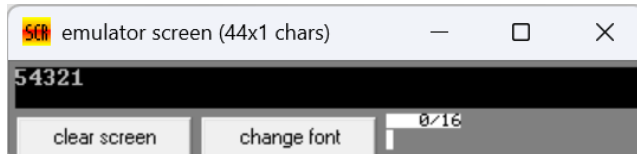
```

01 .MODEL SMALL
02 .STACK 100H
03 .DATA
04     ; Our array of 5 numbers
05     ARR DB 1, 2, 3, 4, 5
06
07 .CODE
08 MAIN PROC
09     MOV AX, @DATA      ; Initialize data segment
10     MOV DS, AX
11
12     ; --- STEP 1: PUSH numbers onto the stack ---
13     MOV CX, 5          ; We have 5 numbers, so loop 5 times
14     MOV SI, 0          ; Start at the first position (index 0)
15
16 PUSH_DATA:
17     MOV AL, ARR[SI]    ; Get a number from the array
18     PUSH AX            ; Put it on the stack
19     INC SI             ; Move to the next number
20     LOOP PUSH_DATA     ; Repeat until all 5 are on the stack
21
22     ; --- STEP 2: POP numbers and PRINT them ---
23     MOV CX, 5          ; Loop 5 times again
24
25 PRINT_DATA:
26     POP AX             ; Take the top number off (it will be '5' first)
27
28     ADD AL, 48         ; Convert number to text (ASCII)
29     MOV DL, AL         ; Move it to DL for printing
30     MOV AH, 2          ; Instruction to print a character
31     INT 21H           ; Call DOS to print
32
33     LOOP PRINT_DATA    ; Repeat until the stack is empty
34
35     ; End the program
36     MOV AH, 4CH
37     INT 21H
38 MAIN ENDP
39 END MAIN

```

Figure 109: Assembly Code for Reversing an Array and Displaying Elements

Output



```

54321

```

Figure 110: Emulator Output showing Reversed Array "54321"

LAB 9

Loops

In high-level languages, you have luxury cars like foreach. In Assembly, we build the engine from scratch. A **loop** is simply a sequence of instructions that repeats until a specific condition is met.

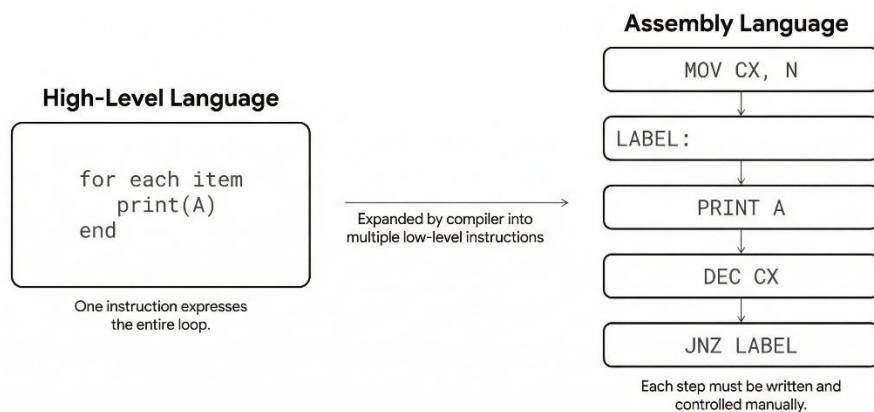


Figure 111: Diagram Comparing Loop Execution: High-Level vs Assembly Language

The Loop Control Mechanism

Every loop, whether in Java or Assembly, needs five things to work without crashing your computer:

1. Initialization: Setting the starting value (usually in the CX register).
2. Condition Check: Checking if we should keep going or stop.
3. Body Execution: The actual work (e.g., printing a character).
4. Update: Changing the counter (incrementing or decrementing).

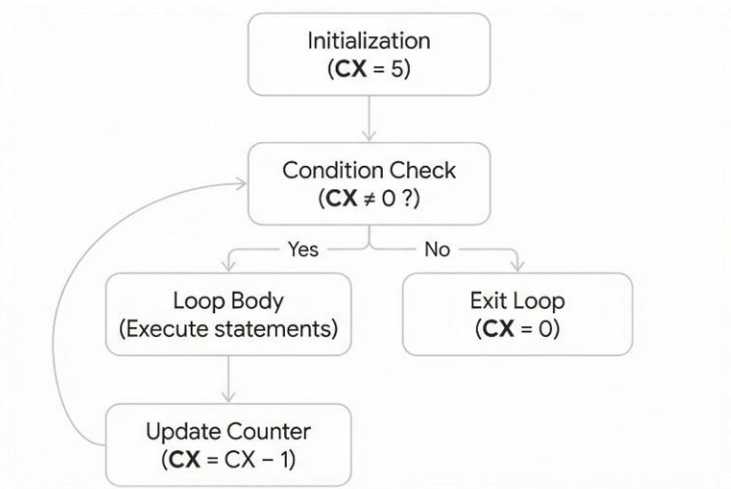


Figure 112: Loop Control Mechanism

5. Termination: Breaking out when the job is done.

The Power Players: Registers Used in Loops

- CX (The Counter Register): This is the VIP of loops. The LOOP instruction specifically looks at CX to decide when to stop.
- General Purpose (AX, BX, DX): Used for calculations or holding data during the loop.
- Flag Register: The "brain" of the CPU. Instructions like CMP (Compare) set flags (like the Zero Flag ZF), which tell the CPU whether to jump or stay.

The LOOP Instruction

The LOOP instruction is the "Easy Button" of 8086 Assembly.

Syntax: LOOP label

How it works: 1. It automatically subtracts 1 from CX ($CX = CX - 1$).

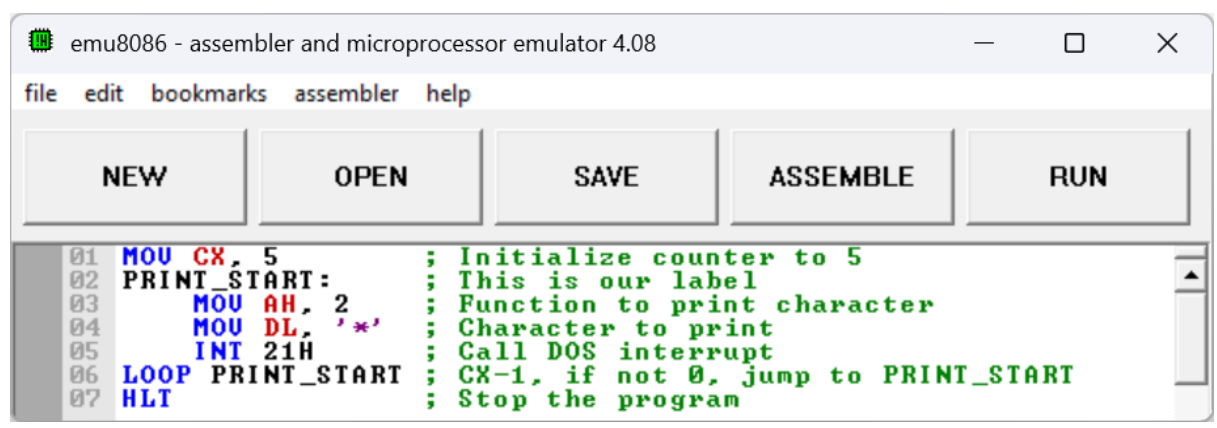
2. It checks if CX is zero.

3. If CX is **NOT** zero, it jumps back to the label.

4. If CX is zero, it moves to the next line of code.

Example:

Print 'Hello' 5 Times



```
01 MOV CX, 5           ; Initialize counter to 5
02 PRINT_START:       ; This is our label
03     MOV AH, 2       ; Function to print character
04     MOV DL, 'H'     ; Character to print
05     INT 21H         ; Call DOS interrupt
06 LOOP PRINT_START   ; CX-1, if not 0, jump to PRINT_START
07 HLT               ; Stop the program
```

Figure 113: Basic Loop to Print 'Hello' 5 Times

Taking Control: Conditional Jumps

Sometimes, LOOP isn't enough. What if you want to loop until a user presses 'Q'? That's where Conditional Jumps come in. They look at the Flag Register to make decisions.

Conditional Jump Instructions and Flag Conditions		
Instruction	Meaning	Flag Condition
JZ / JE ↗	Jump if Zero / Equal	ZF = 1
JNZ / JNE ?	Jump if Not Zero / Not Equal	ZF = 0
JG / JL ↗	Jump if Greater / Less (Signed)	Varies (uses ZF, SF, OF)
JA / JB ↗	Jump if Above / Below (Unsigned)	CF = 0 / 1

Figure 114: Conditional Jumps

Pro Tip: Use *CMP* (Compare) before a conditional jump. *CMP AX, BX* subtracts *BX* from *AX* internally and sets the flags without changing the registers!

Types of Loops

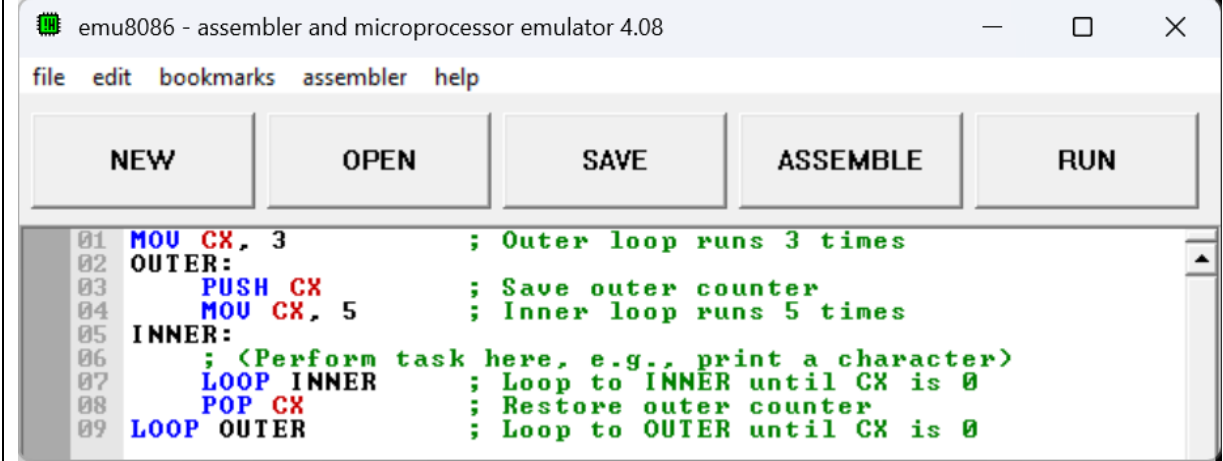
1. Count-Controlled: You know exactly how many times to run (uses CX).
2. Condition-Controlled: Runs until something changes (e.g., "Loop until AL is 0").
3. Infinite Loops: Runs forever. Used in operating system kernels or embedded systems (e.g., JMP label).

Nested Loops: Loops Inside Loops

When you need to create a grid or a pattern, you put one loop inside another.

The Challenge: Both loops want to use CX. To fix this, we PUSH (save) the outer loop's CX onto the stack and POP (restore) it later.

Example



```

01 MOV CX, 3          ; Outer loop runs 3 times
02 OUTER:             ;
03     PUSH CX         ; Save outer counter
04     MOV CX, 5       ; Inner loop runs 5 times
05     INNER:          ;
06         ; <Perform task here, e.g., print a character>
07         LOOP INNER   ; Loop to INNER until CX is 0
08         POP CX       ; Restore outer counter
09     LOOP OUTER      ; Loop to OUTER until CX is 0

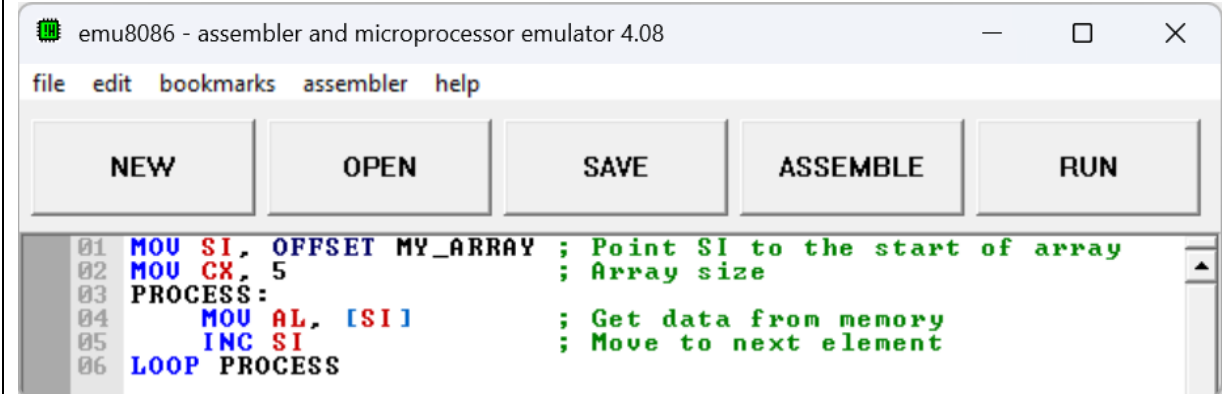
```

Figure 115: Assembly Code Demonstrating Nested Loop Structure

Loops with Memory Access (Arrays)

To move through an array, we use Pointer Registers like SI (Source Index) or DI (Destination Index).

Example



```

01 MOV SI, OFFSET MY_ARRAY ; Point SI to the start of array
02 MOV CX, 5               ; Array size
03 PROCESS:                ;
04     MOV AL, [SI]         ; Get data from memory
05     INC SI              ; Move to next element
06     LOOP PROCESS

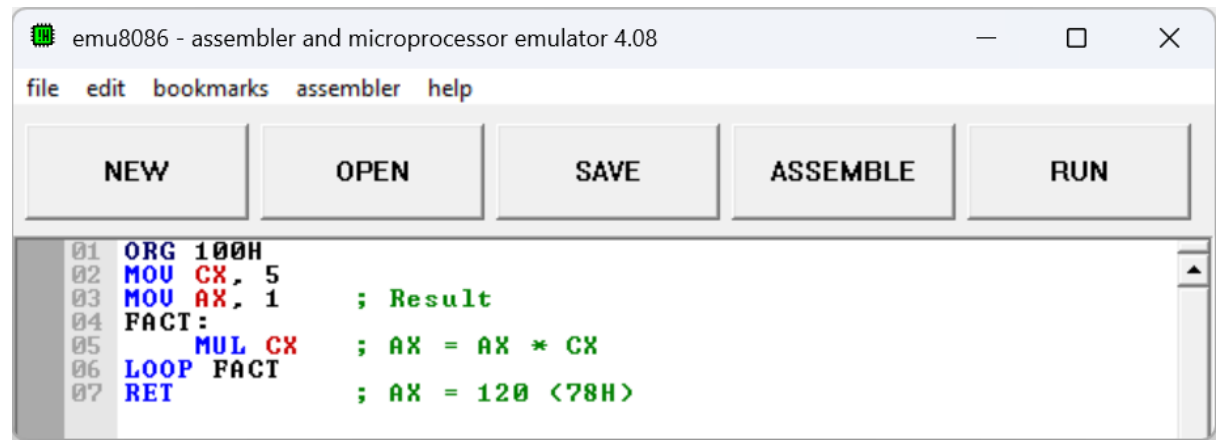
```

Figure 116: Assembly Code for Loop-Based Memory Access Using Pointers

Question no 2

Task: Find Factorial of a Number

Factorial of a Number (e.g., 5!)

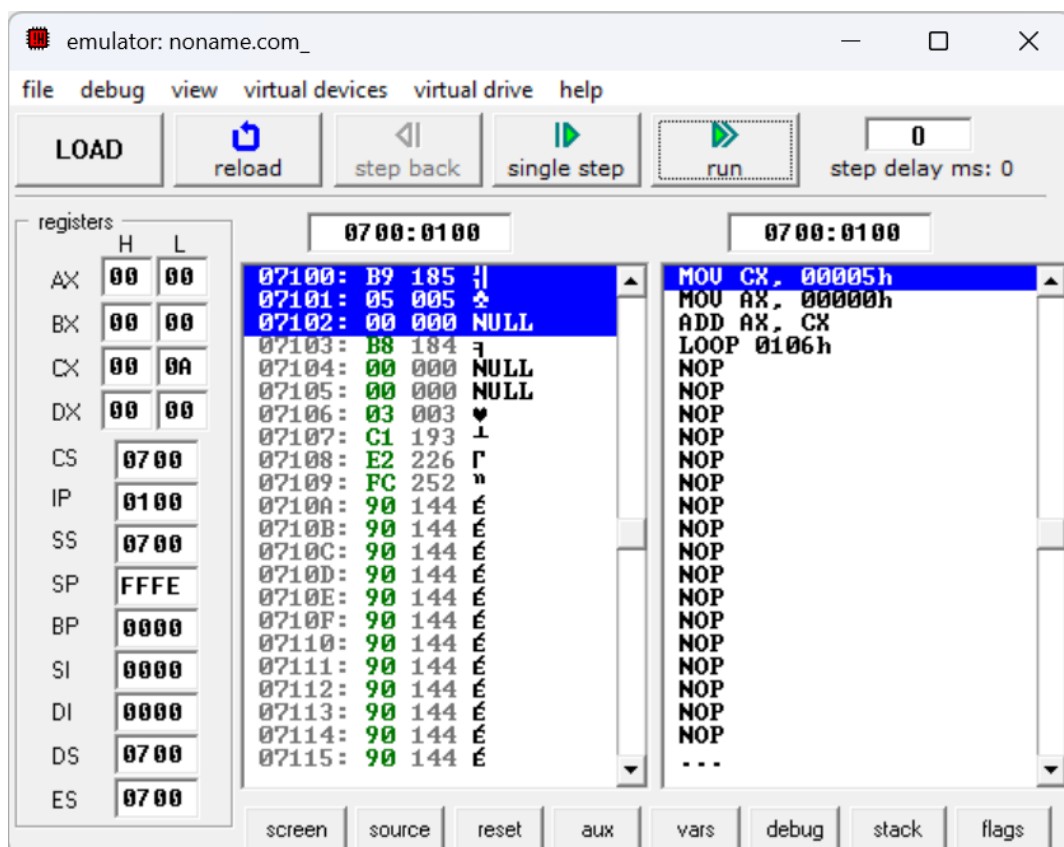


```

01 ORG 100H
02 MOV CX, 5
03 MOV AX, 1      ; Result
04 FACT:
05   MUL CX      ; AX = AX * CX
06   LOOP FACT   ; AX = 120 (<78H>)
07 RET
  
```

Figure 119: Assembly Code for Factorial Calculation

Output



emulator: noname.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	00
BX	00	00
CX	00	0A
DX	00	00
CS	0700	
IP	0100	
SS	0700	
SP	FFFE	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

0700:0100			0700:0100		
07100:	B9	185	MOV	CX,	00005h
07101:	05	005	MOV	AX,	00000h
07102:	00	000	ADD	AX,	CX
07103:	B8	184	LOOP	0106h	
07104:	00	000	NOP		
07105:	00	000	NOP		
07106:	03	003	NOP		
07107:	C1	193	NOP		
07108:	E2	226	NOP		
07109:	FC	252	NOP		
0710A:	90	144	NOP		
0710B:	90	144	NOP		
0710C:	90	144	NOP		
0710D:	90	144	NOP		
0710E:	90	144	NOP		
0710F:	90	144	NOP		
07110:	90	144	NOP		
07111:	90	144	NOP		
07112:	90	144	NOP		
07113:	90	144	NOP		
07114:	90	144	NOP		
07115:	90	144	NOP		

screen source reset aux vars debug stack flags

Figure 120: Emulator Output for Factorial of 5

LAB 10

Jumps

In Assembly (specifically x86/8086), a Jump instruction changes the Flow of Execution by pointing the processor to a different memory address (labeled in your code).

Types of Jumps

1. **Unconditional Jump (JMP):** It always jumps, no matter what. It's like an infinite loop or a "go-to" statement.
2. **Conditional Jumps:** These only jump if a specific condition is met (usually after a CMP or "Compare" instruction).
 - JE: Jump if Equal
 - JNE: Jump if Not Equal
 - JG: Jump if Greater
 - JL: Jump if Less
 - JZ / JNZ: Jump if Zero / Not Zero

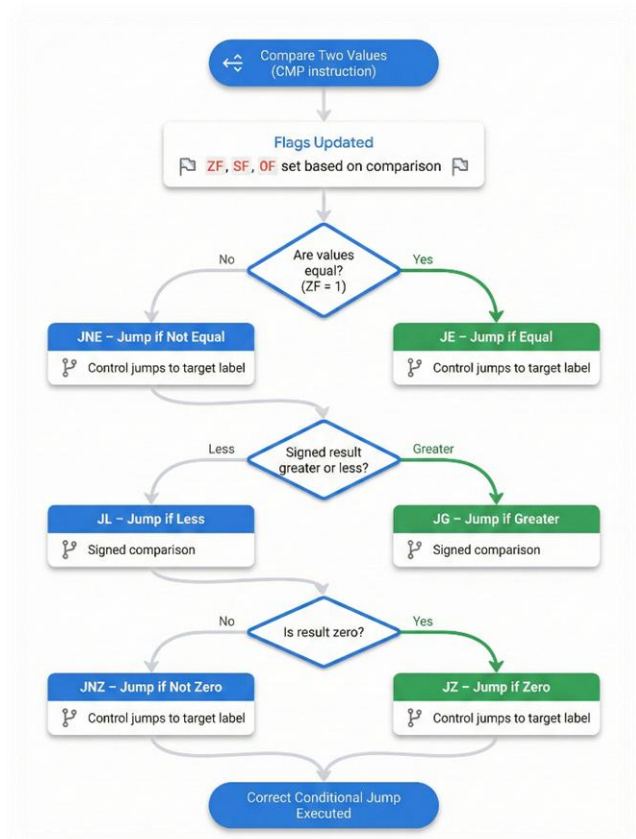
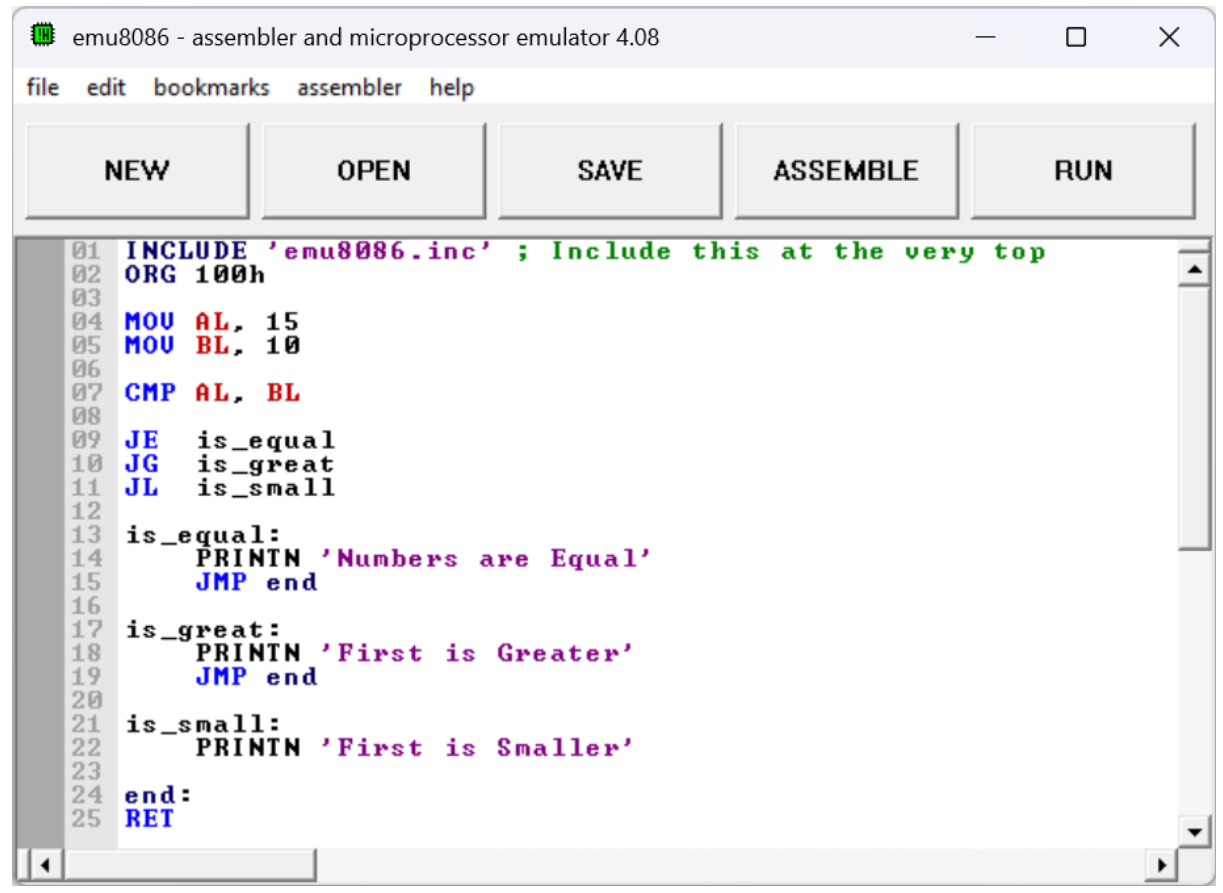


Figure 121: Conditional Jumps

Question no 1

Task: Write a program that compares two numbers stored in registers and displays whether the first is greater, smaller, or equal.

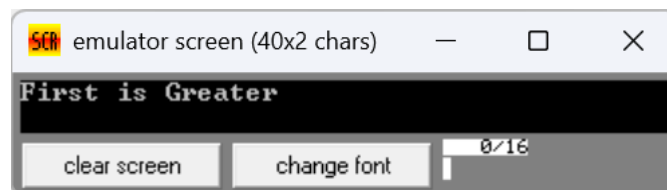
Compare Two Numbers



```
01 INCLUDE 'emu8086.inc' ; Include this at the very top
02 ORG 100h
03
04 MOV AL, 15
05 MOV BL, 10
06
07 CMP AL, BL
08
09 JE is_equal
10 JG is_great
11 JL is_small
12
13 is_equal:
14     PRINTN 'Numbers are Equal'
15     JMP end
16
17 is_great:
18     PRINTN 'First is Greater'
19     JMP end
20
21 is_small:
22     PRINTN 'First is Smaller'
23
24 end:
25 RET
```

Figure 122: Assembly Code for Comparing Two Numbers

Output



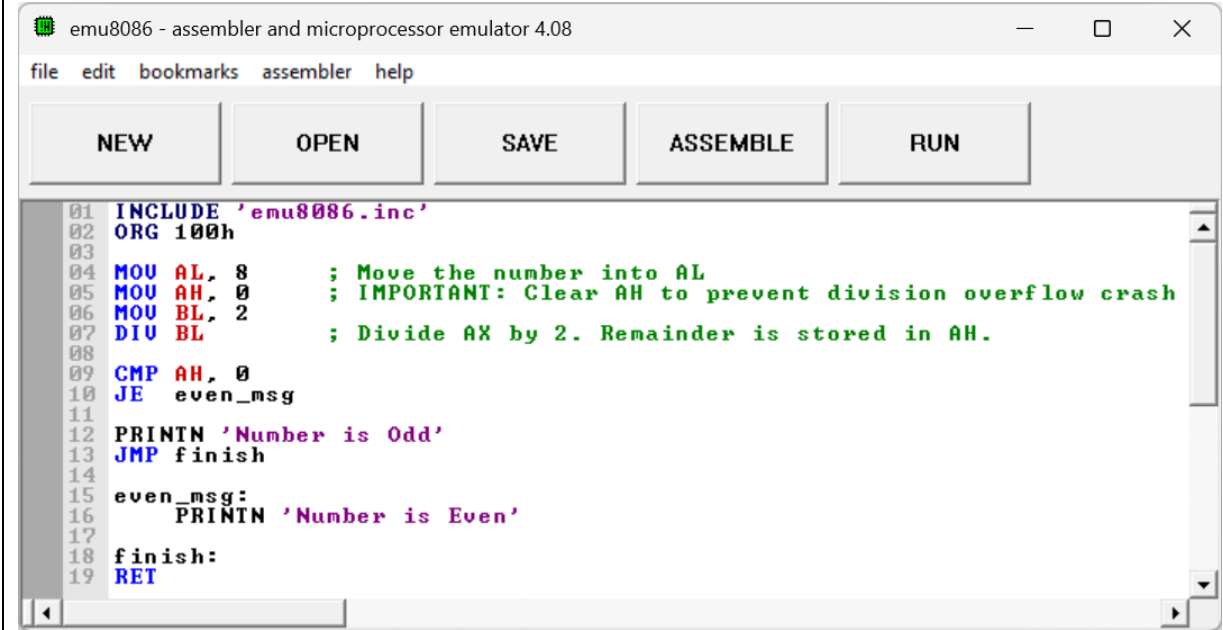
```
emulator screen (40x2 chars)
First is Greater
clear screen  change font  0/16
```

Figure 123: Output Showing Comparison Result (Greater, Smaller, or Equal)

Question no 2

Task: Check if a number is even

Check Even or Odd



```
01 INCLUDE 'emu8086.inc'
02 ORG 100h
03
04 MOV AL, 8      ; Move the number into AL
05 MOV AH, 0      ; IMPORTANT: Clear AH to prevent division overflow crash
06 MOV BL, 2
07 DIV BL        ; Divide AX by 2. Remainder is stored in AH.
08
09 CMP AH, 0
10 JE even_msg
11
12 PRINTN 'Number is Odd'
13 JMP finish
14
15 even_msg:
16     PRINTN 'Number is Even'
17
18 finish:
19 RET
```

Figure 124: Assembly Code to Check for Even or Odd Number

Output

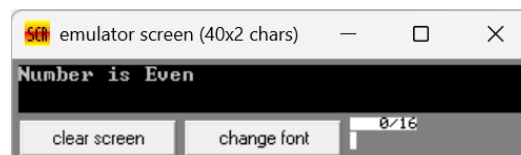
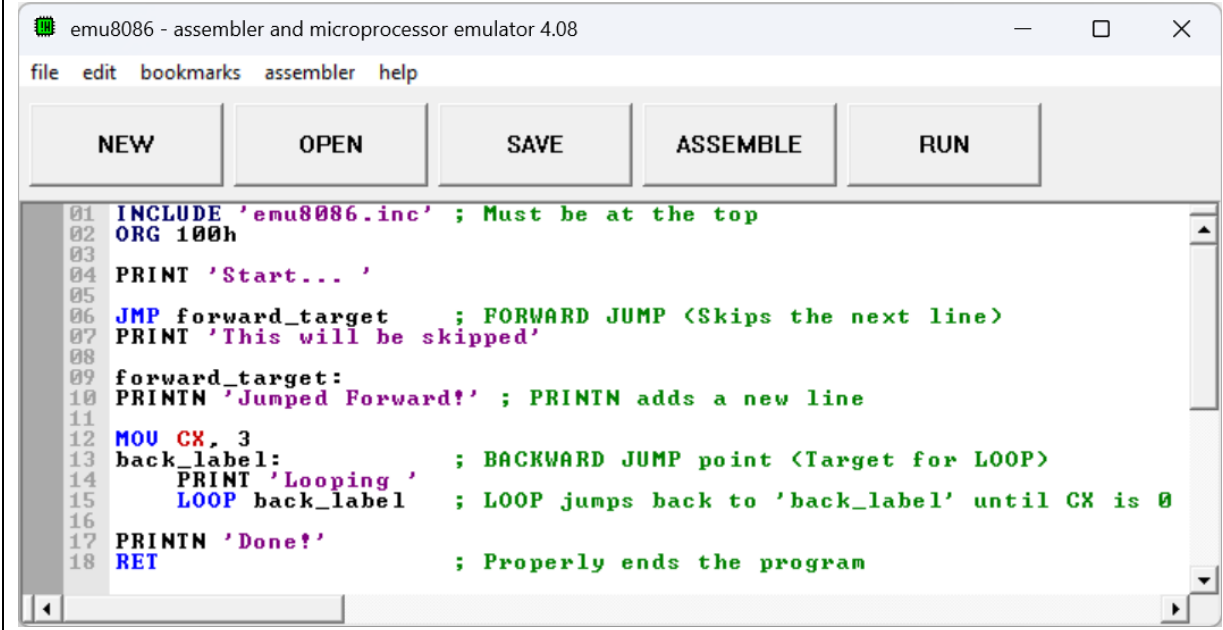


Figure 125: Output Displaying Parity Check Result

Question no 4

Task: Demonstrate short forward and backward jumps

Short Forward and Backward Jumps



```
01 INCLUDE 'emu8086.inc' ; Must be at the top
02 ORG 100h
03
04 PRINT 'Start... '
05
06 JMP forward_target ; FORWARD JUMP <Skips the next line>
07 PRINT 'This will be skipped'
08
09 forward_target:
10 PRINTN 'Jumped Forward!' ; PRINTN adds a new line
11
12 MOV CX, 3
13 back_label: ; BACKWARD JUMP point <Target for LOOP>
14 PRINT 'Looping '
15 LOOP back_label ; LOOP jumps back to 'back_label' until CX is 0
16
17 PRINTN 'Done!'
18 RET ; Properly ends the program
```

Figure 128: Demonstration of Short Forward and Backward Jumps

Output



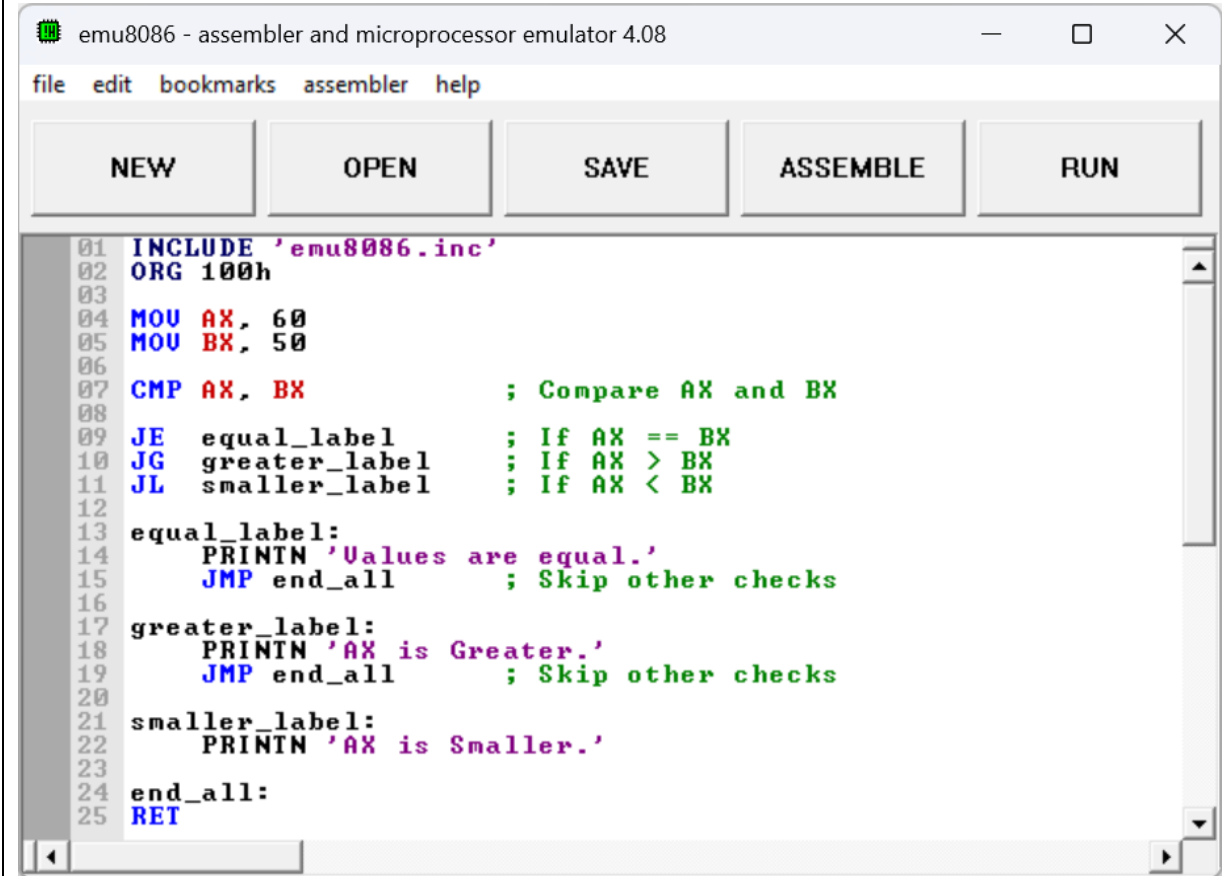
```
emulator screen (42x3 chars)
Start... Jumped Forward!
Looping Looping Looping Done!
clear screen change font 0/16
```

Figure 129: Output Verifying Forward and Backward Jump Execution

Question no 5

Task: Write a code that compares AX and BX and uses multiple conditions to decide the path.

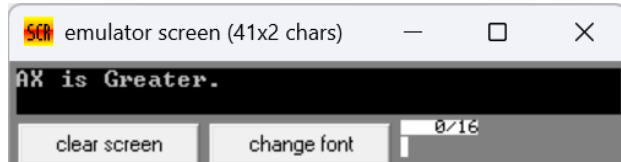
Demonstrating Multiple Jump Types



```
01 INCLUDE 'emu8086.inc'
02 ORG 100h
03
04 MOV AX, 60
05 MOV BX, 50
06
07 CMP AX, BX           ; Compare AX and BX
08
09 JE equal_label       ; If AX == BX
10 JG greater_label     ; If AX > BX
11 JL smaller_label     ; If AX < BX
12
13 equal_label:
14     PRINTN 'Values are equal.'
15     JMP end_all       ; Skip other checks
16
17 greater_label:
18     PRINTN 'AX is Greater.'
19     JMP end_all       ; Skip other checks
20
21 smaller_label:
22     PRINTN 'AX is Smaller.'
23
24 end_all:
25 RET
```

Figure 130: Assembly Code Using Multiple Jump Conditions for Path Decision

Output



```
emulator screen (41x2 chars)
AX is Greater.
clear screen  change font  0/16
```

Figure 131: Final Output for Multiple Jump Types Program

LAB 11

Bubble Sort

To understand and implement the **Bubble Sort algorithm** using 8086 Assembly Language in the EMU8086 environment.

Logic Overview

Bubble Sort works by repeatedly stepping through the list, comparing adjacent elements, and swapping them if they are in the wrong order. This process repeats until the largest numbers "bubble up" to the end of the array.

- The Outer Loop: Controls how many passes we make through the array (N-1 times).
- The Inner Loop: Compares each element with its neighbour.
- The Swap: If the left number is greater than the right number, they switch places.

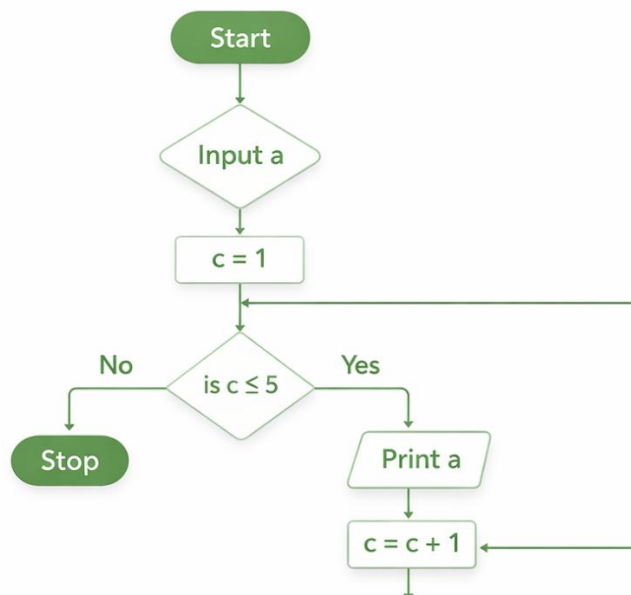
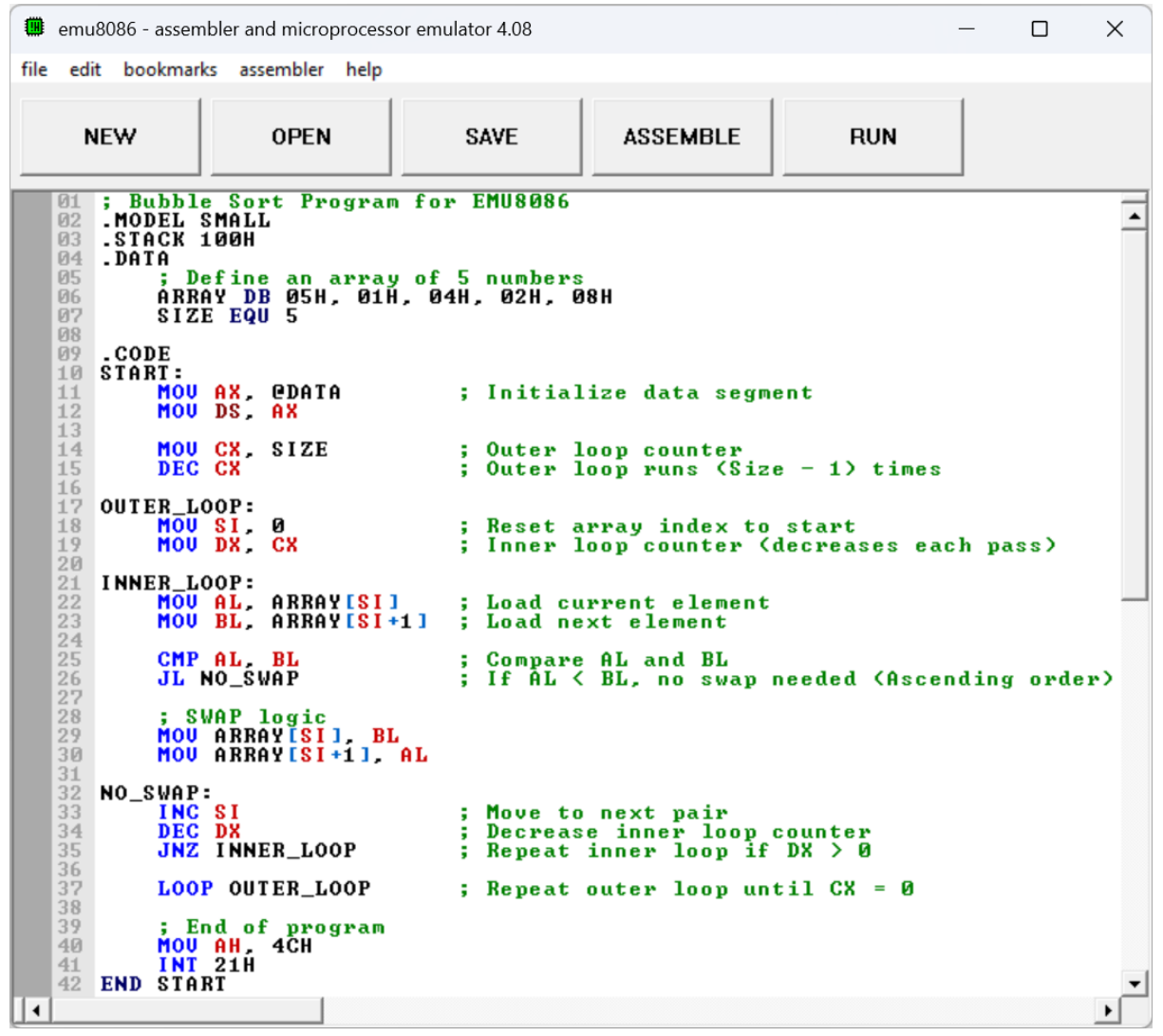


Figure 132: Bubble Sorting Algorithm

Question no 1

Task: Write Bubble Sorting Program

Bubble Sorting



```

01 ; Bubble Sort Program for EMU8086
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 ; Define an array of 5 numbers
06 ARRAY DB 05H, 01H, 04H, 02H, 08H
07 SIZE EQU 5
08
09 .CODE
10 START:
11     MOV AX, @DATA           ; Initialize data segment
12     MOV DS, AX
13
14     MOV CX, SIZE             ; Outer loop counter
15     DEC CX                   ; Outer loop runs (Size - 1) times
16
17 OUTER_LOOP:
18     MOV SI, 0                ; Reset array index to start
19     MOV DX, CX               ; Inner loop counter (decreases each pass)
20
21 INNER_LOOP:
22     MOV AL, ARRAY[SI]        ; Load current element
23     MOV BL, ARRAY[SI+1]      ; Load next element
24
25     CMP AL, BL               ; Compare AL and BL
26     JL NO_SWAP               ; If AL < BL, no swap needed (Ascending order)
27
28     ; SWAP logic
29     MOV ARRAY[SI], BL
30     MOV ARRAY[SI+1], AL
31
32 NO_SWAP:
33     INC SI                   ; Move to next pair
34     DEC DX                   ; Decrease inner loop counter
35     JNZ INNER_LOOP           ; Repeat inner loop if DX > 0
36     LOOP OUTER_LOOP          ; Repeat outer loop until CX = 0
37
38 ; End of program
39
40 MOV AH, 4CH
41 INT 21H
42 END START

```

Figure 133: Bubble Sorting

How the Code Works (Step-by-Step)

1. Initialization: We load the **ARRAY** into memory. **CX** is set to 4 because, for 5 elements, we need 4 passes.
2. The Comparison: **MOV AL, ARRAY[SI]** and **MOV BL, ARRAY[SI+1]** grab two side-by-side numbers.

3. The Decision: **CMP AL, BL** checks which is bigger. If the first one is smaller (JL - Jump if Less), we skip the swap.
4. The Swap: To swap, we simply move the value of **BL** into the position of **AL** and vice versa.
5. Looping: The **JNZ INNER_LOOP** ensures we check every pair in the array, and **LOOP OUTER_LOOP** ensures we repeat the whole process until the array is fully sorted

LAB 12

Shifts

To understand how to move data bits left or right within a register to perform fast multiplication, division, and logical testing.

Explanation

Bit shifting is like moving a row of people on a bench. If everyone moves one seat to the left, the person on the far left falls off, and an empty seat appears on the right.

In computers:

- **SHL (Shift Left):** Moves all bits to the left.
 - **Effect:** Multiplies the number by 2 for every shift.
 - **Example:** 0010 (Decimal 2) shifted left becomes 0100 (Decimal 4).
- **SHR (Shift Right):** Moves all bits to the right.
 - **Effect:** Divides the number by 2 for every shift.
 - **Example:** 0100 (Decimal 4) shifted right becomes 0010 (Decimal 2).
- **The Carry Flag (CF):** This acts as a "bucket" for the bit that falls off the edge. We can check this bucket to see if a specific bit was 1 or 0.

Question no 1

Task: Multiply 03h by 4 using SHL, then add 02h to the result and store it in AL.

Multiply 03h by 4 and Add 02h

```

01  ORG 100h
02
03  MOV AL, 03h      ; Load 03h into AL
04  MOV CL, 2        ; Set shift count to 2 (Multiplying by 4)
05
06  SHL AL, CL       ; Shift AL left by 2 positions (03h * 4 = 0Ch)
07  ADD AL, 02h      ; Add 02h to the result (0Ch + 02h = 0Eh)
08
09  MOV AH, 4Ch      ; Return control to OS
10  INT 21h
11  RET
  
```

Figure 134: Assembly Code for Multiplication and Addition (03h * 4 + 02h)

Output

emulator: noname.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	4C	0E
BX	00	00
CX	00	02
DX	00	00
CS	F400	
IP	0204	
SS	0700	
SP	FFF8	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

F400:0204

F4200:	FF	255	RES
F4201:	FF	255	RES
F4202:	CD	205	=
F4203:	21	033	!
F4204:	CF	207	±
F4205:	00	000	NULL
F4206:	00	000	NULL
F4207:	00	000	NULL
F4208:	00	000	NULL
F4209:	00	000	NULL
F420A:	00	000	NULL
F420B:	00	000	NULL
F420C:	00	000	NULL
F420D:	00	000	NULL
F420E:	00	000	NULL
F420F:	00	000	NULL
F4210:	00	000	NULL
F4211:	00	000	NULL
F4212:	00	000	NULL
F4213:	00	000	NULL
F4214:	00	000	NULL
F4215:	00	000	NULL

BIOS DI

INT 021h
IRET
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
...

screen source reset aux vars debug stack flags

Figure 135: Register Output for Multiplication Task

Question no 2

Task: Divide 40h by 2 using SHR, then compare the result with 15h and jump to a label if it is greater.

Divide 40h by 2 and Compare

```

01 ORG 100h
02
03 MOV AL, 40h      ; Load 40h (64 decimal)
04 SHR AL, 1        ; Divide by 2 (Shift Right 1). Result = 20h
05
06 CMP AL, 15h      ; Compare result (20h) with 15h
07 JA Is_Greater    ; Jump if Above (Greater)
08
09 JMP Stop         ; Skip if not greater
10
11 Is_Greater:
12 MOV BL, 01h      ; Marker: Move 1 to BL to show we jumped
13
14 Stop:
15 MOV AH, 4Ch      ; Exit
16 INT 21h
17 RET
  
```

Figure 136: Assembly Code for Division and Comparison (40h / 2)

Output

registers	H	L
AX	4C	20
BX	00	01
CX	00	11
DX	00	00
CS	F400	
IP	0204	
SS	0700	
SP	FFF8	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

F400:0204		F400:0204	
F4200:	FF 255 RES	BIOS DI	
F4201:	FF 255 RES	INT 021h	
F4202:	CD 005	INTI	
F4203:	21 033	ADD EBX + SI, AL	
F4204:	CF 207	ADD EBX + SI, AL	
F4205:	00 000 NULL	ADD EBX + SI, AL	
F4206:	00 000 NULL	ADD EBX + SI, AL	
F4207:	00 000 NULL	ADD EBX + SI, AL	
F4208:	00 000 NULL	ADD EBX + SI, AL	
F4209:	00 000 NULL	ADD EBX + SI, AL	
F420A:	00 000 NULL	ADD EBX + SI, AL	
F420B:	00 000 NULL	ADD EBX + SI, AL	
F420C:	00 000 NULL	ADD EBX + SI, AL	
F420D:	00 000 NULL	ADD EBX + SI, AL	
F420E:	00 000 NULL	ADD EBX + SI, AL	
F420F:	00 000 NULL	ADD EBX + SI, AL	
F4210:	00 000 NULL	ADD EBX + SI, AL	
F4211:	00 000 NULL	ADD EBX + SI, AL	
F4212:	00 000 NULL	ADD EBX + SI, AL	
F4213:	00 000 NULL	ADD EBX + SI, AL	
F4214:	00 000 NULL	ADD EBX + SI, AL	
F4215:	00 000 NULL	ADD EBX + SI, AL	

Figure 137: Register Output for Division Task

Question no 3

Task: Shift DL left by 1 and jump to a label if the Carry Flag (CF) is set

Shift Left and Jump on Carry

```

01 ORG 100h
02
03 MOV DL, 80h      ; Load 80h (Binary 1000 0000). MSB is 1.
04
05 SHL DL, 1        ; Shift Left. The '1' falls into Carry Flag.
06
07 JC Carry_Set     ; Jump if Carry Flag = 1
08 JMP Exit         ; Skip if no carry
09
10 Carry_Set:
11 MOV AX, 1111h    ; Marker to show carry was detected
12
13 Exit:
14 MOV AH, 4Ch      ; Exit
15 INT 21h
16 RET
  
```

Figure 138: Assembly Code for Shift Left and Carry Jump

Output

registers	H	L
AX	4C	11
BX	00	00
CX	00	10
DX	00	00
CS	F400	
IP	0204	
SS	0700	
SP	FFF8	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

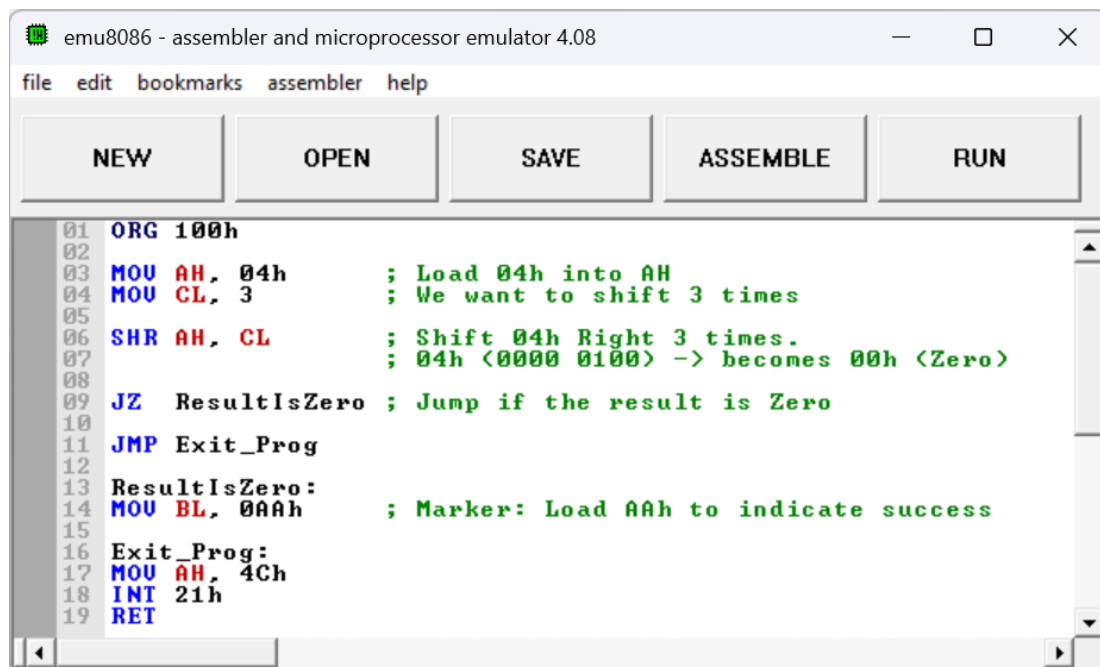
F400:0204		F400:0204	
F4200:	FF 255	RES	BIOS DI
F4201:	FF 255	RES	INT 021h
F4202:	CD 205	=	IRET
F4203:	21 033	!	ADD [BX + SI], AL
F4204:	CF 207	±	ADD [BX + SI], AL
F4205:	00 000	NULL	ADD [BX + SI], AL
F4206:	00 000	NULL	ADD [BX + SI], AL
F4207:	00 000	NULL	ADD [BX + SI], AL
F4208:	00 000	NULL	ADD [BX + SI], AL
F4209:	00 000	NULL	ADD [BX + SI], AL
F420A:	00 000	NULL	ADD [BX + SI], AL
F420B:	00 000	NULL	ADD [BX + SI], AL
F420C:	00 000	NULL	ADD [BX + SI], AL
F420D:	00 000	NULL	ADD [BX + SI], AL
F420E:	00 000	NULL	ADD [BX + SI], AL
F420F:	00 000	NULL	ADD [BX + SI], AL
F4210:	00 000	NULL	ADD [BX + SI], AL
F4211:	00 000	NULL	ADD [BX + SI], AL
F4212:	00 000	NULL	ADD [BX + SI], AL
F4213:	00 000	NULL	ADD [BX + SI], AL
F4214:	00 000	NULL	ADD [BX + SI], AL
F4215:	00 000	NULL	...

Figure 139: Register Output for Shift Left Task

Question no 4

Task: Shift AH right by the number in CL using SHR and jump to a label if the result is zero.

Shift Right until Zero

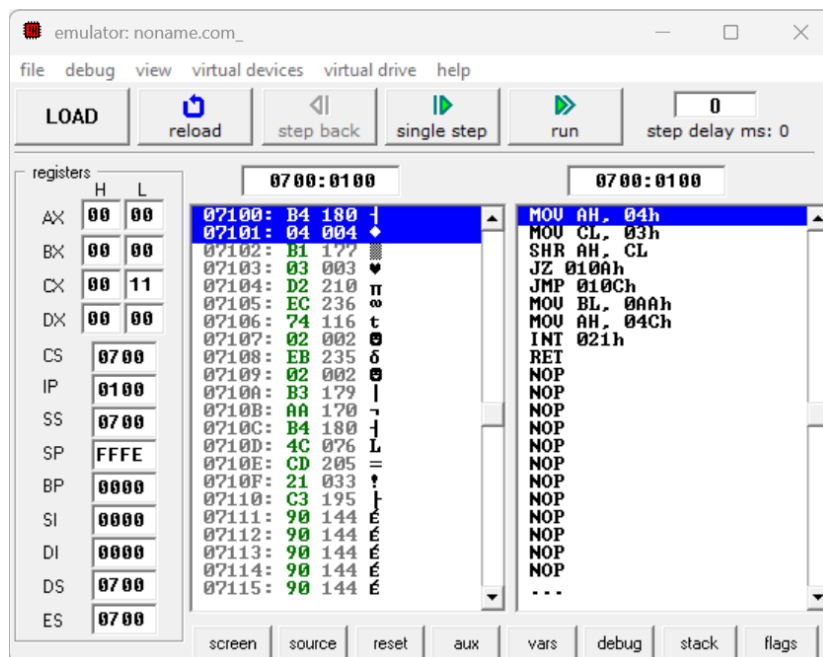


```

01  ORG 100h
02
03  MOV AH, 04h      ; Load 04h into AH
04  MOV CL, 3        ; We want to shift 3 times
05
06  SHR AH, CL       ; Shift 04h Right 3 times.
07                  ; 04h (0000 0100) -> becomes 00h (Zero)
08
09  JZ ResultIsZero  ; Jump if the result is Zero
10
11  JMP Exit_Prog
12
13  ResultIsZero:
14  MOV BL, 0AAh     ; Marker: Load AAh to indicate success
15
16  Exit_Prog:
17  MOV AH, 4Ch
18  INT 21h
19  RET
  
```

Figure 140: Assembly Code for Shift Right Until Zero

Output



The emulator window shows the following register values:

Register	Value
AX	00 00
BX	00 00
CX	00 11
DX	00 00
CS	0700
IP	0100
SS	0700
SP	FFFE
BP	0000
SI	0000
DI	0000
DS	0700
ES	0700

The memory dump shows the following instructions:

```

07100: B4 180 | MOV AH, 04h
07101: 04 004 | MOV CL, 03h
07102: B1 177 | SHR AH, CL
07103: 03 003 | JZ 010Ah
07104: D2 210 | JMP 010Ch
07105: EC 236 | MOV BL, 0AAh
07106: 74 116 | MOV AH, 04Ch
07107: 02 002 | INT 021h
07108: EB 235 | RET
07109: 02 002 | NOP
0710A: B3 179 | NOP
0710B: AA 170 | NOP
0710C: B4 180 | NOP
0710D: 4C 076 | NOP
0710E: CD 205 | NOP
0710F: 21 033 | NOP
07110: C3 195 | NOP
07111: 90 144 | NOP
07112: 90 144 | NOP
07113: 90 144 | NOP
07114: 90 144 | NOP
07115: 90 144 | NOP
  
```

Figure 141: Register Output for Shift Right Task

LAB 13

Rotation

To understand how to "rotate" bits in a circular motion, ensuring that data is recycled rather than lost, and how to use the Carry Flag (CF) as a bridge during these rotations.

Explanation

Imagine a row of people sitting in chairs.

- Shift: If everyone moves left, the person on the far left falls off and is lost.
- Rotate: If everyone moves left, the person on the far left runs around to the back and sits in the empty seat on the right. No one is lost.

Types of Rotation

ROL / ROR (Standard Rotation): The bit that leaves one side immediately enters the other side. It is a perfect circle.

RCL / RCR (Rotate through Carry): The Carry Flag (CF) acts like an extra seat.

The bit leaves the register, sits in the Carry Flag for a moment, and the bit that was *previously* in the Carry Flag enters the register.

Question no 1

Task 1: Load AL with 1 and perform SHR AL, 1. When AL becomes zero, should JZ be taken?

Shift Right and Check Zero

```

01  ORG 100h
02
03  MOV AL, 01h      ; Load 1 into AL (0000 0001)
04  SHR AL, 1        ; Shift Right. AL becomes 00h. ZF becomes 1.
05
06  JZ Is_Zero       ; Jump if Zero Flag is Set
07  JMP Stop
08
09  Is_Zero:
10  MOV BL, 0AAh     ; Marker to show jump occurred
11
12  Stop:
13  RET
  
```

Figure 144: Assembly Code for Shift Right and Check Zero (JZ)

Output

emulator: noname.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	00
BX	00	AA
CX	00	00
DX	00	00
CS	F400	
IP	0154	
SS	0700	
SP	FFFA	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

F400:0154

F4150:	FF	255	RES
F4151:	FF	255	RES
F4152:	CD	205	=
F4153:	20	032	SPA
F4154:	CF	207	±
F4155:	00	000	NULL
F4156:	00	000	NULL
F4157:	00	000	NULL
F4158:	00	000	NULL
F4159:	00	000	NULL
F415A:	00	000	NULL
F415B:	00	000	NULL
F415C:	00	000	NULL
F415D:	00	000	NULL
F415E:	00	000	NULL
F415F:	00	000	NULL
F4160:	FF	255	RES
F4161:	FF	255	RES
F4162:	CD	205	=
F4163:	1A	026	→
F4164:	CF	207	±
F4165:	00	000	NULL

BIOS DI
INT 020h
IRET
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
DEC BH, BH
SBB CL, BH
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD BH, BH
DEC BP
ADD BH, CL
ADD [BX + SI], AL
ADD [BX + SI], AL
...

screen source reset aux vars debug stack flags

Figure 145: Register Output for JZ Check

Question no 3

Task: Load AL with any value and perform ROR AL, 1. If LSB = 1, should JC occur?

Rotate Right and Test LSB

```

01 ORG 100h
02
03 MOV AL, 05h      ; Load 05h <Binary 0000 0101>. LSB is 1.
04 ROR AL, 1        ; Rotate Right. LSB<1> moves to CF and MSB.
05                  ; AL becomes 82h <1000 0010>.
06
07 JC  LSB_Was_1    ; Jump if Carry Flag = 1
08 JMP Stop
09
10 LSB_Was_1:
11 MOV DX, 0FFFFh   ; Marker to show LSB was 1
12
13 Stop:
14 RET
  
```

Figure 148: Assembly Code for Rotate Right and Test LSB

Output

emulator: noname.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	00	82
BX	00	00
CX	00	0C
DX	FF	FF
CS	F400	
IP	0154	
SS	0700	
SP	FFFA	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

F400:0154

F4150:	FF	255	RES
F4151:	FF	255	RES
F4152:	CD	205	=
F4153:	20	032	SPA
F4154:	CF	207	±
F4155:	00	000	NULL
F4156:	00	000	NULL
F4157:	00	000	NULL
F4158:	00	000	NULL
F4159:	00	000	NULL
F415A:	00	000	NULL
F415B:	00	000	NULL
F415C:	00	000	NULL
F415D:	00	000	NULL
F415E:	00	000	NULL
F415F:	00	000	NULL
F4160:	FF	255	RES
F4161:	FF	255	RES
F4162:	CD	205	=
F4163:	1A	026	→
F4164:	CF	207	±
F4165:	00	000	NULL

BIOS DI
INT 020h
IRET
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD BH, BH
DEC BP
SBB CL, BH
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD [BX + SI], AL
ADD BH, BH
DEC BP
ADD BH, CL
ADD [BX + SI], AL
ADD [BX + SI], AL
...

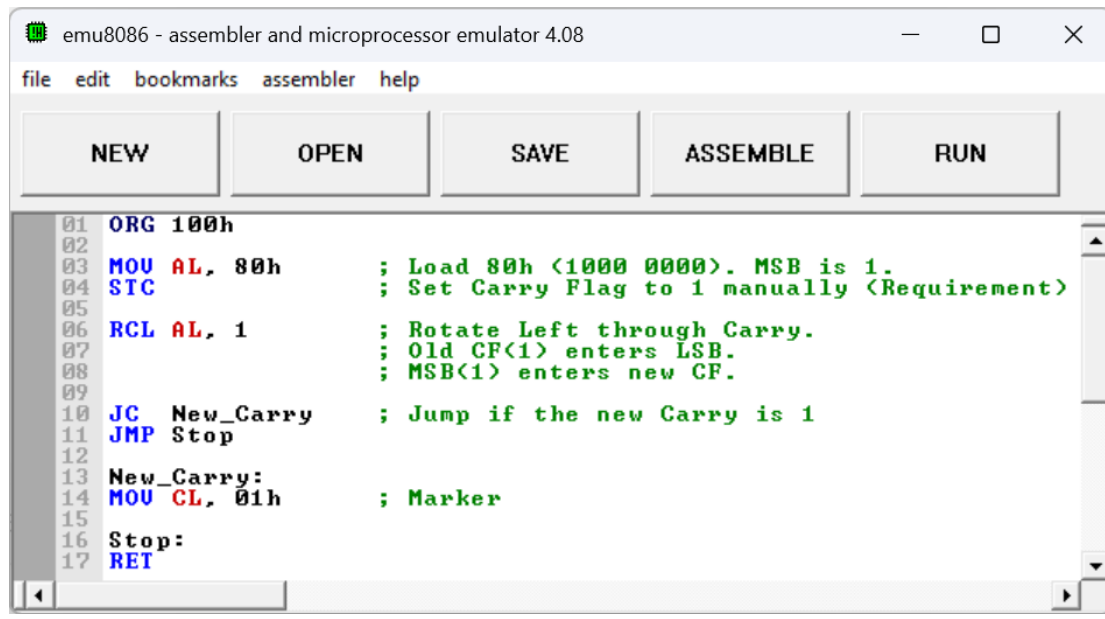
screen source reset aux vars debug stack flags

Figure 149: Register Output for ROR Task

Question no 4

Task: Use RCL AL, 1 when CF = 1. After rotating, should a JC jump happen?

Rotate through Carry Left (RCL)

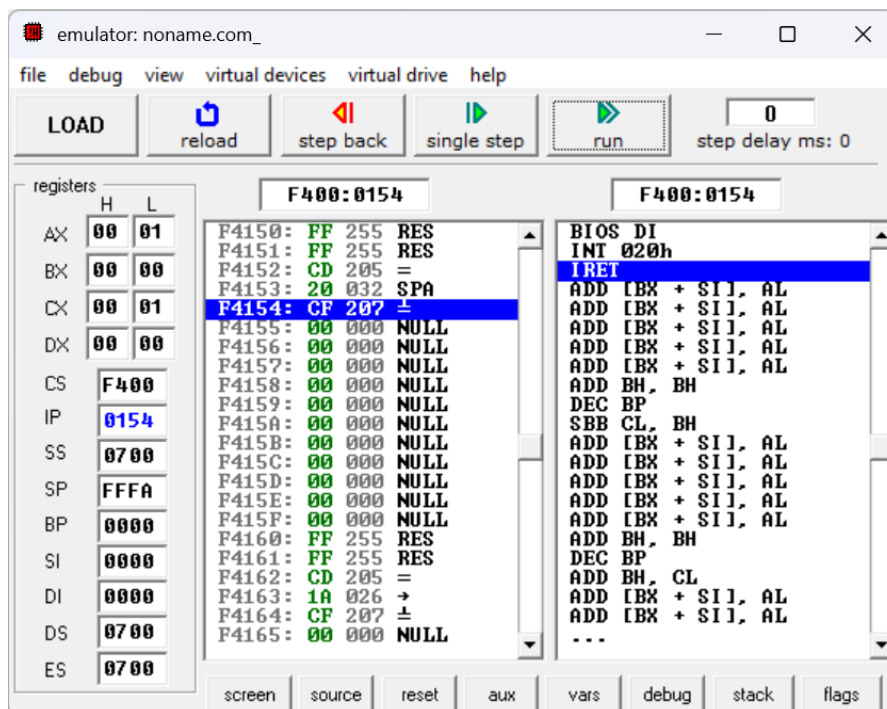


```

01  ORG 100h
02
03  MOV AL, 80h      ; Load 80h (1000 0000). MSB is 1.
04  STC              ; Set Carry Flag to 1 manually (Requirement)
05
06  RCL AL, 1        ; Rotate Left through Carry.
07                  ; Old CF(1) enters LSB.
08                  ; MSB(1) enters new CF.
09
10  JC New_Carry     ; Jump if the new Carry is 1
11  JMP Stop
12
13  New_Carry:
14  MOV CL, 01h      ; Marker
15
16  Stop:
17  RET
  
```

Figure 150: Assembly Code for Rotate through Carry Left (RCL)

Output



registers	H	L
AX	00	01
BX	00	00
CX	00	01
DX	00	00
CS	F400	
IP	0154	
SS	0700	
SP	FFFA	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

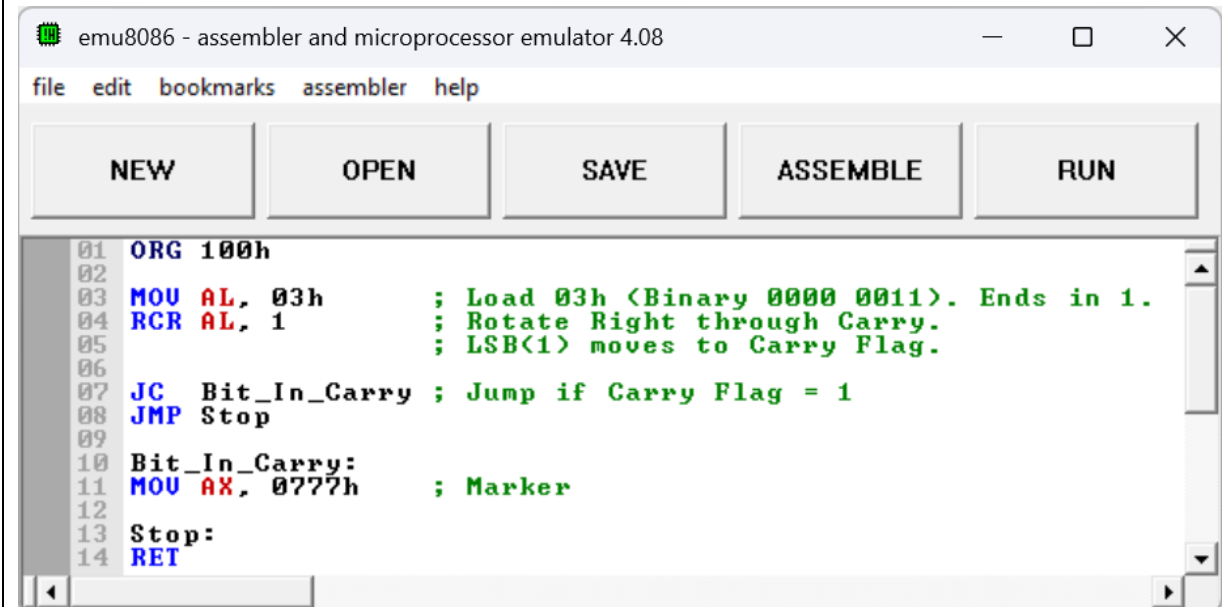
F400:0154		F400:0154	
F4150:	FF 255 RES	BIOS DI	
F4151:	FF 255 RES	INT 020h	
F4152:	CD 205 =	IRET	
F4153:	20 032 SPA	ADD [BX + SI], AL	
F4154:	CF 207 =	ADD [BX + SI], AL	
F4155:	00 000 NULL	ADD [BX + SI], AL	
F4156:	00 000 NULL	ADD [BX + SI], AL	
F4157:	00 000 NULL	ADD [BX + SI], AL	
F4158:	00 000 NULL	ADD [BX + SI], AL	
F4159:	00 000 NULL	ADD [BX + SI], AL	
F415A:	00 000 NULL	ADD [BX + SI], AL	
F415B:	00 000 NULL	ADD [BX + SI], AL	
F415C:	00 000 NULL	ADD [BX + SI], AL	
F415D:	00 000 NULL	ADD [BX + SI], AL	
F415E:	00 000 NULL	ADD [BX + SI], AL	
F415F:	00 000 NULL	ADD [BX + SI], AL	
F4160:	FF 255 RES	ADD [BX + SI], AL	
F4161:	FF 255 RES	ADD [BX + SI], AL	
F4162:	CD 205 =	ADD [BX + SI], AL	
F4163:	1A 026 +	ADD [BX + SI], AL	
F4164:	CF 207 =	ADD [BX + SI], AL	
F4165:	00 000 NULL	ADD [BX + SI], AL	

Figure 151: Register Output for RCL Task

Question no 5

Task: Perform RCR AL, 1 on a value ending in bit 1. Will that bit rotate into CF and activate JC?

Rotate through Carry Right (RCR)

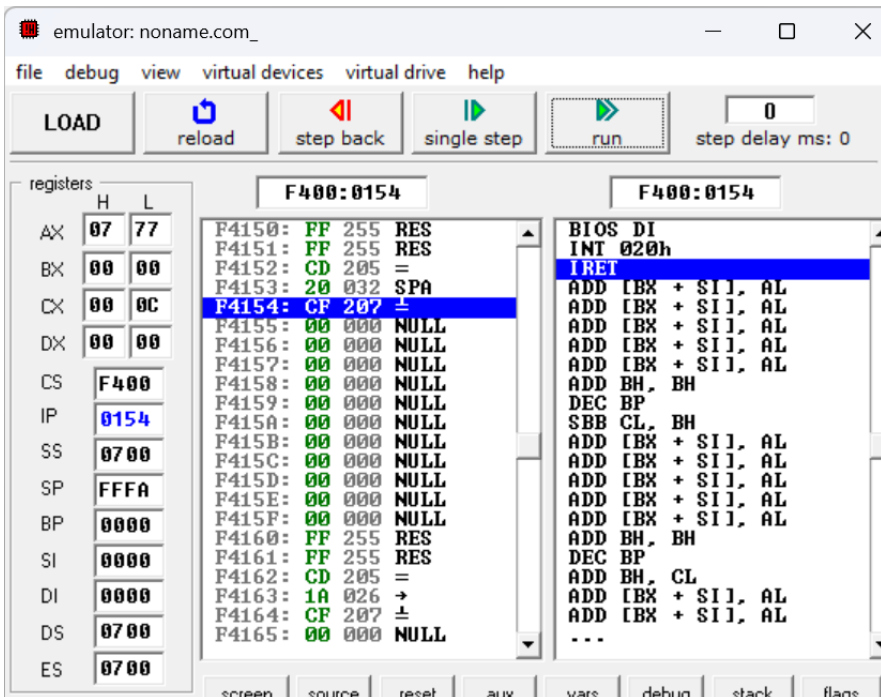


```

01 ORG 100h
02
03 MOV AL, 03h      ; Load 03h (Binary 0000 0011). Ends in 1.
04 RCR AL, 1        ; Rotate Right through Carry.
05                  ; LSB(1) moves to Carry Flag.
06
07 JC Bit_In_Carry  ; Jump if Carry Flag = 1
08 JMP Stop
09
10 Bit_In_Carry:
11 MOV AX, 0777h    ; Marker
12
13 Stop:
14 RET
  
```

Figure 152: Assembly Code for Rotate through Carry Right (RCR)

Output



emulator: noname.com_

file debug view virtual devices virtual drive help

LOAD reload step back single step run step delay ms: 0

registers	H	L
AX	07	77
BX	00	00
CX	00	0C
DX	00	00
CS	F400	
IP	0154	
SS	0700	
SP	FFFA	
BP	0000	
SI	0000	
DI	0000	
DS	0700	
ES	0700	

F400:0154

F4150:	FF	255	RES
F4151:	FF	255	RES
F4152:	CD	205	=
F4153:	20	032	SPA
F4154:	CF	207	±
F4155:	00	000	NULL
F4156:	00	000	NULL
F4157:	00	000	NULL
F4158:	00	000	NULL
F4159:	00	000	NULL
F415A:	00	000	NULL
F415B:	00	000	NULL
F415C:	00	000	NULL
F415D:	00	000	NULL
F415E:	00	000	NULL
F415F:	00	000	NULL
F4160:	FF	255	RES
F4161:	FF	255	RES
F4162:	CD	205	=
F4163:	1A	026	→
F4164:	CF	207	±
F4165:	00	000	NULL

F400:0154

BIOS DI
INT 020h
IRET
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD BH, BH
DEC BP
SBB CL, BH
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD IBX + SI, AL
ADD BH, BH
DEC BP
ADD BH, CL
ADD IBX + SI, AL
ADD IBX + SI, AL
...

screen source reset aux vars debug stack flags

Figure 153: Register Output for RCR Task

LAB 14

Subroutine

To understand and implement the concept of **Subroutines** (also called Procedures) in Assembly Language to write modular, reusable, and organized code.

A **Subroutine** is a block of code written separately from the main program to perform a specific task.

Instead of rewriting the same code multiple times, we "Call" the subroutine whenever needed.

- **CALL:** The instruction used to jump to the subroutine. It saves the return address on the Stack.
- **RET:** The instruction used at the end of a subroutine to return to the main program.
- **PROC / ENDP:** Directives used to define the start and end of a procedure.

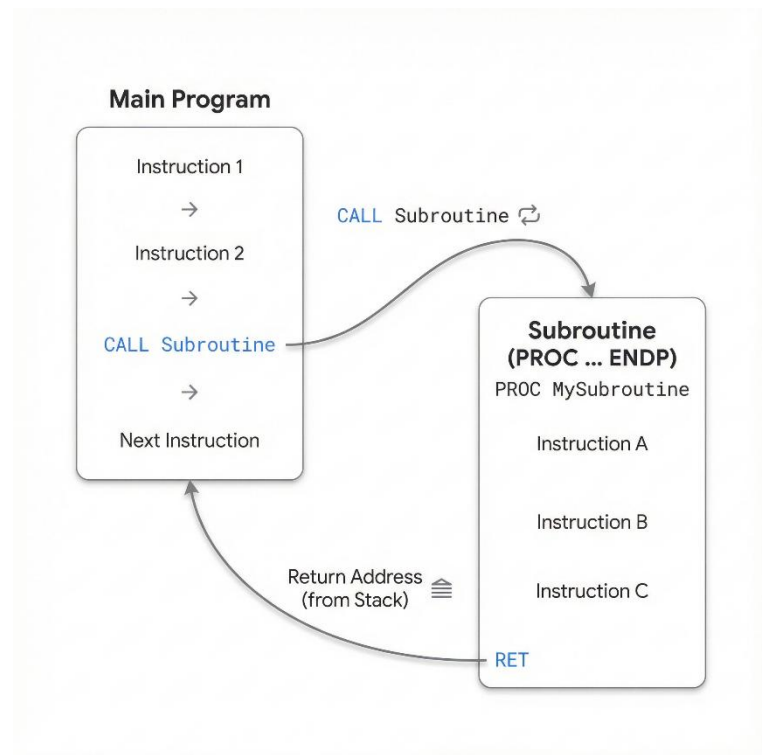


Figure 154: Block of Code

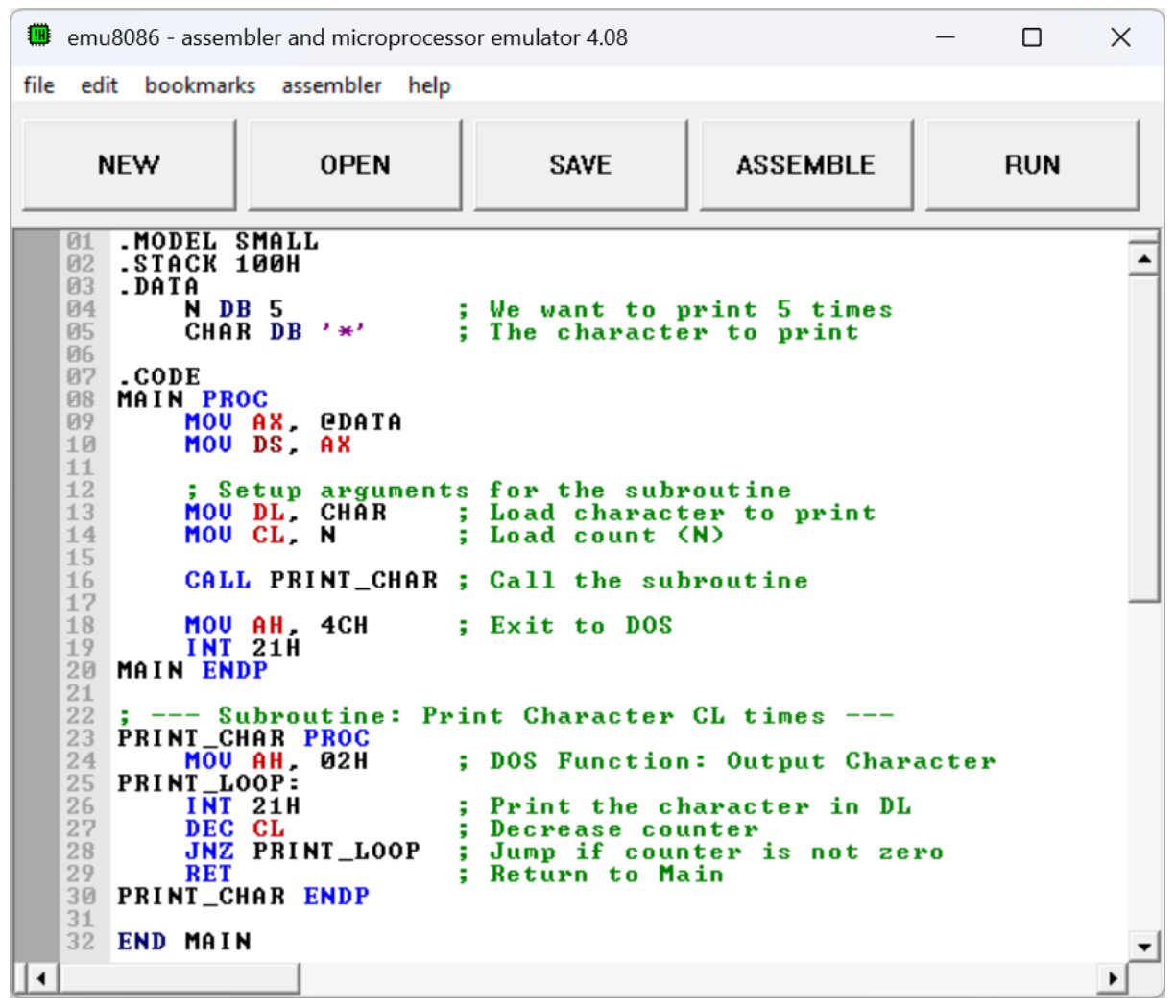
Why use Subroutines?

1. Reduces code size (less typing!).
2. Makes code easier to read and debug.
3. Allows code reusability.

Question no 1

Task: Write a program that defines a subroutine to print a specific character a specific number of times.

Print a Character N Times Using Subroutine



```

01 .MODEL SMALL
02 .STACK 100H
03 .DATA
04     N DB 5           ; We want to print 5 times
05     CHAR DB '*'      ; The character to print
06
07 .CODE
08 MAIN PROC
09     MOV AX, @DATA
10     MOV DS, AX
11
12     ; Setup arguments for the subroutine
13     MOV DL, CHAR      ; Load character to print
14     MOV CL, N          ; Load count (N)
15
16     CALL PRINT_CHAR    ; Call the subroutine
17
18     MOV AH, 4CH        ; Exit to DOS
19     INT 21H
20 MAIN ENDP
21
22 ; --- Subroutine: Print Character CL times ---
23 PRINT_CHAR PROC
24     MOV AH, 02H        ; DOS Function: Output Character
25 PRINT_LOOP:
26     INT 21H            ; Print the character in DL
27     DEC CL             ; Decrease counter
28     JNZ PRINT_LOOP     ; Jump if counter is not zero
29     RET                ; Return to Main
30 PRINT_CHAR ENDP
31
32 END MAIN
  
```

Figure 155: Assembly Code for Printing a Character N Times Using Subroutine

Output

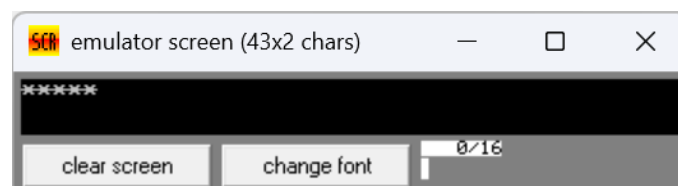
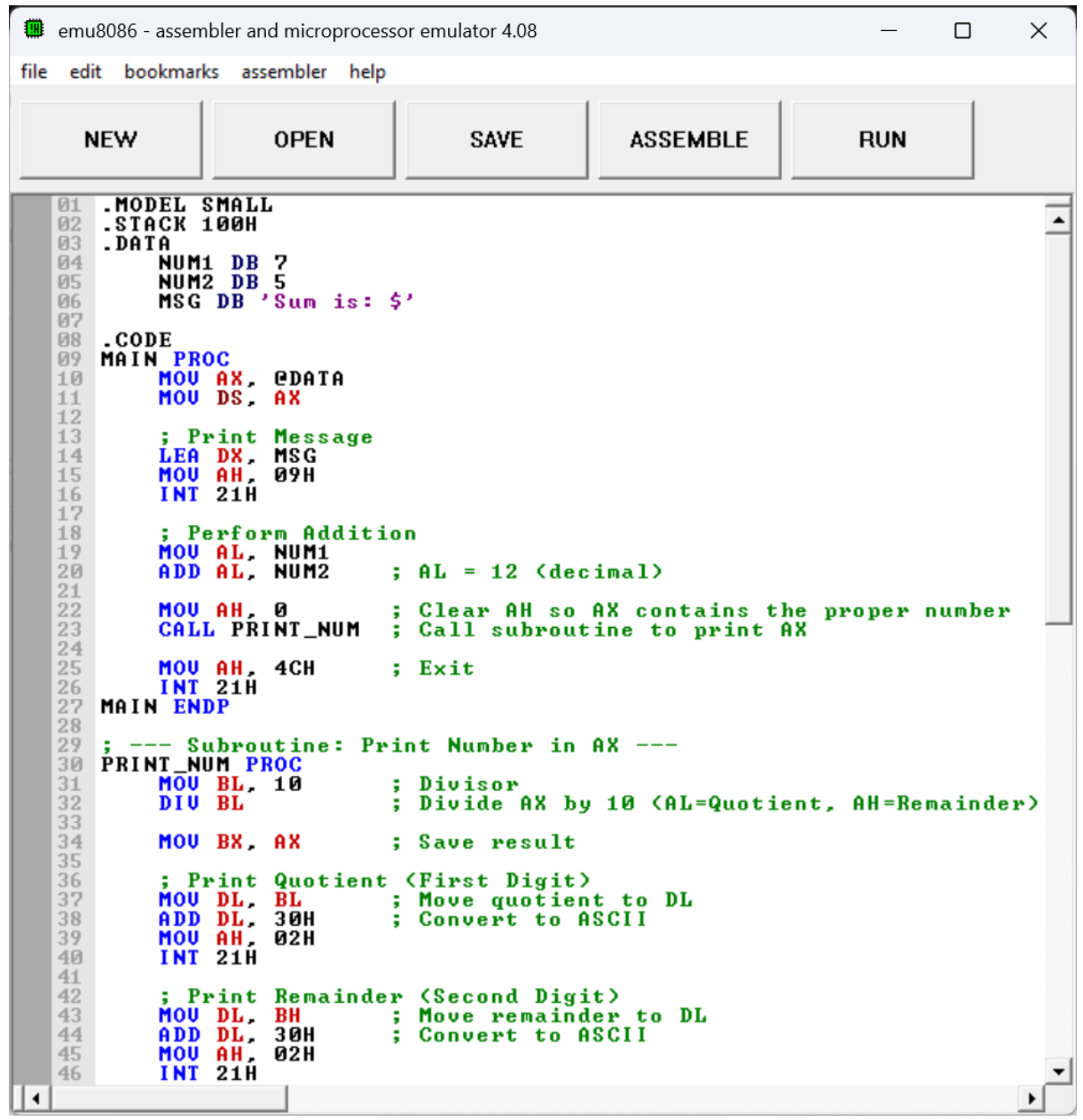


Figure 156: Output Screen for Character Printing Subroutine

Question no 2

Task: Add two single-digit numbers and display the result correctly (even if it is two digits, e.g., $5 + 7 = 12$).

Add Two Numbers Using Subroutine



```

01 .MODEL SMALL
02 .STACK 100H
03 .DATA
04     NUM1 DB 7
05     NUM2 DB 5
06     MSG DB 'Sum is: $'
07
08 .CODE
09 MAIN PROC
10     MOV AX, @DATA
11     MOV DS, AX
12
13     ; Print Message
14     LEA DX, MSG
15     MOV AH, 09H
16     INT 21H
17
18     ; Perform Addition
19     MOV AL, NUM1
20     ADD AL, NUM2      ; AL = 12 (decimal)
21
22     MOV AH, 0         ; Clear AH so AX contains the proper number
23     CALL PRINT_NUM    ; Call subroutine to print AX
24
25     MOV AH, 4CH       ; Exit
26     INT 21H
27 MAIN ENDP
28
29 ; --- Subroutine: Print Number in AX ---
30 PRINT_NUM PROC
31     MOV BL, 10        ; Divisor
32     DIV BL            ; Divide AX by 10 (AL=Quotient, AH=Remainder)
33
34     MOV BX, AX        ; Save result
35
36     ; Print Quotient (First Digit)
37     MOV DL, BL        ; Move quotient to DL
38     ADD DL, 30H       ; Convert to ASCII
39     MOV AH, 02H
40     INT 21H
41
42     ; Print Remainder (Second Digit)
43     MOV DL, BH        ; Move remainder to DL
44     ADD DL, 30H       ; Convert to ASCII
45     MOV AH, 02H
46     INT 21H

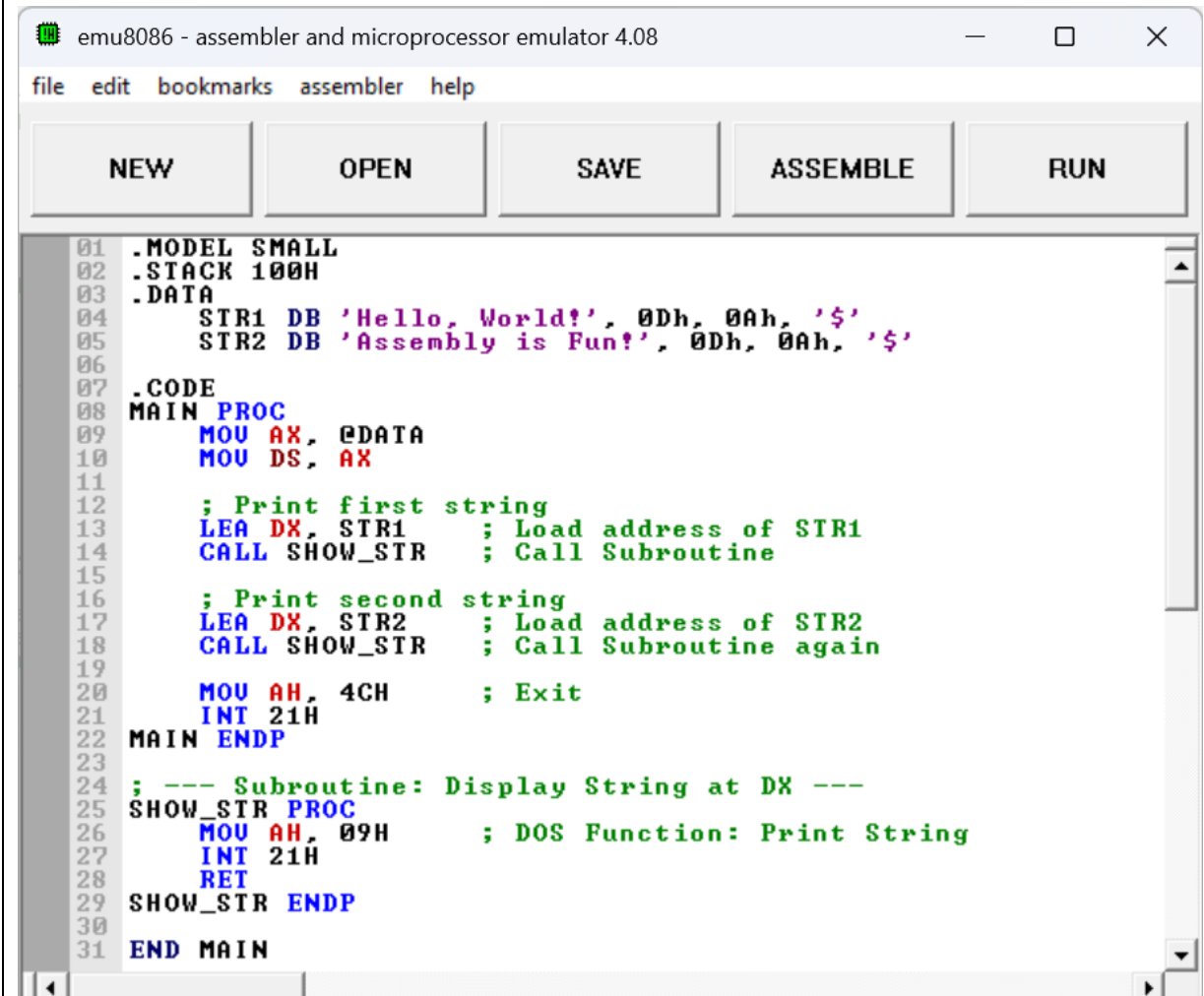
```

Figure 157: Assembly Code for Adding Two Numbers Using Subroutine

Question no 3

Task: Create a reusable subroutine that can take any string address and print it to the screen.

Display a String Using Subroutine

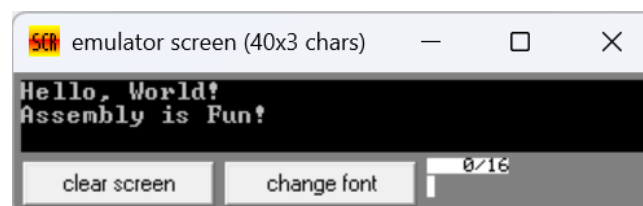


```

01 .MODEL SMALL
02 .STACK 100H
03 .DATA
04     STR1 DB 'Hello, World!', 0Dh, 0Ah, '$'
05     STR2 DB 'Assembly is Fun!', 0Dh, 0Ah, '$'
06
07 .CODE
08 MAIN PROC
09     MOV AX, @DATA
10     MOV DS, AX
11
12     ; Print first string
13     LEA DX, STR1      ; Load address of STR1
14     CALL SHOW_STR     ; Call Subroutine
15
16     ; Print second string
17     LEA DX, STR2      ; Load address of STR2
18     CALL SHOW_STR     ; Call Subroutine again
19
20     MOV AH, 4CH        ; Exit
21     INT 21H
22 MAIN ENDP
23
24 ; --- Subroutine: Display String at DX ---
25 SHOW_STR PROC
26     MOV AH, 09H        ; DOS Function: Print String
27     INT 21H
28     RET
29 SHOW_STR ENDP
30
31 END MAIN
  
```

Figure 158: Assembly Code for Displaying a String Using Subroutine

Output



```

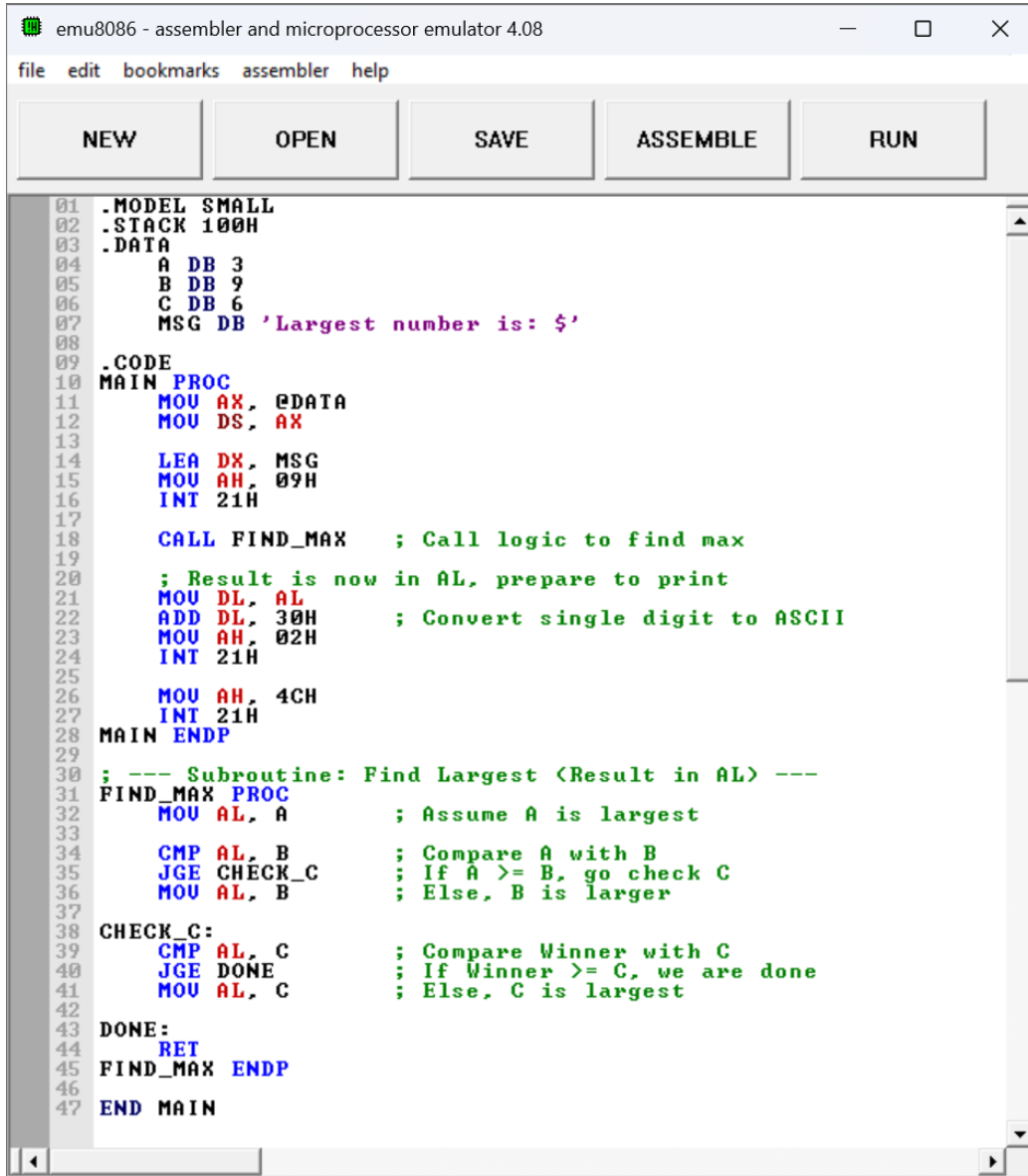
SCA emulator screen (40x3 chars)
Hello, World!
Assembly is Fun!
clear screen  change font  0/16
  
```

Figure 159: Output Screen for String Display Subroutine

Question no 4

Task: Find the maximum value among three stored numbers and print it.

Largest of Three Numbers

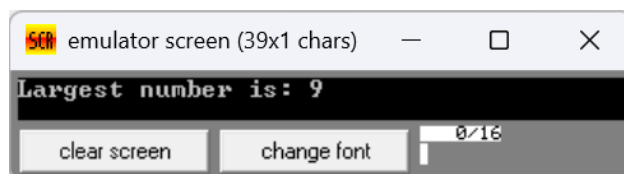


```

01 .MODEL SMALL
02 .STACK 100H
03 .DATA
04     A DB 3
05     B DB 9
06     C DB 6
07     MSG DB 'Largest number is: $'
08
09 .CODE
10 MAIN PROC
11     MOV AX, @DATA
12     MOV DS, AX
13
14     LEA DX, MSG
15     MOV AH, 09H
16     INT 21H
17
18     CALL FIND_MAX    ; Call logic to find max
19
20     ; Result is now in AL, prepare to print
21     MOV DL, AL
22     ADD DL, 30H      ; Convert single digit to ASCII
23     MOV AH, 02H
24     INT 21H
25
26     MOV AH, 4CH
27     INT 21H
28 MAIN ENDP
29
30 ; --- Subroutine: Find Largest (Result in AL) ---
31 FIND_MAX PROC
32     MOV AL, A        ; Assume A is largest
33
34     CMP AL, B        ; Compare A with B
35     JGE CHECK_C      ; If A >= B, go check C
36     MOV AL, B        ; Else, B is larger
37
38 CHECK_C:
39     CMP AL, C        ; Compare Winner with C
40     JGE DONE         ; If Winner >= C, we are done
41     MOV AL, C        ; Else, C is largest
42
43 DONE:
44     RET
45 FIND_MAX ENDP
46
47 END MAIN
  
```

Figure 160: Assembly Code for Finding Largest of Three Numbers

Output



```

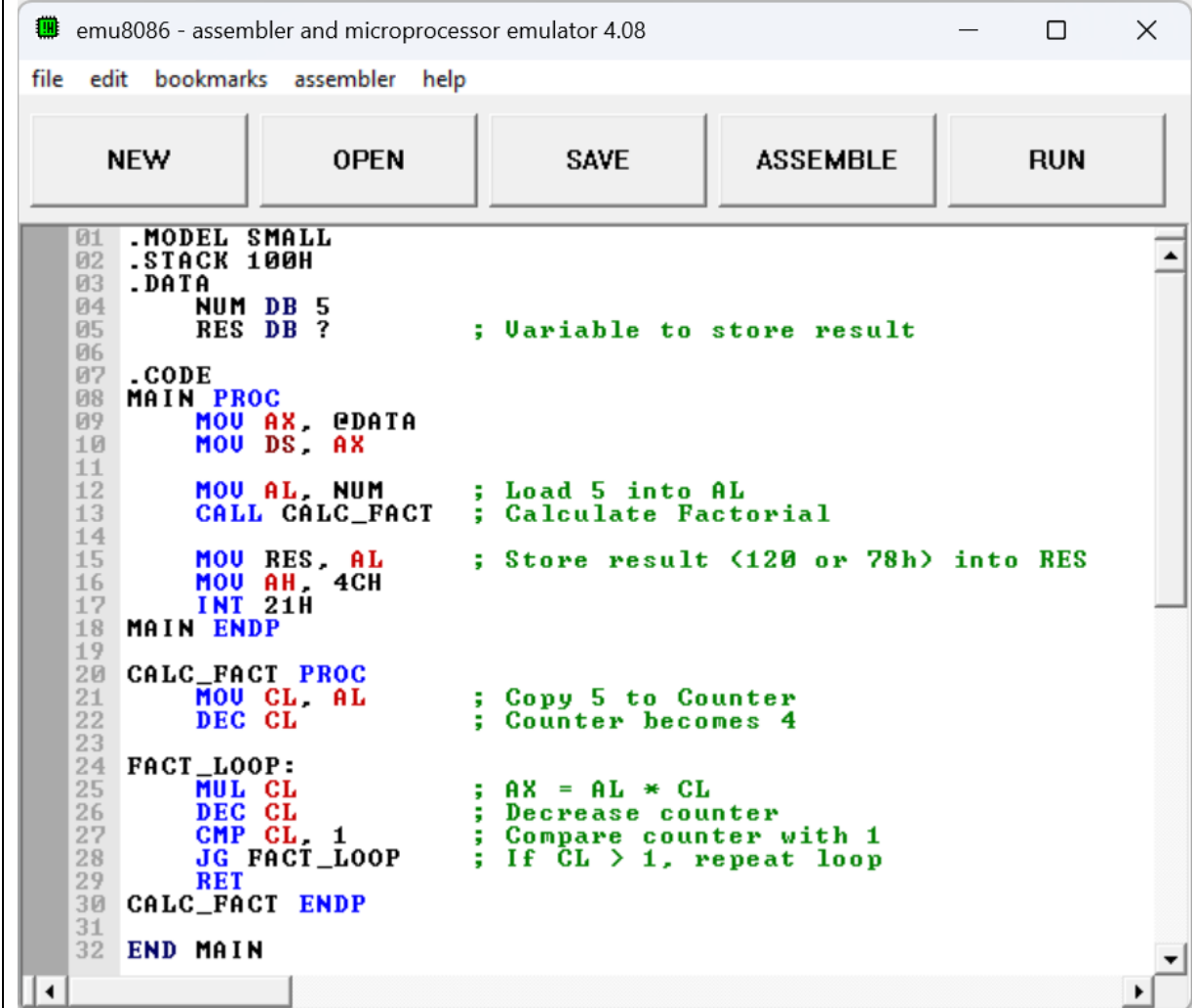
emulator screen (39x1 chars)
Largest number is: 9
clear screen  change font  0/16
  
```

Figure 161: Output Screen for Maximum Value Program

Question no 5

Task: Calculate 5! ($5 \times 4 \times 3 \times 2 \times 1 = 120$) using a subroutine.

Factorial of 5



```

01 .MODEL SMALL
02 .STACK 100H
03 .DATA
04     NUM DB 5
05     RES DB ?           ; Variable to store result
06
07 .CODE
08 MAIN PROC
09     MOV AX, @DATA
10     MOV DS, AX
11
12     MOV AL, NUM         ; Load 5 into AL
13     CALL CALC_FACT      ; Calculate Factorial
14
15     MOV RES, AL         ; Store result (120 or 78h) into RES
16     MOV AH, 4CH
17     INT 21H
18 MAIN ENDP
19
20 CALC_FACT PROC
21     MOV CL, AL          ; Copy 5 to Counter
22     DEC CL              ; Counter becomes 4
23
24 FACT_LOOP:
25     MUL CL              ; AX = AL * CL
26     DEC CL              ; Decrease counter
27     CMP CL, 1           ; Compare counter with 1
28     JG FACT_LOOP        ; If CL > 1, repeat loop
29     RET
30 CALC_FACT ENDP
31
32 END MAIN
  
```

Figure 162: Assembly Code for Factorial Calculation Using Subroutine

Output

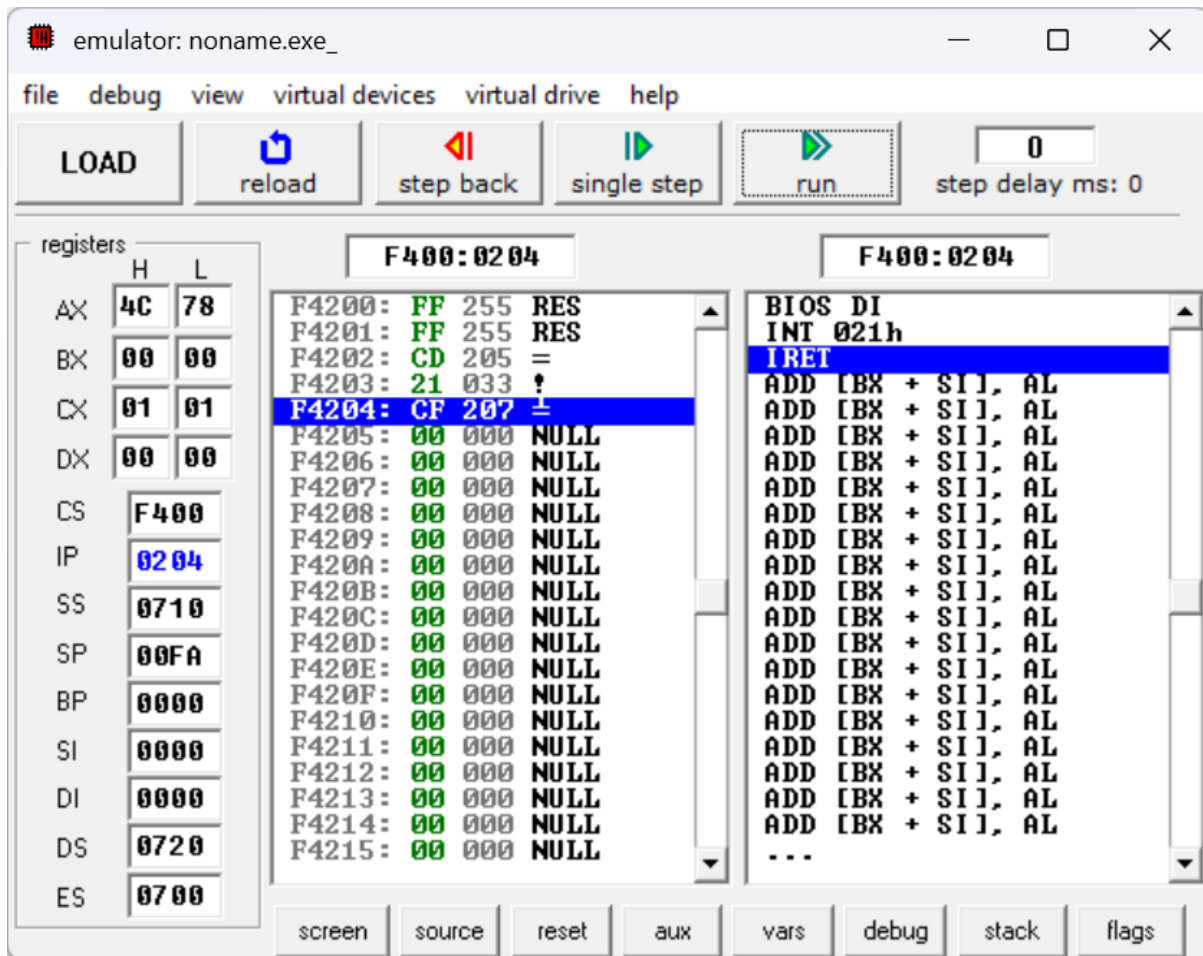


Figure 163: Output Screen for Factorial Subroutine

LAB 15

Subroutine

The Stack Architecture In 8086 assembly, the stack is a reserved area of memory used to temporarily store data, return addresses, and register values in a Last-In, First-Out (LIFO) manner. The Stack Pointer (SP) register always points to the top of the stack.

PUSH Instruction: The PUSH instruction places data onto the stack. Since the stack grows downward in memory (towards lower addresses), the SP is decremented by 2 before the data is stored.

- Syntax: **PUSH** source (e.g., **PUSH AX**)
- Operation: $SP = SP - 2$
 $\rightarrow [SP] = \text{Data}$

POP Instruction: The POP instruction retrieves data from the top of the stack and places it into a destination register.

After retrieval, the **SP** is incremented by 2.

- Syntax: **POP** destination
(e.g., **POP BX**)

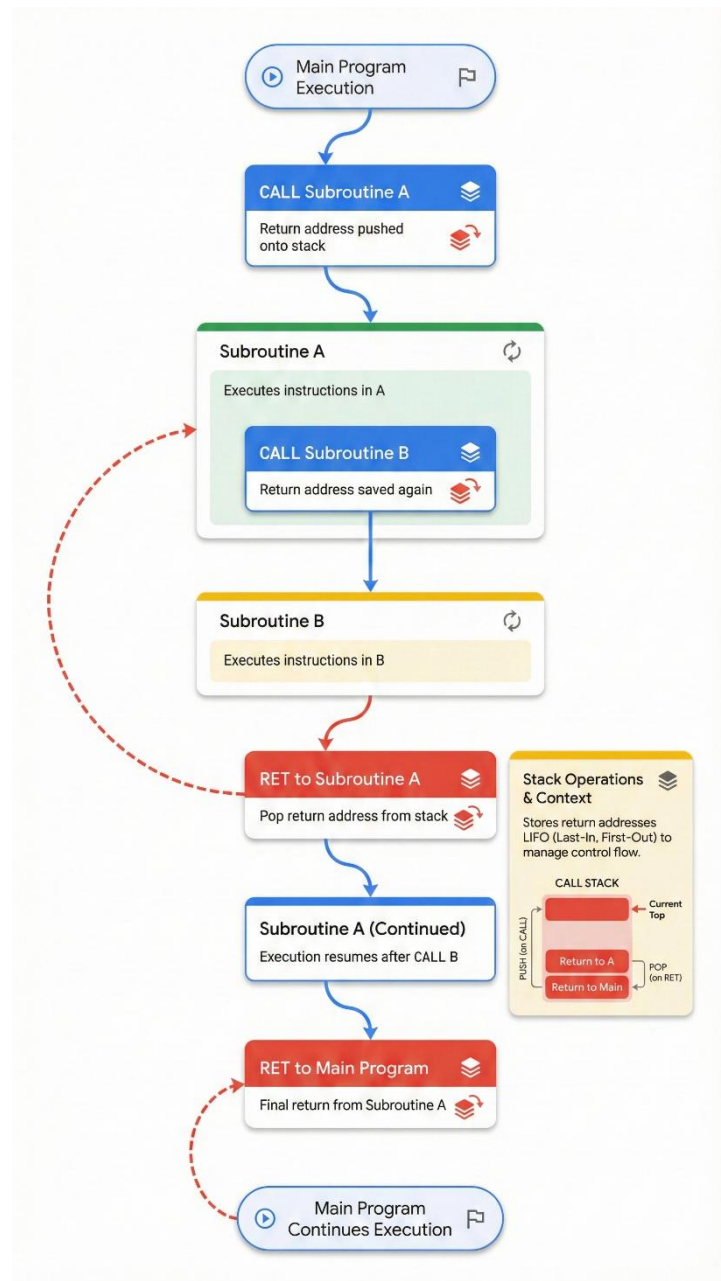


Figure 164: Flow

- Operation: **Data** = [SP] $\rightarrow$ SP = SP + 2

Nested Subroutines: A nested subroutine occurs when the main program calls a subroutine (e.g., Subroutine A), and that subroutine subsequently calls another (e.g., Subroutine B).

- The processor uses the stack to keep track of return addresses.
- When CALL is executed, the address of the next instruction is pushed onto the stack.
- When RET is executed, the address is popped back into the Instruction Pointer (IP) to resume execution.